

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Xây dựng nền tảng chia sẻ và lưu trữ tác phẩm nghệ thuật số tích hợp công nghệ tìm kiếm đa phương thức

HOÀNG CHÍ THANH

thanh.hc225929@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: TS. ĐỖ QUỐC HUY

Khoa: Khoa học máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Xây dựng nền tảng chia sẻ và lưu trữ tác phẩm nghệ thuật số tích hợp công nghệ tìm kiếm đa phương thức

HOÀNG CHÍ THANH

thanh.hc225929@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: TS. ĐỖ QUỐC HUY

Chữ kí GVHD

Khoa: Khoa học máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 06/2026

LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành đến giảng viên hướng dẫn cùng các thầy cô thuộc Viện Công nghệ Thông tin và Truyền thông, Trường Đại học Bách khoa Hà Nội đã định hướng và trang bị những kiến thức chuyên môn quý báu trong suốt quá trình học tập.

Đồng thời, em xin bày tỏ lòng biết ơn sâu sắc đến gia đình và bạn bè đã luôn là chỗ dựa tinh thần, động viên và hỗ trợ mọi mặt để em yên tâm hoàn thành đồ án này.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Sự bùng nổ của nghệ thuật số kéo theo nhu cầu lớn về các nền tảng chia sẻ ảnh trực tuyến chuyên biệt. Tuy nhiên, các hệ thống hiện tại thường gặp khó khăn trong việc kiểm duyệt nội dung độc hại trên quy mô lớn, ngăn chặn vấn nạn sao chép tác phẩm, và cung cấp công cụ tìm kiếm ngữ nghĩa thông minh thay vì chỉ dựa vào thẻ băm truyền thống. Bên cạnh đó, việc xử lý các tác vụ tính toán nặng ngay trên luồng yêu cầu chính dễ gây nghẽn hệ thống và làm giảm trải nghiệm người dùng. Để giải quyết các hạn chế này, đồ án lựa chọn hướng tiếp cận xây dựng hệ thống theo kiến trúc module hóa kết hợp với cơ chế xử lý bất đồng bộ qua hàng đợi nền nhằm tối ưu hiệu năng và đảm bảo khả năng mở rộng linh hoạt. Trên cơ sở đó, tác giả đã tiến hành xây dựng nền tảng chia sẻ và lưu trữ tác phẩm nghệ thuật số tích hợp công nghệ tìm kiếm đa phương thức sử dụng Next.js cho phía giao diện và NestJS cho phía dịch vụ, lưu trữ dữ liệu trên PostgreSQL và tối ưu tìm kiếm toàn văn bằng Meilisearch. Hệ thống tích hợp Clerk để xác thực người dùng và Stripe để xử lý mô hình thành viên trả phí cho nhà sáng tạo. Điểm nhấn công nghệ của giải pháp là việc ứng dụng mô hình CLIP sinh vector embedding phục vụ tìm kiếm ngữ nghĩa qua pgvector, sử dụng thuật toán Perceptual Hashing (pHash) phát hiện ảnh trùng lặp, và tích hợp NSFWJS tự động phân loại nội dung nhạy cảm. Toàn bộ các tiến trình nặng này được điều phối bất đồng bộ bởi BullMQ và Redis để giải phóng luồng xử lý chính. Đóng góp chính của đồ án là đề xuất và hiện thực hóa thành công một kiến trúc phân tán thực tế, giải quyết hài hòa giữa trải nghiệm người dùng, quyền lợi kinh tế của nhà sáng tạo và hiệu quả quản trị tự động. Kết quả sau cùng đạt được là một khung hệ thống hoàn chỉnh, vận hành ổn định với độ trễ thấp và đáp ứng tốt các yêu cầu vận hành thực tế.

Sinh viên thực hiện

(Ký và ghi rõ họ tên)

Hoàng Chí Thanh

ABSTRACT

The boom of digital art has driven a massive demand for specialized online image-sharing platforms. However, existing systems often struggle to moderate harmful content at scale, prevent artwork plagiarism, and provide intelligent semantic search instead of relying solely on traditional hashtags. Furthermore, executing computationally heavy tasks directly on the main request path can easily cause system bottlenecks and degrade user experience. To address these limitations, this thesis adopts an approach that constructs the system based on a modular architecture combined with an asynchronous background queue mechanism to optimize performance and ensure flexible scalability. On this basis, a platform for sharing and storing digital artworks integrating multimodal search technology has been developed using Next.js for the frontend and NestJS for backend services, utilizing PostgreSQL as the relational database and Meilisearch for optimized full-text search. The system integrates Clerk for authentication and Stripe to handle paid membership models for creators. The technological highlight of this solution lies in applying the CLIP model to generate vector embeddings for semantic search via pgvector, utilizing the Perceptual Hashing (pHash) algorithm for duplicate image detection, and integrating NSFWJS for automated sensitive content classification. All of these intensive processes are coordinated asynchronously by BullMQ and Redis to free up the main execution path. The main contribution of this thesis is the proposal and successful implementation of a practical distributed architecture that harmoniously balances user experience, financial rights for creators, and automated administration efficiency. The final outcome is a complete system framework that operates stably with low latency and effectively meets real-world operational requirements.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	2
1.3 Định hướng giải pháp.....	4
1.4 Bố cục đồ án	5
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU.....	7
2.1 Khảo sát hiện trạng	7
2.1.1 Khảo sát hệ thống hiện có	7
2.1.2 Các chức năng cần có cho hệ thống.....	8
2.2 Tổng quan chức năng	9
2.2.1 Biểu đồ use case tổng quát	9
2.2.2 Biểu đồ use case phân rã Quản lý tài khoản	10
2.2.3 Biểu đồ use case phân rã Quản lý tác phẩm	11
2.2.4 Biểu đồ use case phân rã Tìm kiếm và khám phá nội dung.....	12
2.2.5 Biểu đồ use case phân rã Tương tác cộng đồng	12
2.2.6 Biểu đồ use case phân rã Quản lý thành viên và thanh toán	13
2.2.7 Biểu đồ use case phân rã Kiểm duyệt và quản trị.....	14
2.2.8 Quy trình nghiệp vụ	15
2.3 Đặc tả chức năng	20
2.3.1 Đặc tả use case Đăng nhập và đồng bộ tài khoản.....	20
2.3.2 Đặc tả use case Đăng tải tác phẩm	20
2.3.3 Đặc tả use case Tìm kiếm và khám phá tác phẩm	21
2.3.4 Đặc tả use case Tương tác với tác phẩm	22
2.3.5 Đặc tả use case Đăng ký gói thành viên của người sáng tạo.....	23
2.3.6 Đặc tả use case Kiểm duyệt báo cáo và Khiếu nại	24
2.4 Yêu cầu phi chức năng	25
CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG	28
3.1 Công nghệ phát triển ứng dụng Frontend và Backend.....	28
3.1.1 Kiến trúc Backend với NestJS	28

3.1.2 Framework Frontend Next.js và Thư viện UI Mantine	28
3.2 Cơ sở dữ liệu và Quản lý trạng thái dữ liệu.....	29
3.2.1 Hệ quản trị cơ sở dữ liệu PostgreSQL.....	29
3.2.2 Công cụ ánh xạ dữ liệu Prisma ORM	29
3.3 Nền tảng Lý thuyết và Công nghệ Tìm kiếm.....	30
3.3.1 Công cụ tìm kiếm toàn văn Meilisearch.....	30
3.3.2 Lý thuyết học biểu diễn và Tìm kiếm ngữ nghĩa với pgvector	30
3.4 Xử lý nền và hàng đợi	31
3.4.1 Hàng đợi tin nhắn BullMQ và Bộ nhớ đệm Redis.....	31
3.5 Nền tảng Lý thuyết cho Kiểm duyệt và Xử lý Ảnh.....	31
3.5.1 Lý thuyết Băm nhận thức trong phát hiện trùng lặp	31
3.5.2 Phân loại nội dung nhạy cảm bằng NSFWJS	32
3.6 Các Dịch vụ Tích hợp Bên ngoài.....	32
3.6.1 Quản lý danh tính và Xác thực với Clerk	32
3.6.2 Cổng thanh toán Stripe	33
CHƯƠNG 4. THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG	34
4.1 Thiết kế kiến trúc hệ thống	34
4.1.1 Lựa chọn kiến trúc phần mềm	34
4.1.2 Thiết kế tổng quan.....	34
4.1.3 Thiết kế chi tiết gói	36
4.2 Thiết kế chi tiết.....	41
4.2.1 Thiết kế giao diện	41
4.2.2 Thiết kế lớp	44
4.2.3 Thiết kế cơ sở dữ liệu	49
4.3 Xây dựng ứng dụng.....	52
4.3.1 Thư viện và công cụ sử dụng.....	52
4.3.2 Kết quả đạt được	53
4.3.3 Minh họa các chức năng chính	54
4.4 Kiểm thử.....	57
4.4.1 Kịch bản kiểm thử: Đăng tải tác phẩm.....	57
4.4.2 Kịch bản kiểm thử: Xác thực và Đồng bộ tài khoản	58
4.4.3 Kịch bản kiểm thử: Tìm kiếm và Lọc tác phẩm	58

4.4.4 Tổng kết kết quả kiểm thử	59
4.5 Triển khai hệ thống	60
4.5.1 Mô hình triển khai tổng thể	60
4.5.2 Cấu trúc dịch vụ và cấu hình	60
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT.....	62
5.1 Kiến trúc nguyên khối phân rã kết hợp hướng sự kiện	62
5.1.1 Đặt vấn đề	62
5.1.2 Giải pháp	62
5.1.3 Kết quả đạt được	63
5.2 Hệ thống tìm kiếm ngữ nghĩa đa phương thức	63
5.2.1 Đặt vấn đề	63
5.2.2 Giải pháp	64
5.2.3 Kết quả đạt được	64
5.3 Bảo vệ bản quyền nghệ thuật bằng thuật toán băm nhận thức pHash	65
5.3.1 Đặt vấn đề	65
5.3.2 Giải pháp	65
5.3.3 Kết quả đạt được	66
5.4 Tối ưu hiệu năng bằng cơ chế xử lý nền bất đồng bộ.....	66
5.4.1 Đặt vấn đề	66
5.4.2 Giải pháp	66
5.4.3 Kết quả đạt được	67
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	68
6.1 Kết luận	68
6.1.1 Kết quả đạt được và so sánh với các sản phẩm tương tự.....	68
6.1.2 Các hạn chế còn tồn tại	69
6.1.3 Bài học kinh nghiệm	70
6.2 Hướng phát triển.....	70
6.2.1 Hoàn thiện các chức năng hiện có	70
6.2.2 Các hướng phát triển mới.....	71
TÀI LIỆU THAM KHẢO.....	72

DANH MỤC HÌNH VẼ

Hình 2.1	Biểu đồ use case tổng quát của hệ thống	10
Hình 2.2	Biểu đồ use case phân rã Quản lý tài khoản	11
Hình 2.3	Biểu đồ use case phân rã Quản lý tác phẩm	11
Hình 2.4	Biểu đồ use case phân rã Tìm kiếm và khám phá nội dung . .	12
Hình 2.5	Biểu đồ use case phân rã Tương tác cộng đồng	13
Hình 2.6	Biểu đồ use case phân rã Quản lý thành viên và thanh toán . .	14
Hình 2.7	Biểu đồ use case phân rã Kiểm duyệt và quản trị	15
Hình 2.8	Quy trình nghiệp vụ đăng tải và xử lý tác phẩm	16
Hình 2.9	Quy trình nghiệp vụ đăng ký gói thành viên của người sáng tạo	18
Hình 4.1	Biểu đồ phụ thuộc mô-đun tổng quan của hệ thống	35
Hình 4.2	Thiết kế chi tiết nhóm Định danh và Quản lý Người dùng . . .	37
Hình 4.3	Thiết kế chi tiết nhóm Lỗi Nội dung và Xử lý Ảnh	38
Hình 4.4	Thiết kế chi tiết nhóm Tương tác Xã hội và Cảnh báo	39
Hình 4.5	Thiết kế chi tiết nhóm Xử lý Dữ liệu Thông minh	40
Hình 4.6	Thiết kế chi tiết nhóm Quản trị, Vận hành và Doanh thu	41
Hình 4.7	Bảng màu sử dụng cho nền	43
Hình 4.8	Bảng màu chủ đạo (Primary Palette)	43
Hình 4.9	Thiết kế giao diện chức năng Khám phá (Trang chủ)	44
Hình 4.10	Thiết kế giao diện chức năng Chi tiết tác phẩm	44
Hình 4.11	Biểu đồ các lớp chủ đạo trong hệ thống	45
Hình 4.12	Biểu đồ trình tự Use Case: Đăng tải tác phẩm nghệ thuật . . .	47
Hình 4.13	Biểu đồ trình tự Use Case: Tìm kiếm bằng AI	48
Hình 4.14	Biểu đồ trình tự Use Case: Đăng ký gói hội viên	49
Hình 4.15	Biểu đồ Thực thể - Liên kết (ERD) cốt lõi	50
Hình 4.16	Giao diện chức năng Khám phá (Trang chủ)	54
Hình 4.17	Giao diện chức năng Đăng tải tác phẩm	55
Hình 4.18	Giao diện chức năng Chi tiết tác phẩm	56
Hình 4.19	Giao diện chức năng Nâng cấp gói hội viên	56

DANH MỤC BẢNG BIỂU

Bảng 2.1	Tóm tắt các tính năng cốt lõi cần thiết cho hệ thống đề xuất . .	9
Bảng 2.2	Đặc tả use case Đăng nhập và đồng bộ tài khoản	20
Bảng 2.3	Đặc tả use case Đăng tải tác phẩm	21
Bảng 2.4	Đặc tả use case Tìm kiếm và khám phá tác phẩm	22
Bảng 2.5	Đặc tả use case Tương tác với tác phẩm	23
Bảng 2.6	Đặc tả use case Đăng ký gói thành viên của người sáng tạo . .	24
Bảng 2.7	Đặc tả use case Kiểm duyệt báo cáo và Khiếu nại	25
Bảng 4.1	Đặc tả cấu trúc bảng User (Quản lý người dùng)	51
Bảng 4.2	Đặc tả cấu trúc bảng Artwork (Bài đăng tác phẩm)	51
Bảng 4.3	Đặc tả cấu trúc bảng ArtworkImage (Tập dữ liệu hình ảnh) . .	52
Bảng 4.4	Đặc tả cấu trúc bảng Payment (Giao dịch tài chính)	52
Bảng 4.5	Danh sách thư viện và công cụ sử dụng	53
Bảng 4.6	Thống kê số liệu kỹ thuật của hệ thống	54
Bảng 4.7	Kịch bản kiểm thử chức năng Đăng tải tác phẩm	57
Bảng 4.8	Kịch bản kiểm thử chức năng Xác thực tài khoản	58
Bảng 4.9	Kịch bản kiểm thử chức năng Tìm kiếm tác phẩm	59
Bảng 4.10	Tổng hợp kết quả thực thi các trường hợp kiểm thử	59

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Viết tắt	Tên tiếng Anh	Tên tiếng Việt
AI	Artificial Intelligence	Trí tuệ nhân tạo
API	Application Programming Interface	Giao diện lập trình ứng dụng
AWS	Amazon Web Services	Dịch vụ điện toán đám mây của Amazon
CDN	Content Delivery Network	Mạng phân phối nội dung
CI/CD	Continuous Integration / Continuous Deployment	Tích hợp liên tục / Triển khai liên tục
CLIP	Contrastive Language-Image Pretraining	Mô hình huấn luyện đối chiếu ngôn ngữ-hình ảnh
CNN	Convolutional Neural Networks	Mạng nơ-ron tích chập
CSR	Client-Side Rendering	Kết xuất phía máy khách
DCT	Discrete Cosine Transform	Biến đổi Cosine rời rạc
DDD	Domain-Driven Design	Thiết kế hướng tên miền
DMCA	Digital Millennium Copyright Act	Đạo luật bản quyền kỹ thuật số thiên niên kỷ
Embedding		Vector số biểu diễn ngữ nghĩa hoặc đặc trưng thị giác của dữ liệu
ERD	Entity-Relationship Diagram	Biểu đồ Thực thể - Liên kết
HNSW	Hierarchical Navigable Small World	Thuật toán duyệt đồ thị phân cấp
HTTP	Hypertext Transfer Protocol	Giao thức truyền tải siêu văn bản
JSON	JavaScript Object Notation	Định dạng dữ liệu JavaScript
JWT	JSON Web Token	Mã thông báo web JSON
KNN	K-Nearest Neighbors	K láng giềng gần nhất
NSFW	Not Safe For Work	Nội dung không an toàn/nhạy cảm
ORM	Object-Relational Mapping	Ánh xạ quan hệ đối tượng
pHash	Perceptual Hashing	Băm nhận thức
RESTful	Representational State Transfer	Chuyển giao trạng thái đại diện

Viết tắt	Tên tiếng Anh	Tên tiếng Việt
SEO	Search Engine Optimization	Tối ưu hóa công cụ tìm kiếm
SQL	Structured Query Language	Ngôn ngữ truy vấn mang tính cấu trúc
SSO	Single Sign-On	Đăng nhập một lần
SSR	Server-Side Rendering	Kết xuất phía máy chủ
Tier		Gói thành viên giới hạn quyền truy cập
UI	User Interface	Giao diện người dùng
UUID	Universally Unique Identifier	Định danh duy nhất toàn cầu
VPS	Virtual Private Server	Máy chủ ảo dùng riêng
Worker		Tiến trình xử lý nền bất đồng bộ

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Trong những năm gần đây, cùng với sự phát triển của thiết bị di động, mạng xã hội và các công cụ sáng tạo số, nhu cầu đăng tải, lưu trữ, chia sẻ và khai thác nội dung hình ảnh tăng lên rất nhanh. Ảnh không còn chỉ đóng vai trò minh họa cho văn bản, mà đã trở thành một loại nội dung trung tâm trong giao tiếp số, quảng bá cá nhân, xây dựng cộng đồng và phát triển kinh tế sáng tạo. Đối với các lĩnh vực như minh họa số, truyện tranh, thiết kế nhân vật, concept art, nhiếp ảnh, ảnh tạo bởi trí tuệ nhân tạo và các cộng đồng nghệ thuật trực tuyến, một nền tảng chia sẻ ảnh không chỉ là nơi lưu trữ tệp, mà còn là môi trường để người sáng tạo công bố tác phẩm, người xem khám phá nội dung, cộng đồng tương tác và các bên quản trị duy trì trật tự vận hành.

Sự phổ biến của các cộng đồng chia sẻ ảnh đặt ra một bài toán có quy mô lớn hơn so với các hệ thống quản lý ảnh truyền thống. Mỗi tác phẩm thường bao gồm nhiều thành phần: tệp ảnh gốc, phiên bản xem trước, ảnh thu nhỏ, mô tả, thẻ gắn kèm, thông tin tác giả, trạng thái xuất bản, quyền truy cập, bình luận, lượt thích, bộ sưu tập và báo cáo vi phạm. Khi số lượng người dùng và số lượng ảnh tăng, hệ thống phải xử lý đồng thời nhiều yêu cầu khác nhau: phục vụ hiển thị ảnh nhanh, kiểm soát nội dung không phù hợp, hỗ trợ tìm kiếm chính xác, bảo vệ quyền lợi của người sáng tạo và tạo ra cơ chế tương tác bền vững giữa tác giả với người theo dõi.

Một vấn đề cấp thiết trong các nền tảng chia sẻ ảnh là kiểm duyệt nội dung nhạy cảm hoặc độc hại. Với đặc thù cho phép người dùng tự do đăng tải, hệ thống có thể phải tiếp nhận các ảnh không phù hợp với chuẩn cộng đồng, vi phạm chính sách nội dung hoặc gây ảnh hưởng tiêu cực tới người xem. Nếu toàn bộ quá trình kiểm duyệt phụ thuộc vào con người, đội ngũ điều phối phải mở và đánh giá thủ công số lượng lớn ảnh, trong đó có cả nội dung nhạy cảm. Cách làm này tốn thời gian, khó mở rộng, tạo áp lực cho người kiểm duyệt và làm tăng độ trễ xuất bản tác phẩm. Ngược lại, nếu thiếu cơ chế kiểm soát, nền tảng có nguy cơ mất an toàn cộng đồng, giảm chất lượng nội dung và ảnh hưởng tới uy tín của người sáng tạo nghiêm túc.

Một vấn đề khác là phát hiện ảnh trùng lặp hoặc ảnh bị tái đăng trái phép. Trong môi trường nghệ thuật số, một tác phẩm có thể bị tải lại sau khi đã đối kích thước, nén lại, cắt nhẹ hoặc thay đổi định dạng. Việc chỉ so sánh tên tệp, kích thước tệp hoặc mã băm nhị phân không đủ để phát hiện các trường hợp gần giống nhau về mặt thị giác. Nếu bài toán này không được xử lý, nền tảng dễ bị lạm dụng nội dung, kết

quả tìm kiếm bị nhiễu, quyền lợi của tác giả bị ảnh hưởng và công việc kiểm duyệt bản quyền trở nên khó khăn hơn. Đây là vấn đề có ý nghĩa thực tế lớn đối với các cộng đồng sáng tạo, nơi tính nguyên bản của tác phẩm là yếu tố quan trọng.

Bên cạnh đó, tìm kiếm ảnh dựa trên từ khóa còn nhiều hạn chế. Trên nhiều nền tảng, người dùng chỉ có thể tìm kiếm dựa vào tiêu đề, mô tả hoặc tag do tác giả nhập. Cách tìm kiếm này phụ thuộc mạnh vào chất lượng mô tả văn bản và sự trùng khớp giữa từ khóa của người tìm với từ khóa của người đăng. Trong khi đó, nội dung thị giác của ảnh thường chứa những đặc trưng khó mô tả bằng vài từ ngắn, chẳng hạn bố cục, phong cách nét vẽ, màu sắc chủ đạo, cảm xúc nhân vật hoặc mức độ tương đồng giữa một bản phác thảo và một tác phẩm hoàn thiện. Khi người dùng không biết chính xác tag cần nhập, hoặc khi tác giả gắn tag chưa đầy đủ, khả năng khám phá nội dung phù hợp bị giảm đáng kể.

Cuối cùng, các nền tảng chia sẻ ảnh hiện đại cần quan tâm tới nhu cầu tạo thu nhập của người sáng tạo. Nhiều nghệ sĩ không chỉ muốn đăng tải tác phẩm công khai, mà còn muốn cung cấp nội dung độc quyền, ảnh độ phân giải cao, bản phác thảo, tài liệu hậu trường hoặc quyền truy cập sớm cho nhóm người hâm mộ trả phí. Nếu nền tảng chia sẻ ảnh không hỗ trợ cơ chế thành viên, người sáng tạo phải sử dụng thêm các dịch vụ bên ngoài để quản lý đăng ký và phân phối nội dung riêng. Điều này làm phân mảnh trải nghiệm người dùng, giảm khả năng giữ chân cộng đồng trên cùng một nền tảng và khiến việc kiểm soát quyền truy cập nội dung trở nên phức tạp.

Từ các vấn đề trên, có thể thấy bài toán xây dựng nền tảng chia sẻ ảnh trực tuyến có ý nghĩa thực tiễn rõ ràng. Nếu được giải quyết tốt, hệ thống không chỉ hỗ trợ người dùng phổ thông trong việc tìm kiếm và tương tác với nội dung ảnh, mà còn hỗ trợ người sáng tạo bảo vệ tác phẩm, quản lý cộng đồng người theo dõi và phát triển nguồn thu trực tiếp. Các kết quả của bài toán cũng có thể được mở rộng cho những miền ứng dụng khác như thư viện ảnh doanh nghiệp, nền tảng học tập mỹ thuật, kho dữ liệu thiết kế, hệ thống quản lý nội dung truyền thông hoặc các cộng đồng sáng tạo có dữ liệu đa phương tiện.

1.2 Mục tiêu và phạm vi đề tài

Hiện nay đã tồn tại nhiều sản phẩm phục vụ một phần nhu cầu của bài toán chia sẻ ảnh. Các mạng xã hội hình ảnh phổ thông có ưu điểm về khả năng lan truyền nội dung và tương tác nhanh, nhưng thường không tập trung vào quy trình xử lý ảnh chuyên sâu, phát hiện trùng lặp hoặc kiểm duyệt theo đặc thù của cộng đồng nghệ thuật. Các cộng đồng nghệ thuật số như các nền tảng hồ sơ nghệ sĩ hỗ trợ đăng tác phẩm, tag và bình luận, song tìm kiếm vẫn chủ yếu dựa vào từ khóa và mức độ đầy

đủ của mô tả do người đăng tải. Các nền tảng đăng ký hội viên hỗ trợ người sáng tạo thu phí từ người hâm mộ, nhưng thường tách rời khỏi nơi người dùng khám phá và tương tác với tác phẩm. Một số công cụ tìm kiếm ảnh bằng trí tuệ nhân tạo có khả năng hiểu nội dung thị giác tốt hơn, nhưng lại không bao phủ đầy đủ các chức năng xã hội, kiểm duyệt, phân quyền và thanh toán.

Như vậy, hạn chế chung của các hướng tiếp cận hiện tại là sự phân tán chức năng. Người sáng tạo có thể cần một nơi để đăng tác phẩm, một nơi khác để nhận thành viên trả phí, một công cụ riêng để quản lý ảnh và một quy trình riêng để kiểm duyệt hoặc xử lý tranh chấp. Đối với người xem, trải nghiệm khám phá cũng bị hạn chế khi tìm kiếm chỉ dựa trên từ khóa và không tận dụng được đặc trưng thị giác của ảnh. Đối với quản trị viên, việc kiểm duyệt thủ công trên khối lượng ảnh lớn có thể gây quá tải. Từ đó, đề án hướng tới xây dựng một nền tảng web chia sẻ ảnh trực tuyến tích hợp nhiều nhóm chức năng cần thiết trong cùng một hệ thống.

Mục tiêu tổng quát của đề tài là phát triển một nền tảng chia sẻ ảnh trực tuyến phục vụ cộng đồng nghệ thuật số, trong đó người dùng có thể khám phá, tìm kiếm, xem và tương tác với tác phẩm; người sáng tạo có thể đăng tải, quản lý tác phẩm và thiết lập gói thành viên; quản trị viên có thể kiểm duyệt nội dung, xử lý báo cáo và theo dõi hoạt động của hệ thống. Hệ thống được xây dựng dưới dạng ứng dụng web, với phần giao diện dành cho khách, người dùng đã đăng nhập, người sáng tạo và nhóm quản trị.

Về phạm vi chức năng, hệ thống tập trung vào các nhóm chức năng chính. Nhóm thứ nhất là quản lý tài khoản và phân quyền, bao gồm xác thực người dùng, đồng bộ hồ sơ, phân biệt người dùng thường, người sáng tạo và quản trị viên. Nhóm thứ hai là quản lý tác phẩm, bao gồm đăng tải nhiều ảnh, lưu thông tin mô tả, quản lý trạng thái xử lý, hỗ trợ chèn dấu bản quyền và kiểm soát quyền truy cập nội dung. Nhóm thứ ba là tìm kiếm và khám phá, bao gồm tìm kiếm từ khóa, lọc theo thuộc tính, xem nội dung nổi bật, tìm kiếm ngữ nghĩa và gợi ý tác phẩm liên quan. Nhóm thứ tư là tương tác cộng đồng, bao gồm thích, bình luận, lưu vào bộ sưu tập, theo dõi nghệ sĩ và báo cáo vi phạm. Nhóm thứ năm là cơ chế thành viên và thanh toán, bao gồm gói thành viên của người sáng tạo, đăng ký, hủy đăng ký, lịch sử thanh toán, doanh thu và yêu cầu rút tiền. Nhóm cuối cùng là quản trị và kiểm duyệt, bao gồm xử lý báo cáo, duyệt hoặc từ chối tác phẩm, quản lý người dùng, cảnh báo, khóa tài khoản và xem nhật ký thao tác.

Về phạm vi kỹ thuật, đề tài không đặt mục tiêu huấn luyện lại mô hình trí tuệ nhân tạo từ đầu hoặc xây dựng hạ tầng phân tán quy mô rất lớn. Các mô hình và thư viện có sẵn được sử dụng như thành phần hỗ trợ trong hệ thống ứng dụng. Đồ

án cũng không tập trung vào các vấn đề ngoài phạm vi như thương mại hóa sản phẩm, tối ưu hạ tầng đa vùng, xây dựng ứng dụng di động native hoặc giải quyết toàn bộ quy trình pháp lý về bản quyền. Trọng tâm của đề tài là thiết kế và triển khai một nền tảng web có kiến trúc rõ ràng, có khả năng xử lý ảnh bất đồng bộ, hỗ trợ tìm kiếm đa dạng, có kiểm duyệt và có mô hình thành viên cho người sáng tạo.

1.3 Định hướng giải pháp

Từ mục tiêu và phạm vi trên, đề án định hướng xây dựng hệ thống theo kiến trúc modular monolith. Theo định hướng này, backend được triển khai như một ứng dụng thống nhất nhưng được chia thành các module nghiệp vụ rõ ràng như xác thực, người dùng, tác phẩm, tìm kiếm, hàng đợi xử lý, thanh toán, báo cáo, quản trị, khuyến nghị và theo dõi. Cách tiếp cận này phù hợp với quy mô của đề án vì giữ được sự đơn giản trong triển khai, đồng thời vẫn tạo ranh giới mã nguồn đủ rõ để mở rộng hoặc tách dịch vụ trong tương lai.

Đối với xử lý ảnh và các tác vụ tốn tài nguyên, hệ thống sử dụng mô hình xử lý nền thông qua hàng đợi. Khi người sáng tạo đăng tải tác phẩm, các bước nặng như tạo ảnh thu nhỏ, tạo ảnh làm mờ, kiểm tra trùng lặp, kiểm duyệt nội dung và sinh đặc trưng ảnh không được thực hiện trực tiếp trong luồng yêu cầu chính. Định hướng này giúp giảm thời gian chờ của người dùng, hạn chế lỗi quá thời gian chờ và cho phép hệ thống theo dõi trạng thái xử lý của từng tác phẩm.

Đối với tìm kiếm, hệ thống kết hợp tìm kiếm toàn văn và tìm kiếm ngữ nghĩa. Tìm kiếm toàn văn phục vụ các truy vấn dựa trên tiêu đề, mô tả, tag và tên tác giả; tìm kiếm ngữ nghĩa phục vụ các truy vấn tự nhiên hoặc truy vấn dựa trên đặc trưng thị giác của ảnh. Việc kết hợp hai hướng này giúp hệ thống vừa đáp ứng nhu cầu tìm kiếm truyền thống, vừa cải thiện khả năng khám phá nội dung trong trường hợp từ khóa không đủ diễn đạt nội dung ảnh.

Đối với kiểm duyệt và bảo vệ nội dung, hệ thống định hướng sử dụng các kỹ thuật tự động ở bước đầu và vẫn giữ vai trò xử lý cuối cùng cho quản trị viên. Nội dung nhạy cảm được phát hiện bằng mô hình phân loại ảnh; ảnh trùng lặp hoặc gần trùng lặp được phát hiện bằng dấu vân tay thị giác; các trường hợp bị báo cáo hoặc cần xem xét lại được đưa vào quy trình kiểm duyệt. Cách tiếp cận này giúp giảm tải cho người điều phối nhưng không loại bỏ hoàn toàn yếu tố con người trong các quyết định nhạy cảm.

Đối với mô hình kinh tế sáng tạo, hệ thống tích hợp cơ chế gói thành viên của người sáng tạo và thanh toán trực tuyến. Người sáng tạo có thể tạo các gói thành viên với mức giá và quyền lợi khác nhau; người dùng có thể đăng ký để truy cập nội dung giới hạn; hệ thống đồng bộ trạng thái thanh toán để kiểm soát quyền truy

cập. Đây là thành phần quan trọng để nền tảng không chỉ là nơi chia sẻ ảnh mà còn hỗ trợ trực tiếp hoạt động sáng tạo chuyên nghiệp.

Đóng góp chính của đề án là xây dựng một nền tảng chia sẻ ảnh trực tuyến tích hợp các nhóm chức năng thường bị tách rời: đăng tải và xử lý ảnh, kiểm duyệt bán tự động, phát hiện trùng lặp, tìm kiếm lai, tương tác cộng đồng, quản lý thành viên và thanh toán. Kết quả đạt được là một hệ thống web có cấu trúc module, có luồng xử lý nền, có mô hình dữ liệu phục vụ ảnh và tác phẩm, đồng thời có nền tảng để mở rộng sang các chức năng nâng cao như khuyến nghị cá nhân hóa, kiểm duyệt nâng cao và phân tích cộng đồng.

1.4 Bố cục đề án

Phần còn lại của báo cáo đề án tốt nghiệp được tổ chức thành năm chương tiếp theo. Chương 2 trình bày khảo sát và phân tích yêu cầu của hệ thống. Nội dung chương này bao gồm khảo sát hiện trạng các nhóm nền tảng chia sẻ ảnh và các hệ thống liên quan, xác định các tác nhân tham gia, mô tả các chức năng mức cao, xây dựng biểu đồ use case, đặc tả một số use case quan trọng và tổng hợp các yêu cầu phi chức năng. Đây là cơ sở để xác định phạm vi thiết kế và triển khai ở các chương sau.

Chương 3 trình bày nền tảng lý thuyết và công nghệ sử dụng trong đề án. Chương này giới thiệu các công nghệ nền tảng cho backend, frontend, cơ sở dữ liệu, xử lý nền, tìm kiếm, xác thực và thanh toán. Bên cạnh đó, chương cũng trình bày các cơ sở lý thuyết liên quan tới mô hình học biểu diễn ảnh-văn bản, phân loại ảnh nhạy cảm, bám nhận thức và tìm kiếm trong không gian vector. Các nội dung này giúp giải thích vì sao hệ thống lựa chọn những kỹ thuật và công nghệ tương ứng.

Chương 4 trình bày quá trình phân tích thiết kế, triển khai và đánh giá hệ thống. Nội dung chương bao gồm thiết kế kiến trúc tổng thể, thiết kế cơ sở dữ liệu, thiết kế các module backend, thiết kế giao diện chính, mô tả luồng xử lý ảnh, cấu hình các dịch vụ trung gian và phương pháp kiểm thử. Chương này làm rõ cách các yêu cầu đã nêu ở Chương 2 được chuyển hóa thành thành phần kỹ thuật cụ thể.

Chương 5 trình bày các giải pháp và đóng góp nổi bật của đề án. Chương này tập trung vào những điểm có giá trị kỹ thuật và nghiệp vụ của hệ thống, bao gồm tách xử lý ảnh khỏi luồng yêu cầu chính, kiểm duyệt bán tự động, phát hiện ảnh trùng lặp, tìm kiếm ngữ nghĩa, đồng bộ chỉ mục tìm kiếm và mô hình thành viên cho người sáng tạo. Các nội dung trong chương này làm rõ mức độ đóng góp của đề án so với một hệ thống chia sẻ ảnh thông thường.

Chương 6 đưa ra kết luận và hướng phát triển. Chương này tổng kết các kết quả

đã đạt được, đánh giá mức độ đáp ứng mục tiêu ban đầu, nêu các hạn chế còn tồn tại và đề xuất các hướng mở rộng trong tương lai. Những hướng phát triển có thể bao gồm nâng cao chất lượng kiểm duyệt, tối ưu tìm kiếm vector, cải tiến hệ thống khuyến nghị, bổ sung cơ chế xử lý tranh chấp bản quyền và mở rộng hạ tầng triển khai cho quy mô người dùng lớn hơn.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Chương này trình bày quá trình khảo sát hiện trạng và phân tích yêu cầu cho nền tảng chia sẻ ảnh trực tuyến được xây dựng trong đồ án. Nội dung chương bắt đầu từ việc khảo sát các hệ thống tương tự, chỉ ra ưu điểm và hạn chế của từng nhóm nền tảng hiện có. Trên cơ sở đó, chương xác định các chức năng mức cao của hệ thống, mô tả các tác nhân tham gia, xây dựng biểu đồ use case tổng quát và các biểu đồ use case phân rã, trình bày một số quy trình nghiệp vụ quan trọng, sau đó đặc tả chi tiết các use case tiêu biểu. Phần cuối chương tổng hợp các yêu cầu phi chức năng làm cơ sở cho thiết kế kiến trúc, thiết kế cơ sở dữ liệu và triển khai hệ thống ở các chương sau.

2.1 Khảo sát hiện trạng

2.1.1 Khảo sát hệ thống hiện có

Ba nền tảng chia sẻ ảnh và nghệ thuật số nổi bật trên thị trường hiện nay là Pinterest, DeviantArt và Patreon. Đây là những nền tảng lớn, có lượng người dùng đông đảo và nhiều chức năng phong phú, là cơ sở quan trọng để so sánh, phân tích và đề xuất cho hệ thống cần phát triển.

Pinterest

Địa chỉ truy cập: <https://www.pinterest.com>

Pinterest là nền tảng mạng xã hội hình ảnh phổ thông, nơi trọng tâm là khám phá nội dung và tương tác nhanh.

• **Ưu điểm:**

- Tương tác nhanh, dễ tiếp cận, phù hợp khám phá nội dung mới.
- Tốc độ lan truyền nội dung hình ảnh rất cao.

• **Nhược điểm:**

- Tìm kiếm nghệ thuật chuyên sâu chưa tốt.
- Khó kiểm soát nội dung theo quy trình riêng của cộng đồng sáng tạo đặc thù như truyện tranh.

DeviantArt

Địa chỉ truy cập: <https://www.deviantart.com>

DeviantArt là cộng đồng nghệ thuật số lâu đời, tập trung nhiều vào hồ sơ nghệ sĩ và bộ sưu tập tác phẩm.

- **Ưu điểm:**

- Không gian phù hợp cho người sáng tạo với hồ sơ nghệ sĩ chuyên nghiệp.
- Hệ thống tag, album tác phẩm và phân loại phong cách đa dạng.

- **Nhược điểm:**

- Khả năng tìm kiếm phụ thuộc nhiều vào tag và mô tả do người đăng nhập thủ công.
- Chưa nhấn mạnh xử lý nền tự động và phát hiện ảnh trùng lặp triệt để.

Patreon

Địa chỉ truy cập: <https://www.patreon.com>

Patreon là nền tảng hỗ trợ người sáng tạo kiếm tiền thông qua quan hệ trả phí với người hâm mộ.

- **Ưu điểm:**

- Hỗ trợ người sáng tạo thiết lập gói hội viên linh hoạt.
- Cung cấp giải pháp nhận doanh thu trực tiếp từ người hâm mộ.

- **Nhược điểm:**

- Tách rời khỏi nền tảng khám phá ảnh, khiến người dùng phải chuyển đổi ứng dụng.
- Quyền truy cập nội dung độc quyền thường phải đồng bộ thủ công giữa nơi trưng bày tác phẩm và nơi thanh toán.

2.1.2 Các chức năng cần có cho hệ thống

Mặc dù mỗi nền tảng có định hướng khác nhau, chúng đều giải quyết một phần của bài toán mà đề án hướng tới. Hệ thống đề xuất trong đề án sẽ giải quyết các hạn chế bằng cách tích hợp trực tiếp cơ chế chia sẻ ảnh, xử lý nền, kiểm duyệt nội dung, tìm kiếm lai và quản lý hội viên trên cùng một nền tảng. Bảng 2.1 trình bày tóm tắt các tính năng cốt lõi cần có cho hệ thống để đạt được mục tiêu này.

Bảng 2.1: Tóm tắt các tính năng cốt lõi cần thiết cho hệ thống đề xuất

Nhóm tính năng	Mô tả chi tiết
Tài khoản và phân quyền	Đăng nhập, đồng bộ hồ sơ, quản lý trạng thái nghệ sĩ, kiểm soát quyền quản trị viên.
Quản lý tác phẩm	Đăng tải ảnh, tạo ảnh thu nhỏ, ảnh làm mờ, đóng dấu bản quyền, kiểm duyệt ảnh nhạy cảm và phát hiện ảnh trùng lặp bằng mã băm nhận thức.
Tìm kiếm và khám phá	Tìm kiếm từ khóa, lọc theo tag, tỷ lệ ảnh, tìm kiếm ngữ nghĩa, gợi ý tác phẩm liên quan.
Tương tác cộng đồng	Thích, bình luận, đánh dấu tác phẩm, tạo bộ sưu tập, theo dõi nghệ sĩ.
Hội viên và thanh toán	Thiết lập hạng gói hội viên, đăng ký trả phí định kỳ, giới hạn quyền xem tác phẩm theo gói hội viên, quản lý doanh thu.
Kiểm duyệt và quản trị	Báo cáo vi phạm, duyệt hoặc từ chối tác phẩm, khóa tài khoản, nhật ký hệ thống.

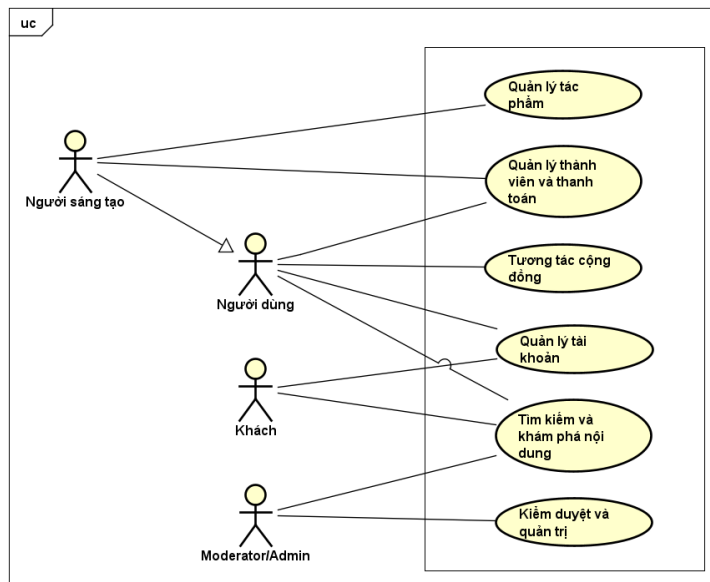
2.2 Tổng quan chức năng

Hệ thống được phân tích theo phương pháp hướng đối tượng với các tác nhân chính và các use case mức cao. Mục tiêu của phần này là mô tả tổng quan phạm vi chức năng, chưa đi vào luồng chi tiết của từng use case. Các use case quan trọng sẽ được đặc tả trong phần 2.3.

2.2.1 Biểu đồ use case tổng quát

Hệ thống có bốn nhóm tác nhân chính. Khách là người chưa đăng nhập, có thể xem nội dung công khai, tìm kiếm, xem trang tác phẩm và trang nghệ sĩ trong phạm vi cho phép. Người dùng là người đã đăng nhập, có thể tương tác với tác phẩm, quản lý bộ sưu tập, theo dõi nghệ sĩ, báo cáo vi phạm và đăng ký các gói thành viên. Người sáng tạo là người dùng có thêm khả năng đăng tải và quản lý tác phẩm, thiết lập gói hội viên, xem người đăng ký và theo dõi doanh thu. Quản trị viên hoặc điều phối viên là nhóm có quyền xử lý báo cáo, kiểm duyệt tác phẩm, quản lý người dùng, quản lý cấu hình hệ thống và theo dõi thống kê vận hành.

Hình 2.1 trình bày biểu đồ use case tổng quát của hệ thống.



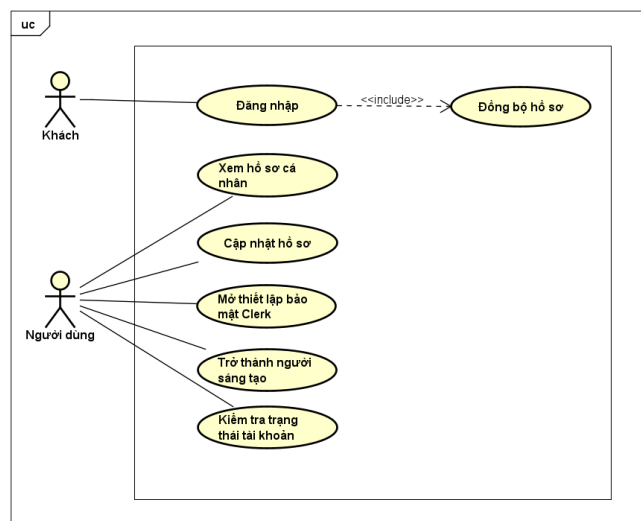
Hình 2.1: Biểu đồ use case tổng quát của hệ thống

Các use case mức cao của hệ thống gồm: quản lý tài khoản, quản lý tác phẩm, tìm kiếm và khám phá nội dung, tương tác cộng đồng, quản lý thành viên và thanh toán, kiểm duyệt và quản trị. Nhóm quản lý tài khoản bao phủ đăng nhập, đồng bộ hồ sơ, cập nhật thông tin cá nhân và trở thành người sáng tạo. Nhóm quản lý tác phẩm bao phủ đăng tải, xử lý ảnh, chỉnh sửa, xem trạng thái và quản lý quyền truy cập theo tier. Nhóm tìm kiếm và khám phá bao phủ tìm kiếm toàn văn, lọc, sắp xếp, tìm kiếm ngữ nghĩa, tìm kiếm bằng phác thảo và gợi ý nội dung. Nhóm tương tác cộng đồng bao phủ thích, bình luận, lưu vào bộ sưu tập, theo dõi và báo cáo vi phạm. Nhóm thành viên và thanh toán bao phủ gói nền tảng, gói hội viên của hoạ sĩ, đăng ký, hủy đăng ký, lịch sử thanh toán, doanh thu và thanh toán. Nhóm kiểm duyệt và quản trị bao phủ xử lý báo cáo, duyệt hoặc từ chối tác phẩm, quản lý người dùng, nhật ký hoạt động hệ thống và thống kê.

2.2.2 Biểu đồ use case phân rã Quản lý tài khoản

Use case Quản lý tài khoản liên quan đến cả người dùng thường và người sáng tạo. Hệ thống sử dụng Clerk cho xác thực, nhưng vẫn duy trì hồ sơ người dùng nội bộ để lưu vai trò, trạng thái artist, trạng thái bị khóa, thông tin thanh toán Stripe và các quan hệ xã hội. Khi người dùng đăng nhập lần đầu, backend thực hiện đồng bộ dữ liệu, tạo hoặc cập nhật bản ghi người dùng dựa trên định danh Clerk.

Hình 2.2 trình bày biểu đồ use case phân rã của nhóm chức năng Quản lý tài khoản.



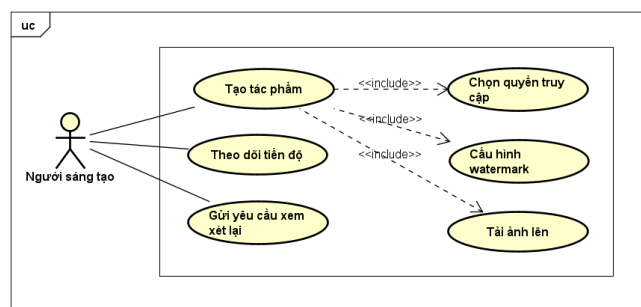
Hình 2.2: Biểu đồ use case phân rã Quản lý tài khoản

Các use case con gồm đăng nhập, xem hồ sơ cá nhân, cập nhật hồ sơ, mở trang thiết lập bảo mật của Clerk, trở thành người sáng tạo, xem bảng điều khiển cá nhân và kiểm tra trạng thái tài khoản. Đối với người dùng bị khóa, middleware phía frontend gọi backend để kiểm tra trạng thái và chuyển hướng người dùng sang trang thông báo chặn nếu cần.

2.2.3 Biểu đồ use case phân rã Quản lý tác phẩm

Use case Quản lý tác phẩm là trung tâm của hệ thống. Người sáng tạo có thể đăng tải một tác phẩm gồm một hoặc nhiều ảnh, nhập tiêu đề, mô tả, tag, chọn chế độ hiển thị, chọn hạng hội viên yêu cầu nếu là nội dung giới hạn, cấu hình watermark và theo dõi trạng thái xử lý. Sau khi tạo tác phẩm, hệ thống đưa tác phẩm vào hàng đợi để worker xử lý ảnh bất đồng bộ.

Hình 2.3 trình bày biểu đồ use case phân rã của nhóm chức năng Quản lý tác phẩm.



Hình 2.3: Biểu đồ use case phân rã Quản lý tác phẩm

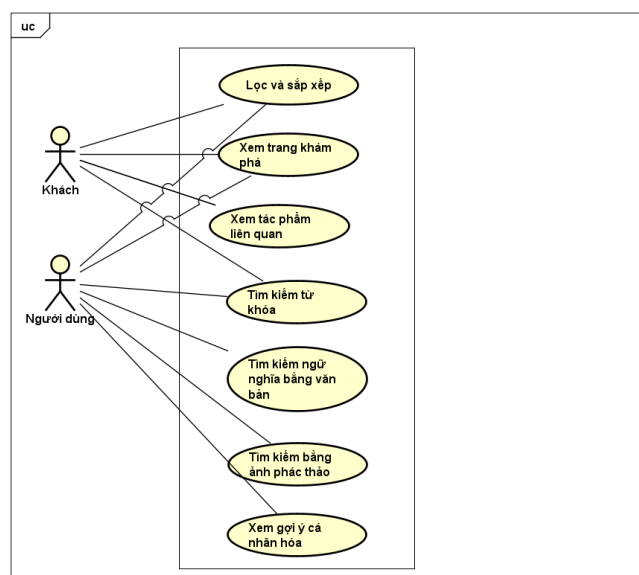
Các use case con gồm tạo tác phẩm, tải ảnh lên, cấu hình watermark, chọn quyền truy cập, gửi xử lý nền, xem tiến độ xử lý, xem tác phẩm đã xuất bản, chỉnh sửa

thông tin, quảng bá tác phẩm, xem tác phẩm liên quan và gửi khiếu nại khi tác phẩm bị đưa vào trạng thái cần xử lý. Hệ thống phân biệt trạng thái tác phẩm như đang xử lý, đã xuất bản, thất bại, cần xử lý và đang được xem xét lại. Thiết kế này phù hợp với thực tế xử lý ảnh vì kiểm duyệt và phát hiện trùng lặp có thể xảy ra ở từng ảnh trong cùng một tác phẩm.

2.2.4 Biểu đồ use case phân rã Tìm kiếm và khám phá nội dung

Use case Tìm kiếm và khám phá nội dung phục vụ cả khách và người dùng đã đăng nhập. Khách có thể tìm kiếm và xem nội dung công khai; người dùng đã đăng nhập có thêm khả năng tìm kiếm cá nhân hóa, dùng tìm kiếm bằng phác thảo nếu thỏa điều kiện gói dịch vụ và nhận gợi ý dựa trên lịch sử tương tác.

Hình 2.4 trình bày biểu đồ use case phân rã của nhóm chức năng Tìm kiếm và khám phá nội dung.



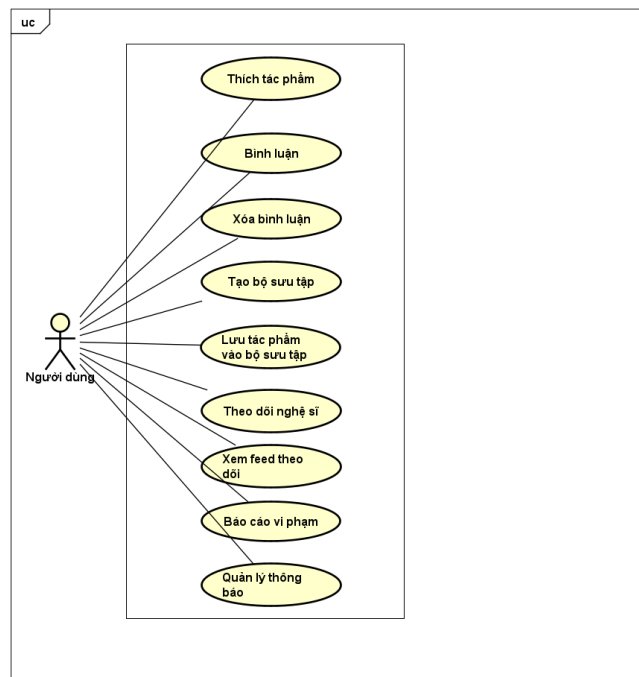
Hình 2.4: Biểu đồ use case phân rã Tìm kiếm và khám phá nội dung

Các use case con gồm tìm kiếm theo từ khóa, lọc theo tag, tỷ lệ ảnh, độ phân giải và nội dung AI, sắp xếp theo thời gian hoặc độ phổ biến, tìm kiếm ngữ nghĩa bằng văn bản, tìm kiếm bằng ảnh phác thảo, xem trang khám phá, xem tác phẩm phổ biến, xem nghệ sĩ nổi bật, xem tag thịnh hành và xem gợi ý tác phẩm liên quan. Hệ thống sử dụng Meilisearch cho tìm kiếm toàn văn và pgvector cho tìm kiếm ngữ nghĩa.

2.2.5 Biểu đồ use case phân rã Tương tác cộng đồng

Use case Tương tác cộng đồng bao phủ các hành vi giúp người dùng hình thành quan hệ xã hội xung quanh tác phẩm. Đây là nhóm chức năng làm cho nền tảng không chỉ là kho lưu trữ ảnh mà trở thành cộng đồng.

Hình 2.5 trình bày biểu đồ use case phân rã của nhóm chức năng Tương tác cộng đồng.



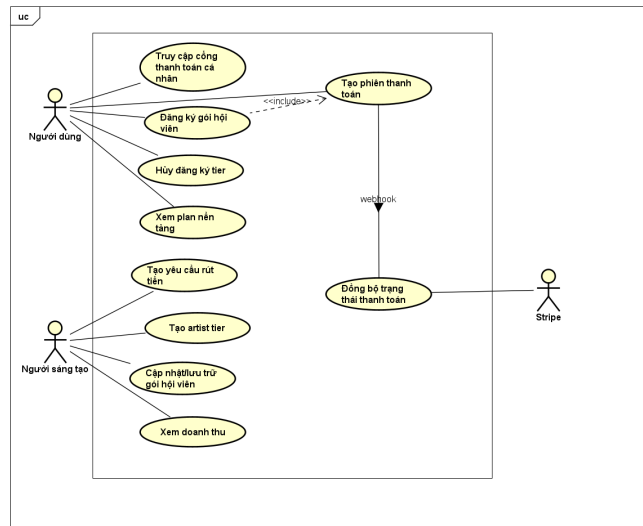
Hình 2.5: Biểu đồ use case phân rã Tương tác cộng đồng

Các use case con gồm thích hoặc bỏ thích tác phẩm, bình luận, xóa bình luận của bản thân, lưu tác phẩm vào bộ sưu tập, tạo bộ sưu tập, thêm hoặc xóa tác phẩm khỏi bộ sưu tập, theo dõi hoặc bỏ theo dõi nghệ sĩ, xem bảng tin theo dõi, báo cáo tác phẩm vi phạm và nhận thông báo. Một số tương tác như lướt xem hoặc lướt thích cũng được dùng để cập nhật thống kê và đồng bộ sang chỉ mục tìm kiếm.

2.2.6 Biểu đồ use case phân rã Quản lý thành viên và thanh toán

Use case Quản lý thành viên và thanh toán bao gồm hai nhánh: đăng ký gói nền tảng hệ thống và đăng ký gói hội viên theo artist. Đăng ký nền tảng hệ thống phục vụ các quyền lợi chung của người dùng, còn hội viên phục vụ quan hệ trả phí giữa người sáng tạo và người hâm mộ.

Hình 2.6 trình bày biểu đồ use case phân rã của nhóm chức năng Quản lý thành viên và thanh toán.



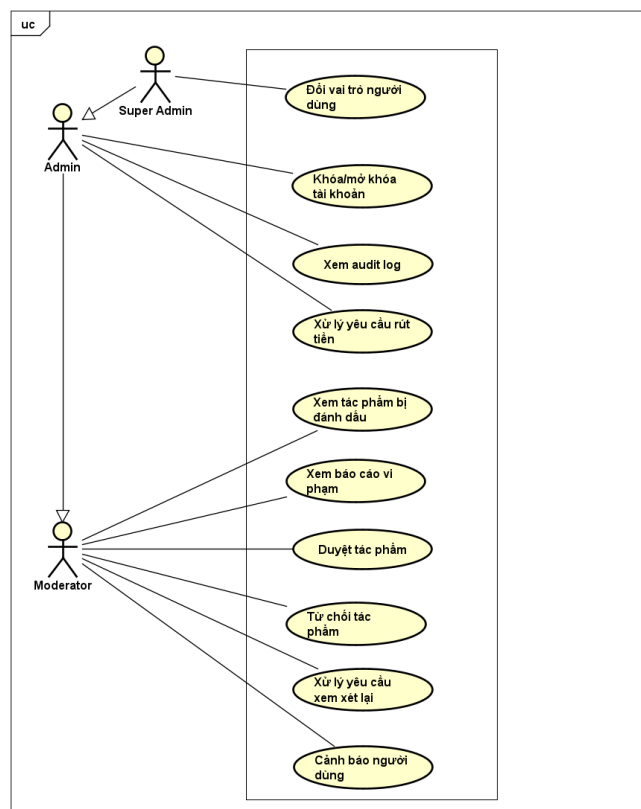
Hình 2.6: Biểu đồ use case phân rã Quản lý thành viên và thanh toán

Các use case con gồm xem danh sách gói hội viên, tạo phiên thanh toán, đồng bộ kết quả hoàn tất thanh toán, mở cổng thông tin khách hàng, hủy thanh toán, tạo gói hội viên, cập nhật gói hội viên, lưu trữ gói hội viên, xem danh sách người đăng ký tier, đăng ký các hạng gói hội viên của artist, hủy đăng ký gói hội viên, xem lịch sử thanh toán, xem doanh thu và tạo yêu cầu rút tiền. Hệ thống tích hợp cổng thanh toán (Stripe) để xử lý thanh toán và webhook để đồng bộ trạng thái về cơ sở dữ liệu nội bộ.

2.2.7 Biểu đồ use case phân rã Kiểm duyệt và quản trị

Use case Kiểm duyệt và quản trị dành cho Moderator, Admin và Super Admin. Đây là nhóm chức năng đảm bảo nền tảng vận hành an toàn, kiểm soát được nội dung vi phạm và truy vết được thao tác quan trọng.

Hình 2.7 trình bày biểu đồ use case phân rã của nhóm chức năng Kiểm duyệt và quản trị.



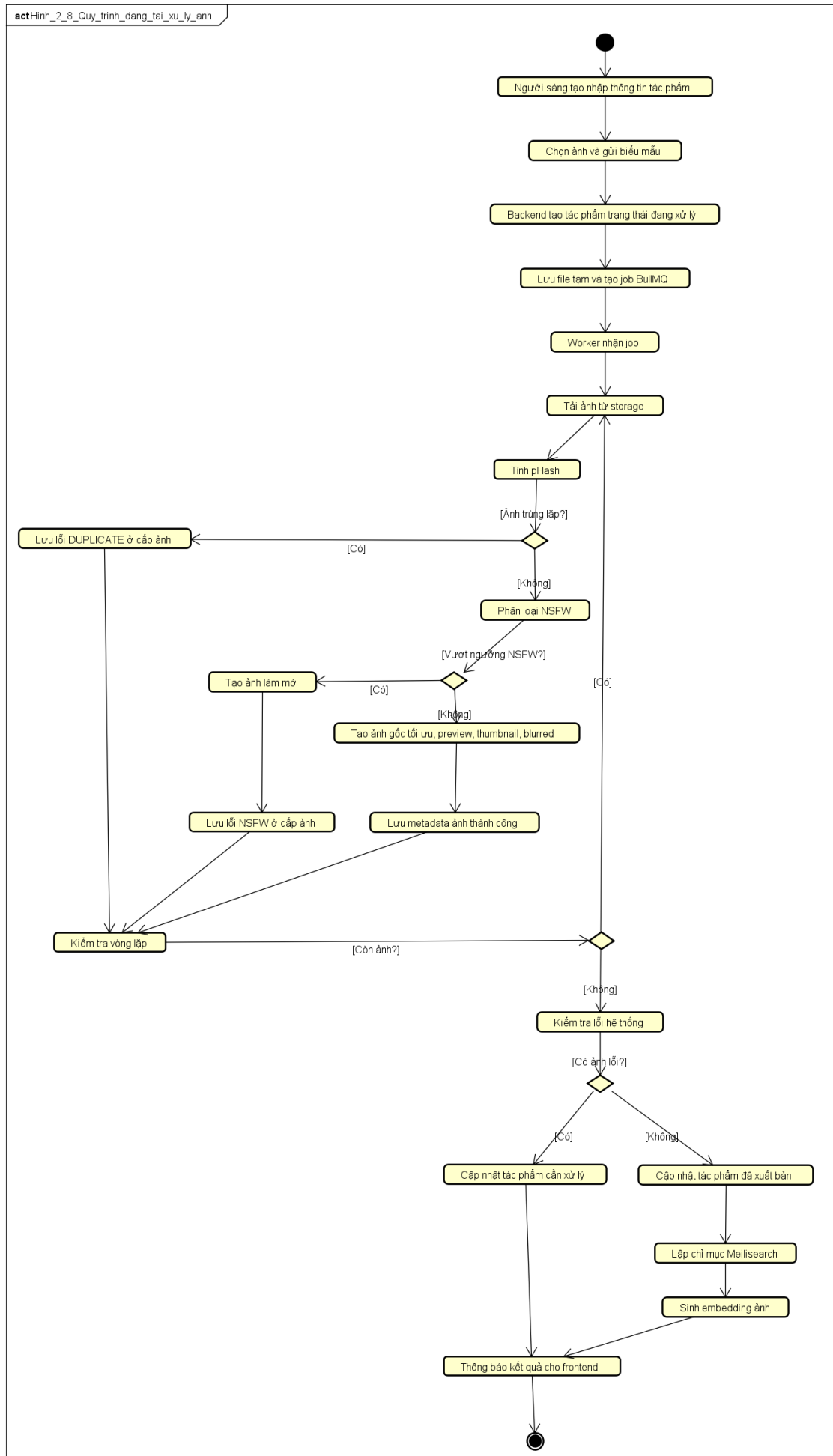
Hình 2.7: Biểu đồ use case phân rã Kiểm duyệt và quản trị

Các use case con gồm xem danh sách tác phẩm bị đánh dấu, xem báo cáo vi phạm của tác phẩm, duyệt tác phẩm, từ chối tác phẩm, xử lý yêu cầu xem xét lại, xem lịch sử báo cáo đã giải quyết, cảnh báo người dùng, khóa hoặc mở khóa người dùng, thay đổi vai trò người dùng, xem thống kê, cấu hình trọng số xếp hạng, cấu hình tìm kiếm khám phá, xem nhật ký hệ thống, quản lý nhân sự điều phối và xử lý yêu cầu rút tiền. Hệ thống cần đảm bảo chỉ người có vai trò phù hợp mới truy cập được các chức năng này.

2.2.8 Quy trình nghiệp vụ

Quy trình nghiệp vụ quan trọng nhất của hệ thống là quy trình đăng tải và xử lý tác phẩm. Quy trình này kết hợp nhiều use case: đăng nhập, tạo tác phẩm, tải ảnh, đưa job vào hàng đợi, xử lý ảnh, kiểm tra trùng lặp, kiểm duyệt NSFW, lưu biến thể ảnh, lập chỉ mục tìm kiếm, sinh embedding và thông báo kết quả cho người sáng tạo.

Hình 2.8 mô tả luồng xử lý chi tiết của quy trình đăng tải và xử lý tác phẩm.

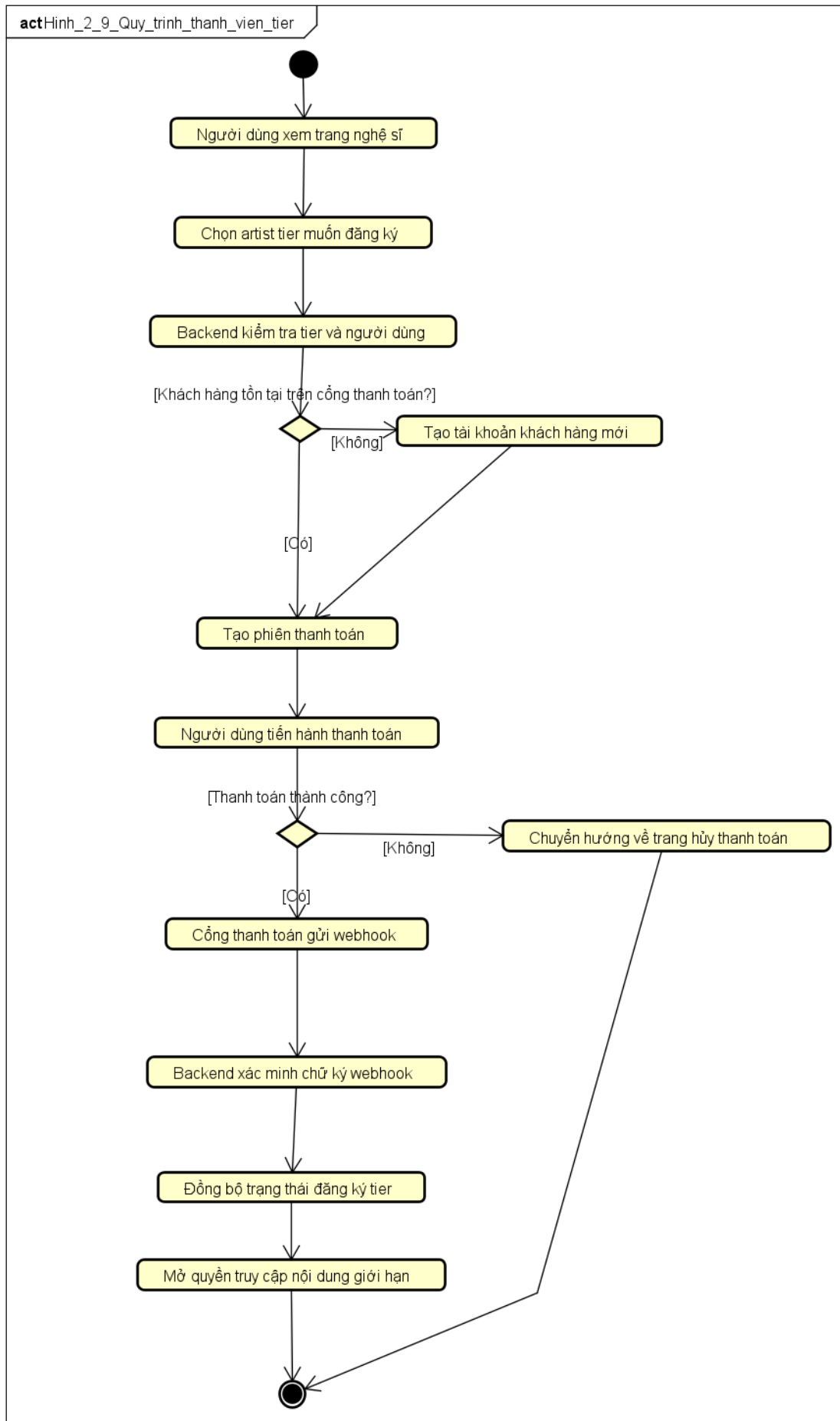


Hình 2.8: Quy trình nghiệp vụ đăng tải và xử lý tác phẩm

Mô tả sơ bộ quy trình như sau. Người sáng tạo đăng nhập và truy cập trang tải lên. Sau khi nhập thông tin tác phẩm và chọn ảnh, frontend gửi yêu cầu tạo tác phẩm tới backend. Backend lưu metadata, lưu tệp tạm vào hệ thống lưu trữ, tạo job xử lý ảnh và trả về mã job cho frontend theo dõi. Worker nhận job, tải từng ảnh, tính pHash để phát hiện trùng lặp, phân loại NSFW, tạo ảnh gốc tối ưu, ảnh xem trước, ảnh thu nhỏ và ảnh làm mờ. Nếu ảnh hợp lệ, thông tin ảnh được lưu với trạng thái thành công; nếu ảnh trùng lặp hoặc nhạy cảm, hệ thống lưu metadata lỗi ở cấp ảnh. Khi tất cả ảnh đã được xử lý, nếu không có lỗi, tác phẩm được xuất bản và lập chỉ mục tìm kiếm; nếu có lỗi, tác phẩm chuyển sang trạng thái cần xử lý để người sáng tạo hoặc quản trị viên xem xét.

Quy trình nghiệp vụ thứ hai là quy trình đăng ký gói hội viên. Quy trình này kết hợp các use case xem hạng gói hội viên công khai, tạo phiên thanh toán, thanh toán qua cổng thanh toán, webhook đồng bộ đăng ký và kiểm tra quyền truy cập nội dung giới hạn.

Hình 2.9 mô tả luồng xử lý chi tiết của quy trình đăng ký gói thành viên.



Hình 2.9: Quy trình nghiệp vụ đăng ký gói thành viên của người sáng tạo

Người dùng truy cập trang nghệ sĩ và xem các hạng gói hội viên đang hoạt động. Khi chọn đăng ký một gói hội viên, backend kiểm tra người dùng, đảm bảo khách hàng trên cổng thanh toán tồn tại và tạo phiên thanh toán. Sau khi người dùng thanh toán, cổng thanh toán gửi webhook về backend. Backend xác minh chữ ký webhook, đọc metadata của đăng ký, cập nhật bảng đăng ký gói hội viên và ghi nhận thanh toán. Từ thời điểm gói hội viên đăng ký ở trạng thái hoạt động, người dùng được phép truy cập các tác phẩm đặt quyền truy cập theo tier tương ứng.

2.3 Đặc tả chức năng

Phần này đặc tả sáu use case quan trọng của hệ thống. Mỗi use case được trình bày theo cấu trúc gồm mã use case, tên use case, tác nhân, tiền điều kiện, tóm tắt, luồng sự kiện chính, luồng sự kiện thay thế và hậu điều kiện. Các use case được lựa chọn là những chức năng có ảnh hưởng lớn tới kiến trúc và nghiệp vụ của hệ thống.

2.3.1 Đặc tả use case Đăng nhập và đồng bộ tài khoản

Bảng 2.2 mô tả chi tiết đặc tả use case Đăng nhập và đồng bộ tài khoản.

Bảng 2.2: Đặc tả use case Đăng nhập và đồng bộ tài khoản

Use Case ID	UC-01
Tên Use Case	Đăng nhập và đồng bộ tài khoản
Tác nhân	Khách, Người dùng
Tiền điều kiện	Người dùng có tài khoản hợp lệ trên dịch vụ xác thực Clerk hoặc có khả năng đăng ký tài khoản mới.
Tóm tắt	Người dùng đăng nhập vào hệ thống. Backend xác minh token Clerk và tạo hoặc cập nhật hồ sơ người dùng trong cơ sở dữ liệu nội bộ.
Luồng sự kiện chính	1. Khách truy cập trang đăng nhập. 2. Hệ thống hiển thị giao diện đăng nhập của Clerk. 3. Người dùng nhập thông tin xác thực hoặc chọn phương thức đăng nhập hỗ trợ. 4. Clerk xác thực thành công và cấp phiên đăng nhập. 5. Frontend gọi API cần xác thực kèm token. 6. Backend xác minh token, lấy định danh Clerk của người dùng. 7. Backend tìm bản ghi người dùng theo định danh Clerk. 8. Nếu bản ghi chưa tồn tại, backend tạo hồ sơ người dùng mới gồm email, username, tên hiển thị và avatar. 9. Hệ thống trả về thông tin người dùng nội bộ cho frontend.
Luồng sự kiện thay thế	Thông tin xác thực không hợp lệ: Clerk từ chối đăng nhập và yêu cầu người dùng nhập lại thông tin. Tài khoản bị khóa: Sau khi xác thực, frontend gọi API kiểm tra hồ sơ; nếu người dùng bị khóa, hệ thống chuyển hướng sang trang thông báo chặn. Lỗi đồng bộ hồ sơ: Backend trả về lỗi, frontend hiển thị thông báo và yêu cầu người dùng thử lại.
Hậu điều kiện	Người dùng có phiên đăng nhập hợp lệ; cơ sở dữ liệu nội bộ có bản ghi người dùng tương ứng với định danh Clerk.

2.3.2 Đặc tả use case Đăng tải tác phẩm

Bảng 2.3 mô tả chi tiết đặc tả use case Đăng tải tác phẩm.

Bảng 2.3: Đặc tả use case Đăng tải tác phẩm

Use Case ID	UC-02
Tên Use Case	Đăng tải tác phẩm
Tác nhân	Người sáng tạo
Tiền điều kiện	Người dùng đã đăng nhập, không bị khóa tài khoản và có quyền đăng tải tác phẩm.
Tóm tắt	Người sáng tạo tạo tác phẩm mới, tải lên một hoặc nhiều ảnh. Hệ thống xử lý ảnh qua hàng đợi nền, kiểm tra trùng lặp, kiểm duyệt NSFW và xuất bản nếu hợp lệ.
Luồng sự kiện chính	1. Người sáng tạo truy cập trang tải lên. 2. Hệ thống hiển thị biểu mẫu tạo tác phẩm. 3. Người sáng tạo nhập tiêu đề, mô tả, tag, loại tác phẩm, trạng thái AI, quyền truy cập và cấu hình watermark nếu có. 4. Người sáng tạo chọn một hoặc nhiều ảnh và gửi biểu mẫu. 5. Backend kiểm tra dữ liệu đầu vào và tạo bản ghi tác phẩm ở trạng thái đang xử lý. 6. Backend lưu tệp ảnh tạm và tạo job xử lý ảnh trong hàng đợi. 7. Worker tải ảnh, tính pHash, kiểm tra trùng lặp, phân loại NSFW, tạo các biến thể ảnh và lưu metadata ảnh. 8. Nếu tất cả ảnh hợp lệ, hệ thống cập nhật tác phẩm sang trạng thái đã xuất bản. 9. Hệ thống lập chỉ mục tác phẩm vào Meilisearch và sinh embedding cho ảnh. 10. Frontend hiển thị thông báo xử lý hoàn tất.
Luồng sự kiện thay thế	Dữ liệu nhập không hợp lệ: Backend trả về lỗi validation, frontend yêu cầu người dùng sửa biểu mẫu. Ảnh bị trùng lặp: Worker lưu metadata lỗi ở cấp ảnh, tác phẩm chuyển sang trạng thái cần xử lý. Ảnh bị phát hiện NSFW: Worker tạo ảnh làm mờ, lưu điểm phát hiện và chuyển tác phẩm sang trạng thái cần xử lý. Lỗi xử lý ảnh: Tác phẩm chuyển sang trạng thái thất bại, hệ thống ghi log phục vụ điều tra.
Hậu điều kiện	Tác phẩm được xuất bản nếu hợp lệ; nếu có lỗi, tác phẩm được lưu ở trạng thái cần xử lý hoặc thất bại để người dùng và quản trị viên theo dõi.

2.3.3 Đặc tả use case Tìm kiếm và khám phá tác phẩm

Bảng 2.4 mô tả chi tiết đặc tả use case Tìm kiếm và khám phá tác phẩm.

Bảng 2.4: Đặc tả use case Tìm kiếm và khám phá tác phẩm

Use Case ID	UC-03
Tên Use Case	Tìm kiếm và khám phá tác phẩm
Tác nhân	Khách, Người dùng
Tiền điều kiện	Hệ thống có dữ liệu tác phẩm đã xuất bản và chỉ mục tìm kiếm đã được cập nhật.
Tóm tắt	Tác nhân tìm kiếm tác phẩm theo từ khóa, tag, bộ lọc, truy vấn ngữ nghĩa hoặc ảnh phác thảo.
Luồng sự kiện chính	1. Tác nhân truy cập trang tìm kiếm hoặc khám phá. 2. Hệ thống hiển thị ô tìm kiếm, bộ lọc và danh sách nội dung ban đầu. 3. Tác nhân nhập từ khóa hoặc chọn bộ lọc như tag, tỷ lệ ảnh, độ phân giải, nội dung AI và cách sắp xếp. 4. Frontend gửi truy vấn tới backend. 5. Backend truy vấn Meilisearch đối với tìm kiếm toàn văn hoặc truy vấn pgvector đối với tìm kiếm ngữ nghĩa. 6. Backend trả về danh sách tác phẩm phù hợp cùng thông tin tác giả, thumbnail, tag và chỉ số tương tác. 7. Frontend hiển thị kết quả và cho phép tác nhân mở trang chi tiết tác phẩm.
Luồng sự kiện thay thế	Không có kết quả: Hệ thống hiển thị thông báo không tìm thấy kết quả phù hợp và gợi ý thay đổi bộ lọc. Tìm kiếm ngữ nghĩa chưa khả dụng: Nếu dịch vụ embedding chưa sẵn sàng, hệ thống trả về thông báo và có thể gợi ý dùng tìm kiếm từ khóa. Người dùng vượt giới hạn tìm kiếm miễn phí: Hệ thống từ chối truy vấn theo chính sách giới hạn và gợi ý nâng cấp gói.
Hậu điều kiện	Danh sách kết quả phù hợp được hiển thị; các truy vấn ngữ nghĩa có thể được cache để tăng hiệu năng cho lần sau.

2.3.4 Đặc tả use case Tương tác với tác phẩm

Bảng 2.5 mô tả chi tiết đặc tả use case Tương tác với tác phẩm.

Bảng 2.5: Đặc tả use case Tương tác với tác phẩm

Use Case ID	UC-04
Tên Use Case	Tương tác với tác phẩm
Tác nhân	Người dùng
Tiền điều kiện	Người dùng đã đăng nhập; tác phẩm tồn tại và người dùng có quyền xem tác phẩm.
Tóm tắt	Người dùng thích, bình luận, lưu vào bộ sưu tập, theo dõi tác giả hoặc báo cáo tác phẩm vi phạm.
Luồng sự kiện chính	1. Người dùng mở trang chi tiết tác phẩm. 2. Hệ thống kiểm tra quyền truy cập đối với tác phẩm công khai hoặc tác phẩm giới hạn theo tier. 3. Người dùng chọn một hành động tương tác: thích, bình luận, lưu vào bộ sưu tập hoặc theo dõi tác giả. 4. Backend xác thực người dùng và kiểm tra điều kiện nghiệp vụ của hành động. 5. Backend ghi nhận tương tác trong cơ sở dữ liệu. 6. Hệ thống cập nhật bộ đếm liên quan như lượt thích, bình luận hoặc trạng thái lưu. 7. Frontend cập nhật giao diện để phản ánh kết quả.
Luồng sự kiện thay thế	Người dùng chưa có quyền xem: Hệ thống yêu cầu đăng nhập hoặc đăng ký tier phù hợp. Bình luận không hợp lệ: Backend từ chối nội dung rỗng hoặc không thỏa quy định. Báo cáo vi phạm: Người dùng chọn lý do, gửi mô tả; hệ thống tạo báo cáo để quản trị viên xử lý. Thao tác trùng lặp: Nếu người dùng đã thích hoặc đã lưu, hệ thống chuyển hành động thành bỏ thích hoặc bỏ lưu tùy ngữ cảnh.
Hậu điều kiện	Tương tác được ghi nhận; bộ đếm và trạng thái giao diện được cập nhật; nếu có báo cáo, báo cáo được đưa vào hàng xử lý kiểm duyệt.

2.3.5 Đặc tả use case Đăng ký gói thành viên của người sáng tạo

Bảng 2.6 mô tả chi tiết đặc tả use case Đăng ký gói thành viên của người sáng tạo.

Bảng 2.6: Đặc tả use case Đăng ký gói thành viên của người sáng tạo

Use Case ID	UC-05
Tên Use Case	Đăng ký gói thành viên của người sáng tạo
Tác nhân	Người dùng, Người sáng tạo, Cổng thanh toán
Tiền điều kiện	Người dùng đã đăng nhập; người sáng tạo có ít nhất một artist tier đang hoạt động; Cổng thanh toán được cấu hình hợp lệ.
Tóm tắt	Người dùng đăng ký một tier của người sáng tạo để truy cập nội dung giới hạn và quyền lợi tương ứng.
Luồng sự kiện chính	1. Người dùng truy cập trang nghệ sĩ hoặc trang hội viên. 2. Hệ thống hiển thị danh sách artist tier công khai. 3. Người dùng chọn tier muốn đăng ký. 4. Backend kiểm tra tier, người dùng và khách hàng trên cổng thanh toán. 5. Backend tạo phiên thanh toán và trả về đường dẫn checkout. 6. Người dùng hoàn tất thanh toán trên cổng thanh toán. 7. Cổng thanh toán gửi webhook về backend. 8. Backend xác minh webhook và đồng bộ trạng thái gói hội viên. 9. Hệ thống cho phép người dùng truy cập nội dung giới hạn theo tier đã đăng ký.
Luồng sự kiện thay thế	Người dùng hủy thanh toán: Cổng thanh toán chuyển người dùng về trang hủy, gói hội viên không được kích hoạt. Webhook không hợp lệ: Backend từ chối sự kiện và ghi log lỗi. Tier bị archive hoặc không tồn tại: Backend từ chối tạo checkout session. Gói hội viên quá hạn hoặc bị hủy: Người dùng mất quyền truy cập nội dung giới hạn sau khi hệ thống đồng bộ trạng thái.
Hậu điều kiện	Nếu thanh toán thành công, gói hội viên nội bộ ở trạng thái hoạt động và quyền truy cập nội dung giới hạn được mở cho người dùng.

2.3.6 Đặc tả use case Kiểm duyệt báo cáo và Khiếu nại

Bảng 2.7 mô tả chi tiết đặc tả use case Kiểm duyệt báo cáo và Khiếu nại.

Bảng 2.7: Đặc tả use case Kiểm duyệt báo cáo và Khiếu nại

Use Case ID	UC-06
Tên Use Case	Kiểm duyệt báo cáo và gửi khiếu nại
Tác nhân	Người dùng, Người sáng tạo, Moderator, Admin
Tiền điều kiện	Có tác phẩm bị báo cáo, bị hệ thống đánh dấu hoặc được người sáng tạo gửi khiếu nại; quản trị viên đã đăng nhập và có quyền xử lý.
Tóm tắt	Điều phối viên xem xét báo cáo hoặc khiếu nại, sau đó duyệt, từ chối, cảnh báo hoặc áp dụng biện pháp xử lý đối với tác phẩm và người dùng liên quan.
Luồng sự kiện chính	1. Moderator hoặc Admin truy cập trang kiểm duyệt. 2. Hệ thống hiển thị danh sách tác phẩm bị đánh dấu hoặc báo cáo đang chờ xử lý. 3. Điều phối viên mở chi tiết tác phẩm, xem ảnh, metadata lỗi, pHash liên quan, báo cáo và lịch sử xử lý. 4. Điều phối viên đưa ra quyết định duyệt hoặc từ chối. 5. Backend cập nhật trạng thái tác phẩm và trạng thái báo cáo. 6. Nếu cần, backend tạo cảnh báo cho người dùng hoặc cập nhật trạng thái khóa tài khoản. 7. Hệ thống ghi nhật ký hệ thống cho thao tác quản trị. 8. Người sáng tạo hoặc người báo cáo nhận được kết quả xử lý qua giao diện hoặc thông báo.
Luồng sự kiện thay thế	Người sáng tạo gửi khiếu nại: Nếu tác phẩm ở trạng thái cần xử lý, hệ thống chuyển tác phẩm sang trạng thái đang xem xét lại. Bằng chứng chưa đủ: Điều phối viên có thể giữ báo cáo ở trạng thái chờ xử lý hoặc yêu cầu thông tin bổ sung. Người dùng vi phạm nhiều lần: Điều phối viên có thể cảnh báo, khóa tạm thời hoặc khóa dài hạn tùy mức độ. Không đủ quyền: Backend từ chối yêu cầu nếu tài khoản không có vai trò phù hợp.
Hậu điều kiện	Báo cáo được cập nhật trạng thái; tác phẩm được duyệt, từ chối hoặc tiếp tục chờ xử lý; thao tác quản trị được ghi nhận trong audit log.

2.4 Yêu cầu phi chức năng

Bên cạnh yêu cầu chức năng, hệ thống cần đáp ứng các yêu cầu phi chức năng để có thể vận hành ổn định trong bối cảnh xử lý dữ liệu đa phương tiện và nhiều vai trò người dùng.

Yêu cầu về hiệu năng. Các API phục vụ thao tác đọc phổ biến như xem danh sách tác phẩm, tìm kiếm, xem chi tiết tác phẩm và xem hồ sơ nghệ sĩ cần có thời gian phản hồi ngắn trong điều kiện tải thông thường. Các tác vụ nặng như thay đổi kích thước ảnh, tạo thumbnail, kiểm duyệt NSFW, phát hiện trùng lặp và sinh embedding không được thực hiện trực tiếp trong request chính mà phải chuyển

sang hàng đợi nền. Tìm kiếm toàn văn cần sử dụng chỉ mục Meilisearch; tìm kiếm ngữ nghĩa cần sử dụng cột vector và cache truy vấn khi phù hợp. Bộ đếm lượt xem và cập nhật thống kê cần được kiểm soát tần suất hoặc gom lô để tránh ghi cơ sở dữ liệu quá dày.

Yêu cầu về độ tin cậy. Hệ thống phải lưu được trạng thái trung gian của tác phẩm trong quá trình xử lý, ví dụ đang xử lý, đã xuất bản, thất bại, cần xử lý hoặc đang xem xét lại. Nếu một ảnh trong tác phẩm bị lỗi, hệ thống cần lưu lỗi ở cấp ảnh thay vì làm mất toàn bộ dữ liệu tác phẩm. Worker cần ghi log lỗi rõ ràng, cập nhật tiến độ và đảm bảo cơ sở dữ liệu không rơi vào trạng thái mâu thuẫn. Các sự kiện thanh toán phải được đồng bộ qua webhook đã xác minh chữ ký, không chỉ dựa vào lệnh chuyển hướng từ frontend.

Yêu cầu về bảo mật và phân quyền. Các API yêu cầu đăng nhập phải được bảo vệ bằng cơ chế xác thực token. Các API quản trị phải kiểm tra vai trò như Moderator, Admin hoặc Super Admin. Người dùng chỉ được chỉnh sửa tác phẩm của chính mình, chỉ được xem nội dung giới hạn khi có gói hội viên phù hợp và chỉ được thao tác với bộ sưu tập của bản thân. Các webhook thanh toán phải được xác minh bằng chữ ký. Hệ thống cần lưu audit log cho các thao tác quản trị quan trọng như duyệt tác phẩm, từ chối tác phẩm, khóa người dùng, đổi vai trò và xử lý yêu cầu rút tiền.

Yêu cầu về tính dễ dùng. Giao diện tải lên cần hiển thị rõ trạng thái xử lý để người sáng tạo biết tác phẩm đang ở bước nào. Khi ảnh bị đánh dấu NSFW hoặc trùng lặp, hệ thống cần cung cấp lý do đủ rõ để người dùng có thể sửa hoặc gửi khiếu nại. Trang tìm kiếm cần cho phép kết hợp từ khóa, tag, bộ lọc và sắp xếp. Trang gói hội viên cần trình bày rõ giá, quyền lợi và trạng thái đăng ký. Trang quản trị cần ưu tiên khả năng quét nhanh danh sách báo cáo, tác phẩm bị đánh dấu và lịch sử xử lý.

Yêu cầu về khả năng bảo trì. Mã nguồn backend cần được chia theo module nghiệp vụ như xác thực, người dùng, tác phẩm, tìm kiếm, hàng đợi, thanh toán, kiểm duyệt và quản trị. Các tích hợp bên ngoài như Cổng thanh toán, Clerk, Meilisearch, Redis và storage cần được đóng gói trong service riêng để giảm phụ thuộc trực tiếp. Cơ sở dữ liệu cần được mô hình hóa rõ quan hệ giữa người dùng, tác phẩm, ảnh, tag, gói hội viên và báo cáo. Những logic có khả năng thay đổi như ngưỡng NSFW, trọng số xếp hạng hoặc giới hạn tìm kiếm nên được đặt trong cấu hình hoặc bảng thiết lập hệ thống.

Yêu cầu về khả năng mở rộng. Kiến trúc modular monolith cần cho phép mở rộng theo chiều ngang ở những điểm có tải cao, đặc biệt là worker xử lý ảnh và dịch

vụ tìm kiếm. Redis và BullMQ cần hỗ trợ nhiều worker tiêu thụ cùng một hàng đợi khi số lượng ảnh tăng. Hệ thống tìm kiếm cần có cơ chế đồng bộ lại chỉ mục khi Meilisearch bị lệch so với cơ sở dữ liệu. Cơ chế tìm kiếm vector ban đầu có thể đặt trong PostgreSQL với pgvector; khi dữ liệu tăng rất lớn, hệ thống cần có khả năng tách sang hạ tầng vector search chuyên dụng mà không làm thay đổi quá nhiều giao diện người dùng.

Yêu cầu về dữ liệu và lưu trữ. Cơ sở dữ liệu PostgreSQL là nguồn dữ liệu chính cho thông tin người dùng, tác phẩm, tương tác, báo cáo, thanh toán và cấu hình. File ảnh cần được lưu ở storage riêng; cơ sở dữ liệu chỉ lưu URL, metadata và trạng thái xử lý. Với mỗi ảnh, hệ thống cần lưu được URL bản gốc, bản xem trước, thumbnail, ảnh làm mờ, pHash, embedding và metadata lỗi nếu có. Dữ liệu thanh toán cần lưu đủ để đối soát, nhưng không lưu thông tin thẻ nhạy cảm vì phần này do cổng thanh toán quản lý.

Yêu cầu về khả năng quan sát. Hệ thống cần cung cấp nhật ký log cho các lỗi xử lý ảnh, lỗi đồng bộ Meilisearch, lỗi webhook thanh toán, lỗi xác thực và lỗi truy vấn AI search. Các thao tác quản trị phải có audit log. Các chỉ số như số lượng tác phẩm, người dùng, báo cáo, lượt xem, lượt thích, doanh thu và payout cần được tổng hợp để hỗ trợ trang tổng quan quản trị.

Tóm lại, chương này đã khảo sát các nhóm hệ thống tương tự, phân tích các hạn chế còn tồn tại và xác định các nhóm chức năng cần thiết cho nền tảng chia sẻ ảnh trực tuyến. Chương này cũng đã xây dựng use case tổng quát, phân rã các use case mức cao, mô tả hai quy trình nghiệp vụ quan trọng và đặc tả sáu use case tiêu biểu. Các yêu cầu phi chức năng được tổng hợp ở cuối chương sẽ là căn cứ để thiết kế kiến trúc, thiết kế cơ sở dữ liệu và triển khai hệ thống trong các chương tiếp theo.

CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

Chương này trình bày chi tiết về các nền tảng lý thuyết và công nghệ được lựa chọn để xây dựng hệ thống chia sẻ ảnh trực tuyến. Để đảm bảo tính nhất quán với các phân tích và yêu cầu đã được định nghĩa ở Chương 2, mỗi công nghệ được trình bày trong chương này đều gắn liền với một bài toán cụ thể. Việc lựa chọn công nghệ được thực hiện thông qua quá trình so sánh với các giải pháp thay thế, từ đó làm nổi bật tính ưu việt và độ phù hợp của công nghệ được chọn đối với bài toán đặt ra trong đề án.

3.1 Công nghệ phát triển ứng dụng Frontend và Backend

3.1.1 Kiến trúc Backend với NestJS

Theo phân tích ở Chương 2, hệ thống cần một kiến trúc vững chắc để xử lý nhiều luồng nghiệp vụ phức tạp như quản lý người dùng, tác phẩm, tìm kiếm, và thanh toán. Kiến trúc của hệ thống cần có tính module hóa cao để dễ dàng bảo trì và mở rộng.

Để giải quyết yêu cầu này, NestJS được lựa chọn làm framework phát triển tầng Backend [1]. NestJS là một framework dành cho Node.js được thiết kế xoay quanh ngôn ngữ TypeScript, áp dụng triệt để các mẫu thiết kế như Dependency Injection (Tiêm phụ thuộc) và Decorator.

Các lựa chọn thay thế phổ biến bao gồm Express.js và Spring Boot. Express.js rất nhẹ và linh hoạt, nhưng hoàn toàn thiếu các quy chuẩn cấu trúc kiến trúc, dẫn đến mã nguồn dễ trở nên hỗn loạn khi dự án phình to. Spring Boot (Java) cung cấp kiến trúc doanh nghiệp rất tốt nhưng lại khác biệt ngôn ngữ với tầng Frontend (chủ yếu dùng JavaScript/TypeScript), làm giảm khả năng dùng chung mã nguồn và kiểu dữ liệu.

Việc chọn NestJS mang lại sự cân bằng hoàn hảo: nó duy trì được tốc độ phát triển nhanh của hệ sinh thái Node.js nhưng lại mang cấu trúc chặt chẽ của các framework doanh nghiệp. Việc chia hệ thống thành các module độc lập (như module người dùng, module tác phẩm, module thanh toán) giúp cô lập logic nghiệp vụ, hoàn toàn đáp ứng được thiết kế Use Case phân rã ở Chương 2.

3.1.2 Framework Frontend Next.js và Thư viện UI Mantine

Bài toán ở Chương 2 chỉ ra rằng hệ thống không chỉ yêu cầu một giao diện tương tác phong phú cho tính năng đăng tải và quản lý bảng điều khiển mà còn cần tối ưu hóa công cụ tìm kiếm (SEO) cho các trang chi tiết tác phẩm nhằm thu hút người dùng bên ngoài.

Next.js (dựa trên React) được lựa chọn làm nền tảng Frontend chính [2]. Các lựa chọn thay thế như React thuần (Client-Side Rendering) hoặc Vue.js. React thuần gặp nhược điểm lớn về SEO do nội dung trang không được kết xuất sẵn tại máy chủ, dẫn đến các bộ máy tìm kiếm khó thu thập dữ liệu ảnh và họa sĩ.

Next.js giải quyết bài toán này thông qua cơ chế Server-Side Rendering (Kết xuất phía máy chủ), giúp các trang công khai hiển thị dữ liệu ngay lập tức khi tải. Đi kèm với Next.js, thư viện Mantine UI được sử dụng để xây dựng giao diện. Khác với TailwindCSS yêu cầu tự viết quá nhiều lớp tiện ích, hoặc Material UI có phong cách thiết kế khá cũ kỹ, Mantine cung cấp một hệ thống thành phần giao diện hiện đại, tính năng tùy biến mạnh mẽ, rất phù hợp để thiết kế một nền tảng nghệ thuật yêu cầu tính thẩm mỹ cao.

3.2 Cơ sở dữ liệu và Quản lý trạng thái dữ liệu

3.2.1 Hệ quản trị cơ sở dữ liệu PostgreSQL

Như đã đề cập ở cấu trúc thực thể trong Chương 2, dữ liệu của hệ thống có tính quan hệ rất cao: một người dùng có thể sở hữu nhiều tác phẩm, một tác phẩm có nhiều ảnh, nhiều tag, và liên kết chặt chẽ với các giao dịch thanh toán định kỳ. Yêu cầu tính toàn vẹn dữ liệu là bắt buộc.

PostgreSQL được lựa chọn làm cơ sở dữ liệu chính [3]. So sánh với các giải pháp thay thế như MongoDB (cơ sở dữ liệu NoSQL) hoặc MySQL. MongoDB linh hoạt trong việc lưu trữ tài liệu không có cấu trúc cố định, nhưng rất yếu trong việc duy trì tính toàn vẹn khi thực hiện các giao dịch phức tạp liên quan đến nhiều bảng (ví dụ: vừa trừ tiền, vừa cập nhật trạng thái gói hội viên). MySQL tốt nhưng hệ sinh thái hỗ trợ mở rộng không mạnh mẽ bằng PostgreSQL, đặc biệt là sự thiếu hụt các tiện ích mở rộng cho tìm kiếm vector gốc. PostgreSQL cung cấp độ tin cậy tuyệt đối cho các quan hệ thực thể khắt khe của hệ thống, đồng thời hỗ trợ lưu trữ mảng và kiểu dữ liệu JSON linh hoạt khi cần thiết.

3.2.2 Công cụ ánh xạ dữ liệu Prisma ORM

Để tương tác với PostgreSQL từ mã nguồn TypeScript, Prisma ORM được sử dụng [4]. Trong lập trình truyền thống, lập trình viên thường viết các câu lệnh SQL thô, hoặc sử dụng các thư viện như TypeORM, Sequelize. Viết SQL thô dễ gây ra lỗi bảo mật, ví dụ như lỗi tiêm nhiễm mã độc (SQL Injection), và khó tái cấu trúc mã nguồn. TypeORM khá phổ biến nhưng cơ chế kiểm tra kiểu dữ liệu không thực sự hoàn hảo.

Prisma nổi bật nhờ cơ chế tự động sinh mã máy khách dựa trên tệp lược đồ. Điều này đảm bảo rằng mọi truy vấn tới cơ sở dữ liệu đều được kiểm tra lỗi cú pháp và

kiểu dữ liệu ngay trong quá trình viết code. Nhờ Prisma, các thao tác phức tạp như truy vấn tác phẩm kèm theo thông tin tác giả và số lượng bình luận được thực hiện một cách an toàn và ngắn gọn, giảm thiểu đáng kể lỗi phát sinh trong thời gian chạy.

3.3 Nền tảng Lý thuyết và Công nghệ Tìm kiếm

3.3.1 Công cụ tìm kiếm toàn văn Meilisearch

Chương 2 đặt ra yêu cầu hệ thống phải hỗ trợ tìm kiếm tác phẩm theo từ khóa một cách nhanh chóng, kết hợp bộ lọc phức tạp theo thể, độ phân giải, tỷ lệ ảnh, và có khả năng chịu lỗi chính tả của người dùng.

Meilisearch được lựa chọn để giải quyết bài toán tìm kiếm toàn văn này [5]. Các giải pháp thay thế bao gồm Elasticsearch và tìm kiếm Full-text gốc của PostgreSQL. Tìm kiếm gốc của PostgreSQL khá chậm khi dữ liệu lớn và không hỗ trợ xử lý lỗi chính tả tự nhiên. Elasticsearch là một giải pháp cấp độ doanh nghiệp cực kỳ mạnh mẽ, nhưng nó đòi hỏi cấu hình hạ tầng khổng lồ, tiêu tốn nhiều tài nguyên bộ nhớ, làm tăng chi phí vận hành không cần thiết cho một đồ án quy mô trung bình.

Meilisearch được viết bằng ngôn ngữ Rust, nổi bật với thiết kế tối giản, tiêu thụ cực ít tài nguyên nhưng mang lại tốc độ tìm kiếm dưới 50 mili giây. Tính năng bỏ qua lỗi gõ phím được bật mặc định, giúp trải nghiệm người dùng trên nền tảng nghệ thuật trở nên tự nhiên hơn khi họ tìm kiếm các tên họa sĩ hoặc các thể tiếng nước ngoài phức tạp.

3.3.2 Lý thuyết học biểu diễn và Tìm kiếm ngữ nghĩa với pgvector

Ngoài tìm kiếm từ khóa, bài toán khám phá nội dung ở Chương 2 chỉ ra rằng người dùng cần tìm kiếm ảnh bằng các mô tả ngữ nghĩa (ví dụ: một bức tranh hoàng hôn trên biển có cô gái đứng quay lưng) hoặc tìm kiếm qua ảnh phác thảo. Lời giải cho bài toán này nằm ở lý thuyết học biểu diễn đa phương thức, cụ thể là mô hình CLIP (Contrastive Language-Image Pretraining) [6].

Lý thuyết của CLIP là huấn luyện đồng thời một bộ mã hóa hình ảnh và một bộ mã hóa văn bản sao cho chúng ánh xạ cả hình ảnh và văn bản vào cùng một không gian toán học. Hệ thống sử dụng lý thuyết này để chuyển đổi từng bức ảnh được tải lên thành một vector số thực gồm 512 chiều. Khoảng cách Cosine giữa các vector này chính là thước đo độ tương đồng về mặt thị giác hoặc ngữ nghĩa. Khoảng cách càng nhỏ, hai bức ảnh càng giống nhau.

Để lưu trữ và truy vấn hàng triệu vector này, hệ thống sử dụng pgvector, một tiện ích mở rộng trực tiếp của PostgreSQL [7]. Giải pháp thay thế là sử dụng các

cơ sở dữ liệu vector chuyên dụng như Milvus hoặc Pinecone. Tuy nhiên, việc vận hành thêm một hệ thống cơ sở dữ liệu riêng biệt sẽ làm tăng độ phức tạp của hạ tầng, đòi hỏi phải đồng bộ hóa dữ liệu liên tục giữa cơ sở dữ liệu quan hệ và cơ sở dữ liệu vector. pgvector giải quyết triệt để vấn đề này bằng cách cho phép lưu trữ trực tiếp vector bên trong bảng dữ liệu của PostgreSQL, hỗ trợ truy vấn các bức ảnh có không gian ngữ nghĩa gần nhất (K-Nearest Neighbors) bằng câu lệnh SQL truyền thống kết hợp toán tử khoảng cách.

3.4 Xử lý nền và hàng đợi

3.4.1 Hàng đợi tin nhắn BullMQ và Bộ nhớ đệm Redis

Theo luồng nghiệp vụ đăng tải tác phẩm ở Chương 2, hệ thống phải thực hiện hàng loạt tác vụ nặng nề khi nhận được ảnh: tính toán băm nhận thức, kiểm duyệt trí tuệ nhân tạo, sinh ảnh thu nhỏ, sinh ảnh làm mờ và trích xuất vector ngữ nghĩa. Nếu thực hiện các tác vụ này đồng bộ trong vòng đời của một yêu cầu HTTP, hệ thống sẽ bị treo và người dùng phải chờ đợi rất lâu.

Để giải quyết, hệ thống áp dụng kiến trúc xử lý nền với hàng đợi tin nhắn BullMQ và hệ thống bộ nhớ đệm Redis [8]. Các lựa chọn thay thế mạnh mẽ hơn bao gồm Apache Kafka hoặc RabbitMQ. Kafka và RabbitMQ rất xuất sắc trong các hệ thống phân tán khổng lồ, nhưng chúng đòi hỏi kiến thức vận hành phức tạp và tài nguyên máy chủ lớn.

BullMQ là một hệ thống hàng đợi dựa trên Redis, cực kỳ nhẹ nhưng vẫn cung cấp đầy đủ các tính năng nâng cao cấp doanh nghiệp như: lập lịch tác vụ, xử lý lại khi thất bại, giới hạn tốc độ xử lý, và báo cáo tiến trình. Khi người dùng tải ảnh lên, Backend chỉ việc ghi thông tin vào cơ sở dữ liệu và ném một công việc vào hàng đợi BullMQ. Các trình làm việc chạy ngầm sẽ âm thầm nhận công việc từ Redis, xử lý toàn bộ các bước cắt xén, kiểm duyệt ảnh và sau đó gửi thông báo hoàn tất về giao diện. Mô hình này giúp luồng đăng tải ảnh trở nên mượt mà và hệ thống không bao giờ bị quá tải cục bộ.

3.5 Nền tảng Lý thuyết cho Kiểm duyệt và Xử lý Ảnh

3.5.1 Lý thuyết Băm nhận thức trong phát hiện trùng lặp

Để giải quyết bài toán phát hiện ảnh đạo nhái, đăng lại nhiều lần được định nghĩa ở Chương 2, hệ thống áp dụng lý thuyết Băm nhận thức.

Trong khoa học máy tính, các thuật toán băm truyền thống (như MD5, SHA-256) hoạt động ở cấp độ byte. Chỉ cần bức ảnh bị thay đổi 1 pixel, mã băm sẽ thay đổi hoàn toàn, khiến chúng trở nên vô dụng trong việc tìm ảnh trùng lặp nếu người dùng cố tình thay đổi kích thước hoặc nén ảnh. Ngược lại, lý thuyết pHash hoạt

động dựa trên đặc trưng cảm nhận thị giác của con người. Nó áp dụng biến đổi Cosine rời rạc (Discrete Cosine Transform - DCT) để trích xuất các tần số không gian thấp của ảnh, bỏ qua các chi tiết nhiễu. Kết quả là một chuỗi bit đại diện cho hình dáng tổng thể của ảnh.

Khi hai bức ảnh có nội dung hình ảnh tương tự nhau, mã pHash của chúng sẽ giống nhau ở hầu hết các bit. Hệ thống đo lường độ sai lệch này thông qua Khoảng cách Hamming. Bằng cách so sánh mã pHash của bức ảnh mới tải lên với toàn bộ mã pHash trong cơ sở dữ liệu, nếu khoảng cách Hamming nhỏ hơn một ngưỡng nhất định, hệ thống tự động gắn cờ báo cáo trùng lặp, giúp giảm thiểu tối đa rác dữ liệu.

3.5.2 Phân loại nội dung nhạy cảm bằng NSFWJS

Kiểm duyệt nội dung nhạy cảm là một yêu cầu bảo mật cộng đồng cốt lõi đã đề cập ở Chương 2. Hệ thống áp dụng thư viện NSFWJS, được xây dựng trên nền tảng TensorFlow.js.

Giải pháp thay thế là tích hợp các API kiểm duyệt đám mây của Google Vision hoặc AWS Rekognition. Tuy nhiên, việc sử dụng các dịch vụ đám mây sẽ phát sinh chi phí tính theo từng lượt gọi API, không khả thi cho một đồ án có lượng ảnh tải lên liên tục.

Lý thuyết đằng sau NSFWJS là việc sử dụng kiến trúc mạng nơ-ron tích chập (Convolutional Neural Networks - CNN), cụ thể là mô hình MobileNet. Điểm ưu việt của MobileNet là nó sử dụng các phép tích chập tách biệt theo chiều sâu (Depthwise Separable Convolution), giúp giảm đáng kể số lượng tham số và khối lượng tính toán. Nhờ lý thuyết tối ưu hóa này, mô hình trí tuệ nhân tạo có thể được nhúng trực tiếp vào trong môi trường Node.js của tầng Backend. Trình làm việc có thể tự thân phân tích điểm số nhạy cảm của từng bức ảnh và tự động làm mờ các bức ảnh vi phạm quy tắc cộng đồng một cách cục bộ, hoàn toàn miễn phí và bảo vệ quyền riêng tư dữ liệu.

3.6 Các Dịch vụ Tích hợp Bên ngoài

3.6.1 Quản lý danh tính và Xác thực với Clerk

Quản lý đăng nhập, cấp lại mật khẩu và bảo mật phiên truy cập là một phần không thể thiếu được mô tả ở mục Quản lý tài khoản (Chương 2). Việc tự xây dựng toàn bộ hệ thống xác thực (Tự tạo bảng mật khẩu, mã hóa Bcrypt, tạo mã thông báo JWT, viết giao diện quên mật khẩu) rất dễ dẫn đến các lỗ hổng bảo mật nghiêm trọng.

Hệ thống sử dụng Clerk làm dịch vụ cung cấp danh tính [9]. So với Auth0 hay

Firestore Authentication, Clerk được thiết kế đặc biệt tối ưu cho hệ sinh thái Next.js App Router và React. Clerk cung cấp sẵn các thành phần giao diện đăng nhập tuyệt đẹp và an toàn. Hệ thống Backend chỉ cần cấu hình bộ trung gian (Middleware) để xác minh tính hợp lệ của mã thông báo (token) do Clerk cấp phát, từ đó ánh xạ định danh người dùng vào cơ sở dữ liệu nội bộ. Sự tách biệt này giúp đảm bảo tuân thủ tiêu chuẩn bảo mật hiện đại.

3.6.2 Cổng thanh toán Stripe

Để hiện thực hóa tính năng Gói hội viên của họa sĩ yêu cầu trả phí định kỳ ở Chương 2, hệ thống tích hợp trực tiếp cổng thanh toán Stripe [10].

Lựa chọn thay thế phổ biến tại Việt Nam là VNPay hoặc Momo. Tuy nhiên, các cổng thanh toán nội địa này được thiết kế chủ yếu cho các giao dịch mua một lần. Chúng không hỗ trợ mạnh mẽ cho các nghiệp vụ đăng ký tự động trừ tiền hàng tháng gói hội viên - vốn là trái tim của hệ thống Gói hội viên.

Stripe là giải pháp hàng đầu thế giới về thanh toán định kỳ. Stripe cung cấp môi trường thử nghiệm hoàn hảo, hệ thống Webhook tức thời để báo cáo cho Backend mỗi khi giao dịch thành công. Thông qua Stripe, hệ thống có thể dễ dàng quản lý vòng đời của một gói hội viên: từ lúc người dùng bắt đầu thanh toán, tự động gia hạn tháng sau, cho đến khi họ quyết định hủy gói. Mọi thao tác này đều được đồng bộ chính xác về cơ sở dữ liệu PostgreSQL của hệ thống để phân quyền truy cập hình ảnh một cách tức thời.

CHƯƠNG 4. THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG

Chương này trình bày chi tiết về quá trình thiết kế, triển khai và đánh giá hệ thống chia sẻ ảnh trực tuyến. Nội dung bắt đầu từ việc thiết kế kiến trúc phần mềm, cấu trúc dữ liệu, thiết kế giao diện người dùng, cho đến các biểu đồ tuần tự mô tả luồng nghiệp vụ cốt lõi. Tiếp theo, chương sẽ mô tả các bước triển khai hệ thống trong thực tế sử dụng Docker. Cuối cùng, chương đưa ra các kết quả thử nghiệm và đánh giá hiệu năng hệ thống nhằm khẳng định hệ thống đáp ứng tốt các yêu cầu đã đặt ra.

4.1 Thiết kế kiến trúc hệ thống

4.1.1 Lựa chọn kiến trúc phần mềm

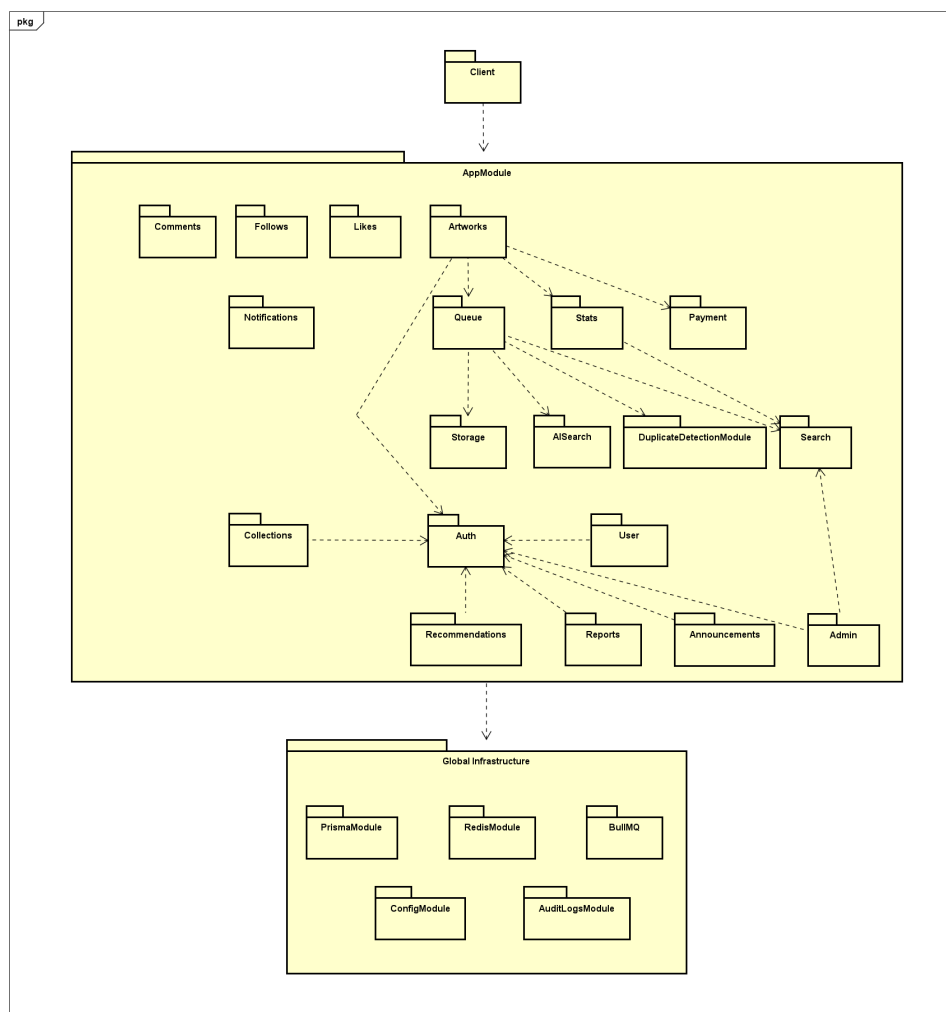
Kiến trúc phần mềm đóng vai trò nền tảng định hình toàn bộ sự phát triển, bảo trì và khả năng mở rộng của hệ thống. Đối với hệ thống chia sẻ ảnh trực tuyến trong đề án này, kiến trúc được lựa chọn là kiến trúc Modular Monolith.

Trong quá trình phân tích yêu cầu bài toán, hệ thống đứng trước sự lựa chọn giữa các mô hình kiến trúc truyền thống và hiện đại. Nhằm đảm bảo sự cân bằng giữa chi phí vận hành và khả năng mở rộng, hệ thống quyết định lựa chọn kiến trúc Modular Monolith kết hợp cơ chế hướng sự kiện Event-Driven. Kiến trúc này chia mã nguồn thành các mô-đun độc lập theo từng miền nghiệp vụ dựa trên khung làm việc NestJS. Chi tiết về lý do phân tích lựa chọn và cách thức hoạt động của kiến trúc này mang tính chất giải pháp đóng góp trọng tâm, do đó sẽ được trình bày chuyên sâu tại **Mục 5.1 của Chương 5**.

Bên cạnh các mô-đun nghiệp vụ, hệ thống sử dụng các mô-đun hạ tầng toàn cục để cung cấp các dịch vụ nền tảng. Các mô-đun này được đánh dấu @Global và tự động khả dụng cho toàn bộ hệ thống mà không cần khai báo phụ thuộc thủ công, bao gồm: PrismaModule hỗ trợ truy cập cơ sở dữ liệu, RedisModule phục vụ bộ nhớ đệm, ConfigModule hỗ trợ cấu hình môi trường, NotificationsModule quản lý thông báo thời gian thực, và AuditLogsModule phụ trách ghi nhật ký kiểm toán.

4.1.2 Thiết kế tổng quan

Hệ thống bao gồm tổng cộng 20 mô-đun nghiệp vụ và 5 mô-đun hạ tầng toàn cục. Biểu đồ gói UML trong Hình 4.1 minh họa các mô-đun chính và mối quan hệ phụ thuộc giữa chúng. Tất cả các mô-đun nghiệp vụ đều nằm bên trong AppModule, giao tiếp với nhau thông qua các dịch vụ được export, và cùng phụ thuộc vào tầng Hạ tầng toàn cục ở phía dưới.



Hình 4.1: Biểu đồ phụ thuộc mô-đun tổng quan của hệ thống

Các mô-đun được nhóm thành các cụm nghiệp vụ logic như sau:

1. **Cụm Nội dung cốt lõi:** Bao gồm mô-đun Artworks quản lý tác phẩm nghệ thuật là trung tâm của hệ thống với số lượng phụ thuộc lớn nhất, và mô-đun Collections phụ trách bộ sưu tập cá nhân.
2. **Cụm Danh tính và Truy cập:** Bao gồm mô-đun Auth phụ trách xác thực qua Clerk và mô-đun Users phụ trách quản lý hồ sơ.
3. **Cụm Tương tác xã hội:** Bao gồm mô-đun Likes, Comments và Follows. Thay vì phụ thuộc trực tiếp, cả ba mô-đun này giao tiếp với mô-đun Notifications thông qua kiến trúc hướng sự kiện. Thiết kế bất đồng bộ này giúp triệt tiêu hoàn toàn sự phụ thuộc, đảm bảo tính khả dụng cao cho hệ thống ngay cả khi dịch vụ gửi thông báo gặp sự cố.
4. **Cụm Thanh toán:** Mô-đun Payments tích hợp trực tiếp với Stripe [10] để quản lý vòng đời gói hội viên định kỳ.
5. **Cụm Xử lý và Thông minh:** Bao gồm mô-đun Queue cho đường ống xử lý

ảnh nền, Stats thống kê lượt xem, Search hỗ trợ tìm kiếm toàn văn qua Meilisearch, AiSearch hỗ trợ tìm kiếm ngữ nghĩa qua mô hình CLIP cục bộ và pgvector, DuplicateDetection phát hiện ảnh trùng lặp qua pHash, và Recommendations cung cấp gợi ý cá nhân hóa. Mô-đun Queue đóng vai trò trung tâm xử lý, kết nối với Storage, Search, DuplicateDetection và AiSearch để thực hiện đường ống xử lý ảnh hoàn chỉnh sau khi người dùng tải lên.

6. Cụm Quản trị: Bao gồm mô-đun Admin phục vụ bảng điều khiển quản trị, Reports xử lý báo cáo vi phạm, và Announcements quản lý thông báo hệ thống. Cả ba đều phụ thuộc vào Auth để xác thực quyền quản trị viên.

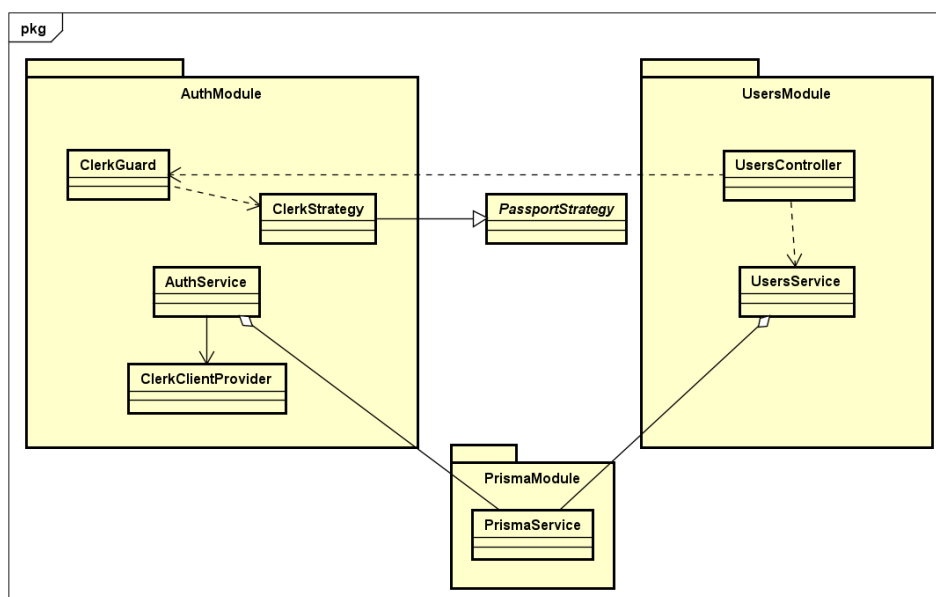
7. Hạ tầng toàn cục: Nằm ở dưới cùng biểu đồ, bao gồm PrismaModule kết nối PostgreSQL [3], RedisModule phục vụ bộ nhớ đệm, BullMQ cung cấp hàng đợi tin nhắn [8], ConfigModule hỗ trợ cấu hình môi trường, và AuditLogsModule phụ trách ghi nhật ký kiểm toán. Tất cả mô-đun nghiệp vụ đều ngầm phụ thuộc vào tầng này.

4.1.3 Thiết kế chi tiết gói

Sau khi xác định kiến trúc tổng thể, do hệ thống có hơn 25 mô-đun phức tạp đan xen nhau, phần này áp dụng nguyên lý thiết kế hướng miền (DDD) để phân tách hệ thống thành 5 cụm có liên quan mật thiết về mặt nghiệp vụ. Việc bóc tách này giúp làm rõ các ranh giới dịch vụ và chức năng trong kiến trúc Modular Monolith. Mỗi biểu đồ thiết kế gói dưới đây sẽ trình bày các lớp và mối quan hệ giữa chúng trong nội bộ nhóm.

1. Nhóm Định danh và Quản lý Người dùng

Nhóm này bao gồm AuthModule và UsersModule, đóng vai trò cửa ngõ kiểm soát quyền truy cập và lưu trữ hồ sơ thành viên của toàn hệ thống. Hình 4.2 minh họa thiết kế chi tiết của nhóm.

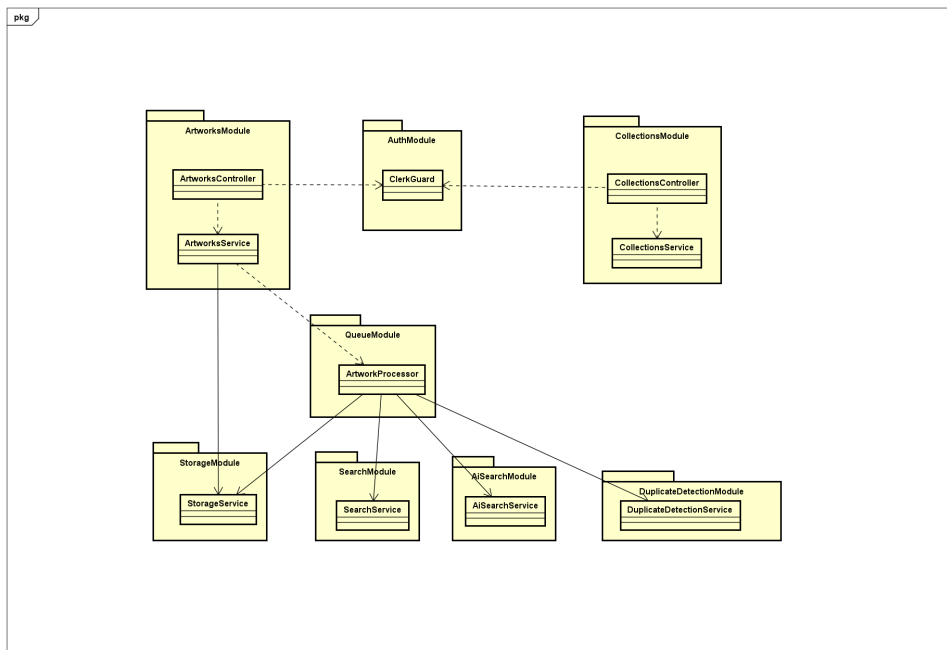


Hình 4.2: Thiết kế chi tiết nhóm Định danh và Quản lý Người dùng

Bên trong AuthModule, lớp ClerkStrategy có quan hệ kế thừa từ lớp trừu tượng PassportStrategy, triển khai chiến lược xác thực dựa trên JWT từ Clerk [9]. ClerkGuard phụ thuộc vào AuthService để kích hoạt luồng xác thực. Điểm đáng chú ý trong thiết kế là AuthService chịu trách nhiệm duy nhất cho việc tra cứu và đồng bộ người dùng từ Clerk thông qua PrismaService, giúp tách biệt hoàn toàn logic xác thực khỏi UsersService.

2. Nhóm Lỗi Nội dung và Xử lý Ảnh

Đây là "trái tim" của hệ thống, xử lý luồng tải lên và phân phối nội dung, bao gồm ArtworksModule, CollectionsModule, StorageModule và QueueModule. Hình 4.3 minh họa thiết kế.

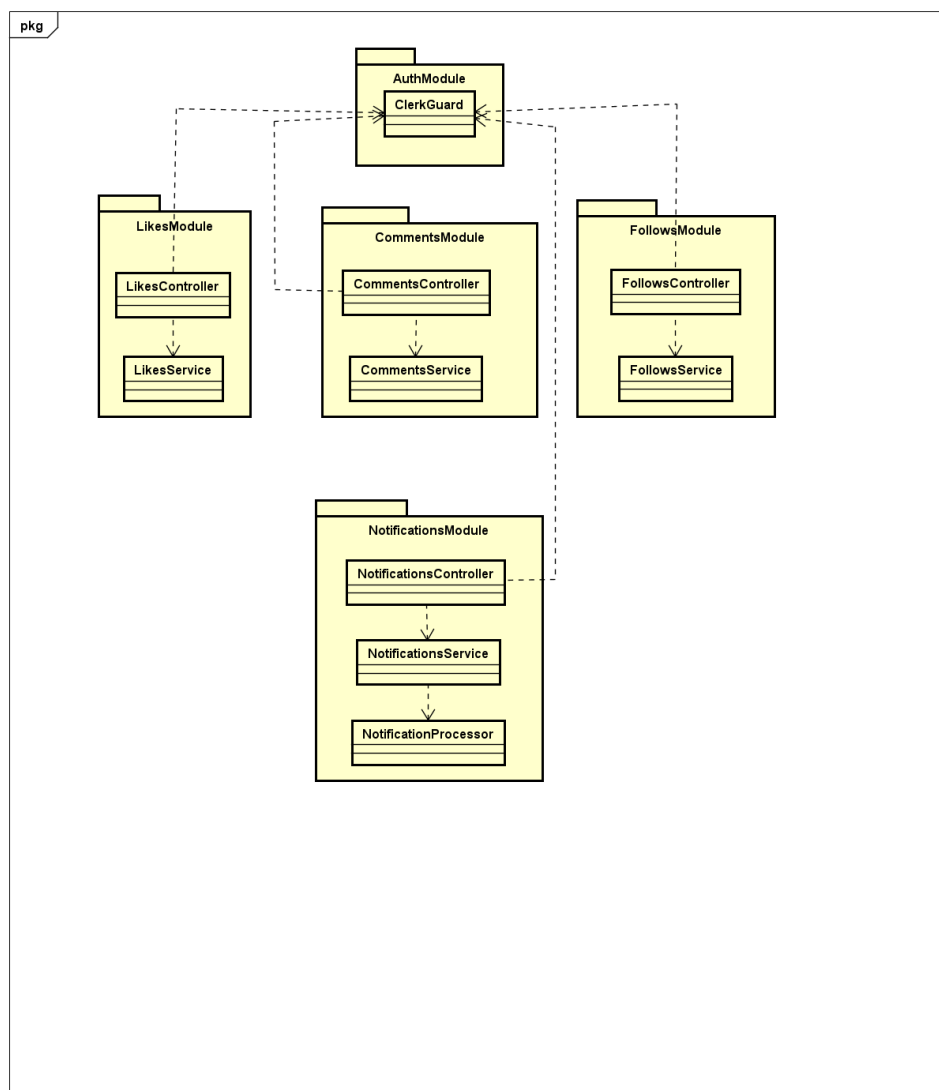


Hình 4.3: Thiết kế chi tiết nhóm Lỗi Nội dung và Xử lý Ảnh

ArtworksController và CollectionsController đều phụ thuộc vào ClerkGuard để bảo vệ các API. Lớp ArtworksService gọi đến lớp ArtworkProcessor (thuộc QueueModule) để khởi tạo quy trình xử lý nền không đồng bộ, giúp luồng chính của máy chủ không bị nghẽn. Từ ArtworkProcessor, các luồng dữ liệu được đẩy thẳng sang StorageService để lưu ảnh vào MinIO, cũng như đẩy đến các dịch vụ AI và tìm kiếm bên ngoài mà không làm ảnh hưởng đến thời gian phản hồi của ArtworksModule.

3. Nhóm Tương tác Xã hội và Cảnh báo

Nhóm này quản lý các hoạt động cộng đồng, bóc tách hoàn toàn logic tương tác khỏi phần lõi nội dung, bao gồm LikesModule, CommentsModule, FollowsModule và NotificationsModule. Hình 4.4 minh họa thiết kế.



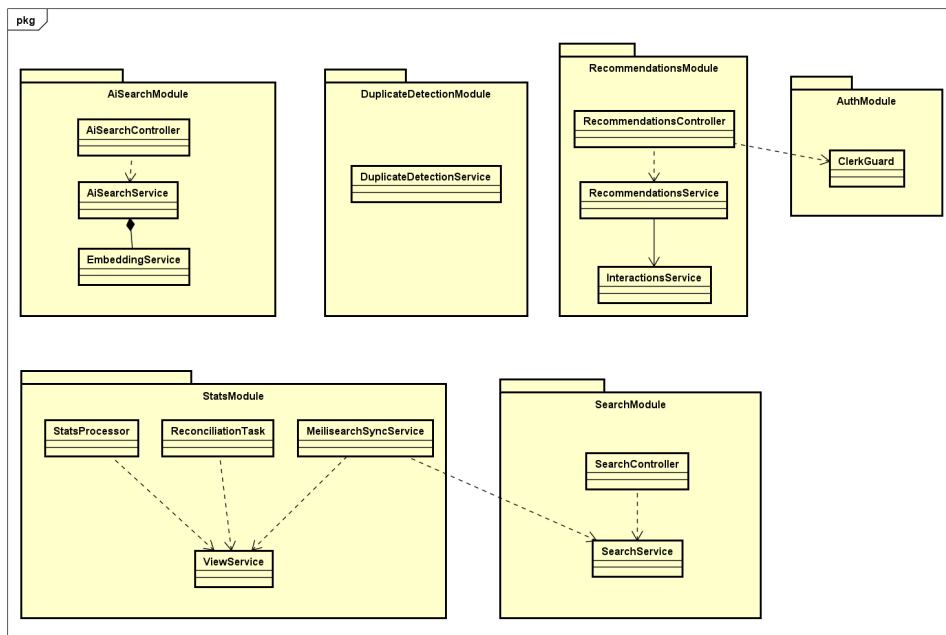
Hình 4.4: Thiết kế chi tiết nhóm Tương tác Xã hội và Cảnh báo

Đặc điểm kiến trúc nổi bật nhất của nhóm này là sự bóc tách hoàn toàn giữa các module tương tác và NotificationsModule. Thiết kế này tuân thủ chặt chẽ nguyên tắc Khớp nổi lỏng lẻo thông qua cơ chế hướng sự kiện và Hàng đợi tin nhắn. Cụ thể, thay vì gọi trực tiếp nhau, LikesService và CommentsService sẽ phát các sự kiện tương tác, còn FollowsService sẽ đẩy tín hiệu theo lô vào BullMQ. NotificationsModule đóng vai trò là một dịch vụ độc lập chuyên lắng nghe các sự kiện này để kích hoạt thông báo theo thời gian thực tới người dùng nhận. Cách tiếp cận này giúp các hành động tương tác (như thả tim, bình luận) được xử lý cực kỳ nhanh chóng mà không làm phình to logic nội bộ, đồng thời ngăn chặn rủi ro sập dây chuyền nếu dịch vụ thông báo gặp sự cố.

4. Nhóm Xử lý Dữ liệu Thông minh

Nhóm này tập trung vào bài toán khám phá nội dung bằng văn bản và trí tuệ nhân tạo, bao gồm SearchModule, AiSearchModule, DuplicateDetectionModule,

RecommendationsModule và StatsModule. Hình 4.5 minh họa thiết kế.

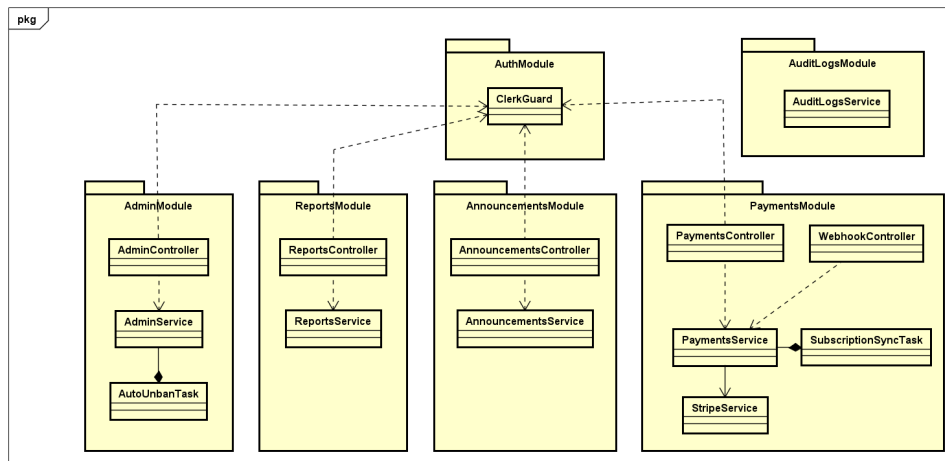


Hình 4.5: Thiết kế chi tiết nhóm Xử lý Dữ liệu Thông minh

Bên trong AiSearchModule, lớp EmbeddingService có quan hệ hợp thành (composition) với AiSearchService, tạo thành một cụm chuyên biệt gọi API CLIP cục bộ để trích xuất vector ngữ nghĩa hình ảnh. MeilisearchSyncService (thuộc StatsModule) phụ thuộc trực tiếp vào SearchService để đồng bộ dữ liệu tìm kiếm siêu tốc độ. Đây là cụm module tập hợp các công nghệ phức tạp nhất như pgvector và mã băm pHash để cung cấp trải nghiệm tìm kiếm ảnh chính xác và gợi ý cá nhân hóa.

5. Nhóm Quản trị, Vận hành và Doanh thu

Nhóm này chịu trách nhiệm quản lý hệ thống Backoffice (dành cho Admin) và xử lý luồng doanh thu, hoàn toàn tách biệt khỏi trải nghiệm lướt web thông thường của người dùng cuối. Nhóm bao gồm AdminModule, ReportsModule, AnnouncementsModule, PaymentsModule và AuditLogsModule. Hình 4.6 minh họa thiết kế.



Hình 4.6: Thiết kế chi tiết nhóm Quản trị, Vận hành và Doanh thu

PaymentsModule cung cấp PaymentsController cho người dùng trích xuất hóa đơn và WebhookController để lắng nghe sự kiện từ cổng thanh toán Stripe. Điểm chung của cụm nghiệp vụ này là sử dụng nhiều tác vụ chạy ngầm (CronJob) có quan hệ song song với Service nội bộ, điển hình như AutoUnbanTask thuộc AdminService (để tự động mở khóa tài khoản) và SubscriptionSyncTask thuộc PaymentsService (để đồng bộ gói hội viên).

4.2 Thiết kế chi tiết

4.2.1 Thiết kế giao diện

1. Đặc tả thông số màn hình

Hệ thống được thiết kế tối ưu hóa cho nền tảng Web Desktop. Giao diện hướng tới đối tượng người dùng sử dụng các thiết bị máy tính cá nhân hoặc laptop có kích thước màn hình từ 13 inch trở lên. Độ phân giải màn hình tối thiểu được hỗ trợ là 1366x768 pixel (chuẩn HD), tuy nhiên độ phân giải lý tưởng nhất để trải nghiệm trọn vẹn không gian nghệ thuật là 1920x1080 pixel (Full HD).

Đặc thù của hệ thống là nền tảng chia sẻ tranh ảnh, do đó yêu cầu về hiển thị màu sắc cực kỳ khắt khe. Ứng dụng hỗ trợ dải màu chuẩn sRGB với hơn 16.7 triệu màu (True Color 24-bit), đảm bảo các tác phẩm nghệ thuật, đặc biệt là ảnh có độ phân giải cao và ảnh do AI tạo ra, được hiển thị một cách trung thực, không bị rỗ màu hay sai lệch sắc độ.

2. Chuẩn hóa thiết kế giao diện

Để tạo ra một hệ thống nhất quán, chuyên nghiệp và thân thiện với người dùng, các thành phần giao diện được chuẩn hóa theo hệ thống thiết kế (Design System) cụ thể như sau:

- **Thiết kế nút (Buttons):** Tất cả các nút bấm trên hệ thống được chuẩn hóa với

góc bo tròn nhẹ (border-radius: 8px) tạo cảm giác hiện đại. Nút kêu gọi hành động chính (Primary Button - ví dụ: "Đăng tải", "Đăng ký gói") sử dụng nền màu đặc (Solid) là màu chủ đạo của dự án. Các nút hành động phụ (Secondary Button - ví dụ: "Hủy", "Xem thêm") chỉ sử dụng đường viền (Outline) hoặc nền xám nhạt để giảm sự chú ý, nhường trọng tâm cho các nút chính. Nút tương tác (Like, Comment) sử dụng biểu tượng (Icon) mỏng, chỉ đổi màu khi người dùng đưa chuột qua (Hover).

- **Điều khiển (Navigation):** Hệ thống sử dụng hai cơ chế điều khiển linh hoạt dựa theo ngữ cảnh. Đối với luồng người dùng lướt xem tranh (Discover), hệ thống áp dụng Thanh điều hướng ngang ghim cố định trên cùng, chứa thanh tìm kiếm lớn ở giữa. Đối với luồng Họa sĩ quản lý tác phẩm (Artist Dashboard), hệ thống chuyển sang Thanh điều khiển dọc bên trái để chứa tối đa không gian hiển thị các bảng biểu thống kê doanh thu và quản lý tệp tin.
- **Vị trí hiển thị thông điệp phản hồi:** Mọi phản hồi trạng thái từ hệ thống (như tải ảnh thành công, lỗi máy chủ, thông báo có người theo dõi mới) đều được chuẩn hóa hiển thị dưới dạng khung thông báo nhỏ (Toast Notification) bật lên ở **góc trên cùng bên phải** màn hình. Mỗi thông điệp sẽ tự động biến mất sau 3 giây để không làm gián đoạn trải nghiệm xem tranh.
- **Kiểu chữ (Typography):** Ứng dụng thống nhất sử dụng duy nhất một họ phông chữ không chân (sans-serif) là "Plus Jakarta Sans". Phông chữ này có độ nét cao, dễ đọc ở kích thước nhỏ gọn và mang lại cảm giác nghệ thuật hiện đại.

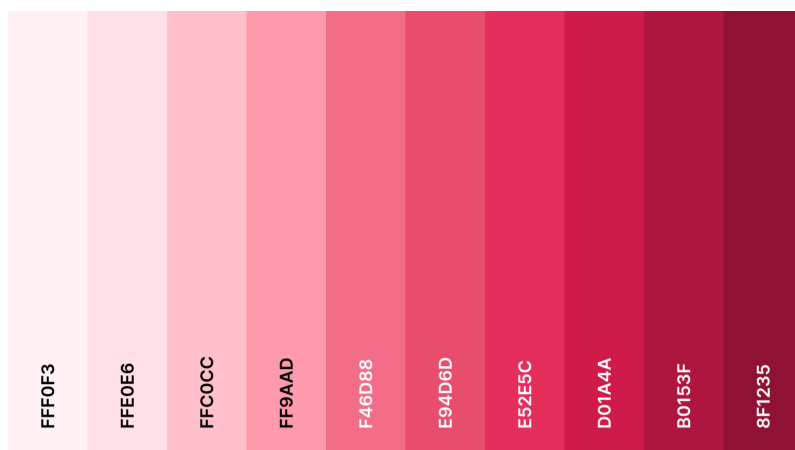
Phối màu (Color Palette):

Hệ thống sử dụng phong cách Chế độ sáng (Light Mode) thanh lịch nhằm tạo phong nền trung tính nhất, giúp mắt người xem không bị phân tâm khi thưởng thức các bức tranh nhiều màu sắc.



Hình 4.7: Bảng màu sử dụng cho nền

Bảng màu nền (Hình 4.7) bao gồm: màu #FFFFFF (Trắng tinh khiết) cho nền chính của trang web để tạo cảm giác không gian vô cực; màu #F8F9FA (Xám cực nhạt) cho các vùng thanh điều hướng; và màu #F3F4F6 cho phần nền của các thẻ (Card) chứa thông tin phụ.



Hình 4.8: Bảng màu chủ đạo (Primary Palette)

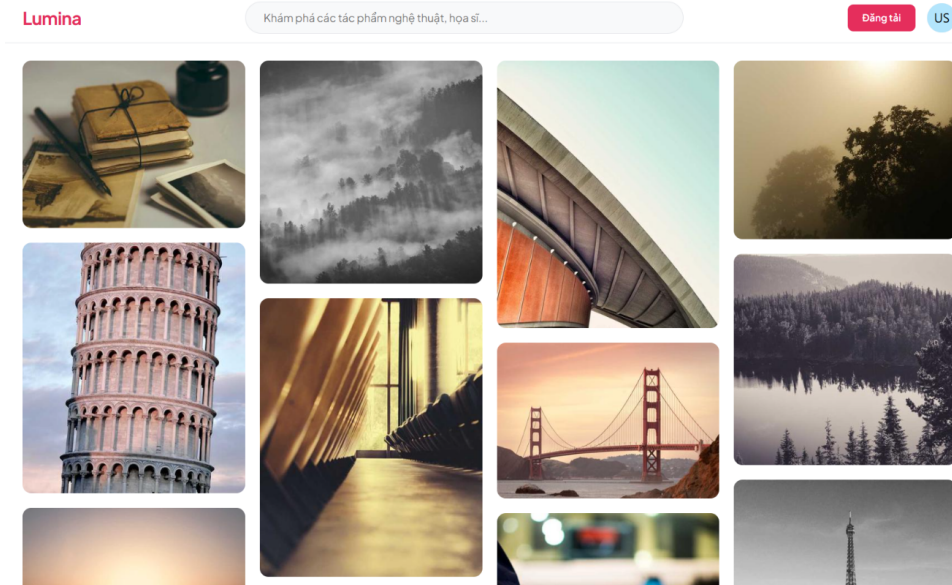
Bảng màu chủ đạo của thương hiệu (Hình 4.8) là màu Hồng dâu (Rose/Pink). Màu chính là #E52E5C, được chia làm 10 sắc độ khác nhau từ nhạt nhất (#FFF0F3) đến đậm nhất (#8F1235) để linh hoạt sử dụng cho các trạng thái hover, active của nút bấm hoặc các biểu đồ thống kê.

3. Hình ảnh minh họa giao diện

Dưới đây là thiết kế giao diện Mockup cho hai chức năng quan trọng nhất của hệ thống thể hiện rõ cách áp dụng các chuẩn hóa trên.

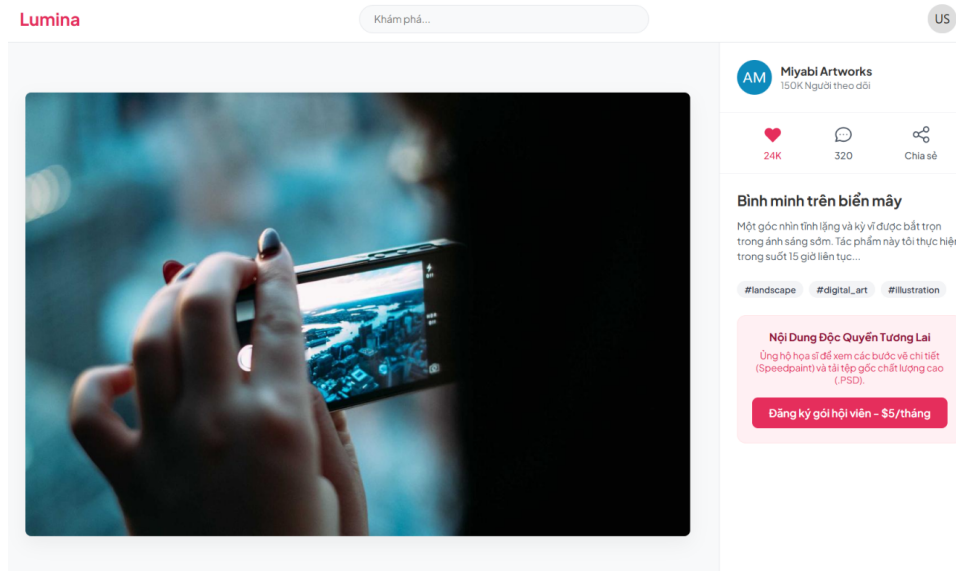
Hình 4.9 minh họa giao diện màn hình Trang chủ, nơi tập trung hiển thị tranh với bố cục lưới đa chiều, thanh Top Navbar màu trắng và các nút tìm kiếm được bo góc. Giao diện này tận dụng tối đa chiều rộng màn hình để người dùng có thể xem

được nhiều tác phẩm cùng lúc.



Hình 4.9: Thiết kế giao diện chức năng Khám phá (Trang chủ)

Hình 4.10 minh họa chức năng xem chi tiết một bức tranh. Không gian lớn nhất ở giữa dành cho tác phẩm trên nền xám nhạt #F8F9FA, lề phải là khu vực tương tác (Like, Comment) sử dụng các nút icon, bên dưới là nút kêu gọi "Đăng ký gói hội viên" nổi bật với màu Hồng dâu #E52E5C.

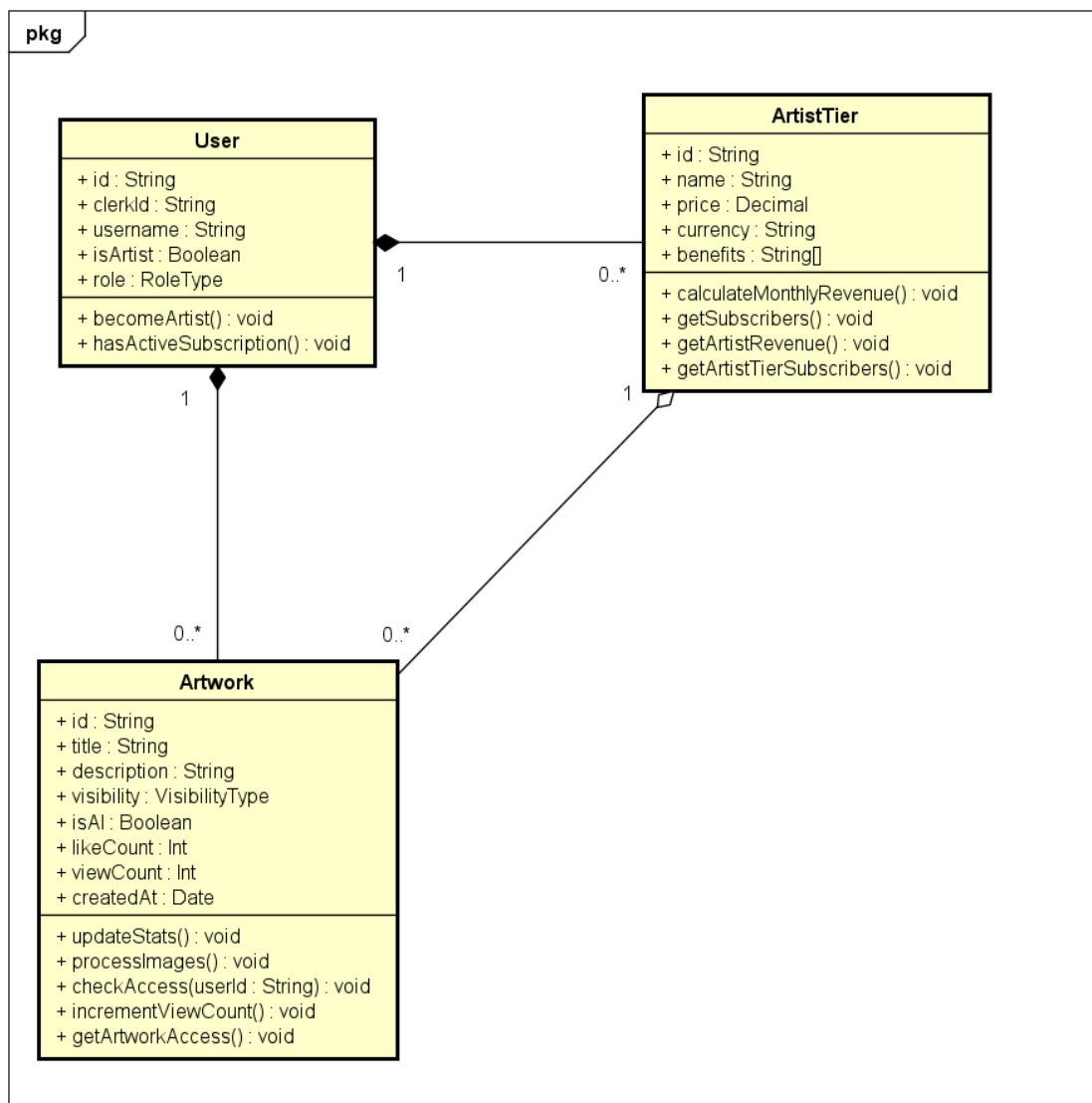


Hình 4.10: Thiết kế giao diện chức năng Chi tiết tác phẩm

4.2.2 Thiết kế lớp

Dựa trên cấu trúc kiến trúc đã định nghĩa, phần này đi sâu vào thiết kế cấu trúc bên trong của các lớp quan trọng nhất hệ thống. Lưu ý, các phương thức dưới đây mô tả hành vi logic gắn liền với thực thể trong thiết kế miền (Domain Design). Trong kiến trúc Modular Monolith thực tế của hệ thống, chúng được hiện thực hóa

thông qua các dịch vụ (Service) tương ứng. Do quy mô nền tảng có hàng chục lớp khác nhau, ba lớp chủ đạo mang tính đại diện cao nhất về mặt lưu trữ và nghiệp vụ được chọn để trình bày chi tiết bao gồm: Artwork, User và ArtistTier. Chi tiết biểu đồ lớp được thể hiện trong Hình 4.11.



Hình 4.11: Biểu đồ các lớp chủ đạo trong hệ thống

Lớp Artwork (Tác phẩm nghệ thuật) Đóng vai trò là thực thể trung tâm, lớp Artwork chứa mọi thông tin về một bài đăng ảnh.

- Thuộc tính: Bao gồm id (khóa chính), title (tiêu đề), description (mô tả), visibility (trạng thái hiển thị: công khai hoặc giới hạn gói hội viên), isAI (đánh dấu ảnh do trí tuệ nhân tạo tạo ra), likeCount và viewCount (thống kê tương tác).
- Phương thức/Hành vi gắn liền: incrementViewCount() để cập nhật lượt xem và getArtworkAccess() để xác thực quyền truy cập. Việc xử lý ảnh nền sẽ

được ủy quyền cho ArtworkProcessor.

Lớp User: Quản lý thông tin định danh và vai trò của cá nhân tham gia hệ thống.

- Thuộc tính: id, clerkId (mã định danh từ nhà cung cấp SSO), username (tên đăng nhập), isArtist (cờ xác định quyền đăng bài và tạo gói hội viên), role (vai trò quản trị).
- Phương thức/Hành vi gắn liền: becomeArtist() để nâng cấp tài khoản thành họa sĩ.

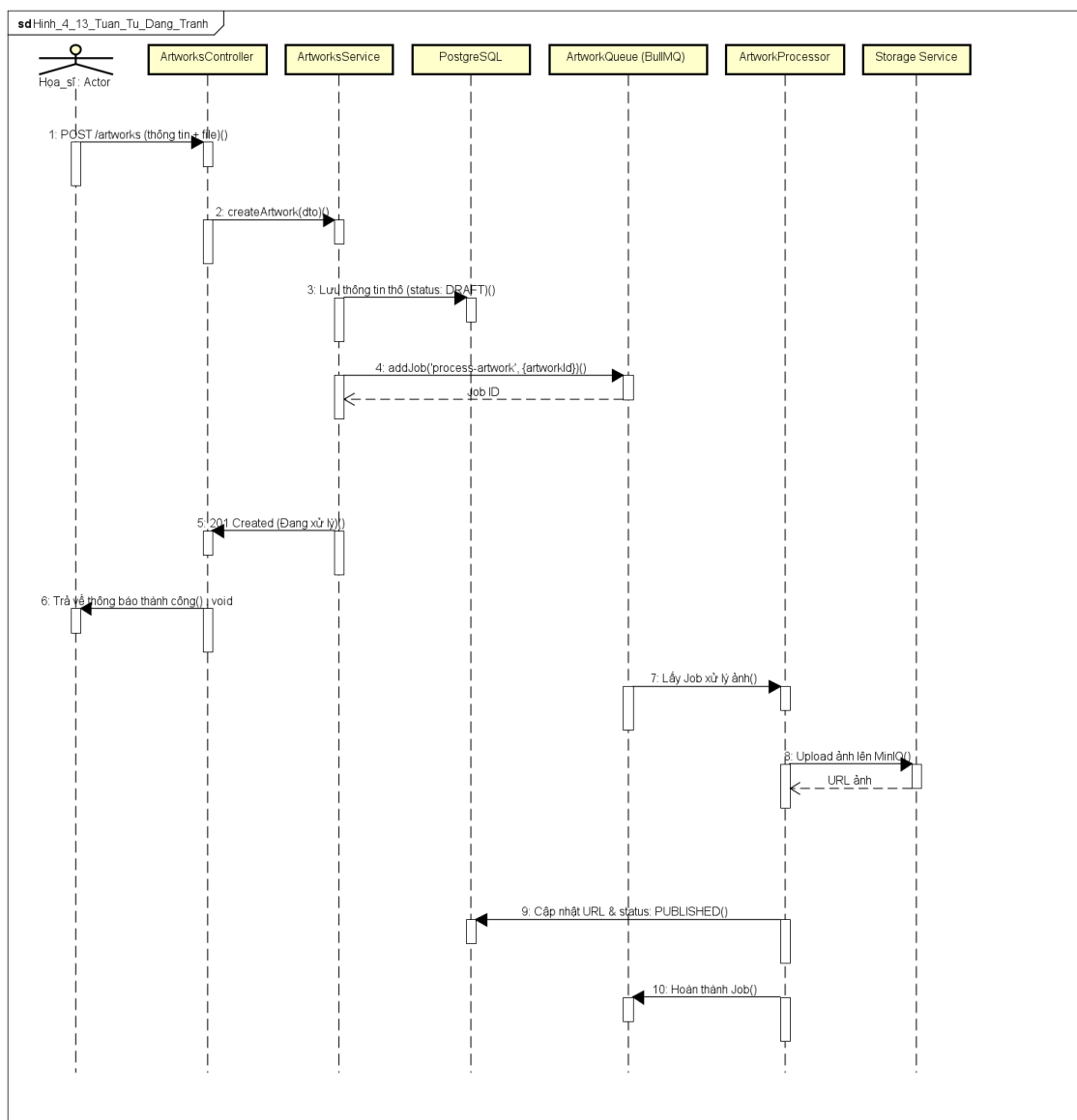
Lớp ArtistTier (Gói hội viên): Thực thể đặc thù cho bài toán kiểm tiền, định nghĩa cấu trúc một gói trả phí.

- Thuộc tính: id, name (tên gói), price (mức giá), currency (loại tiền tệ, mặc định là USD), benefits (danh sách đặc quyền dạng Json).
- Phương thức/Hành vi gắn liền: getArtistRevenue() để tính toán doanh thu, getArtistTierSubscribers() để lấy danh sách người theo dõi gói.

Để minh họa sự phối hợp giữa các đối tượng trong thời gian thực thi, ba biểu đồ trình tự sau đây sẽ mô tả luồng truyền thông điệp cho ba Use Case mang tính sống còn của hệ thống.

1. Biểu đồ trình tự: Đăng tải tác phẩm nghệ thuật

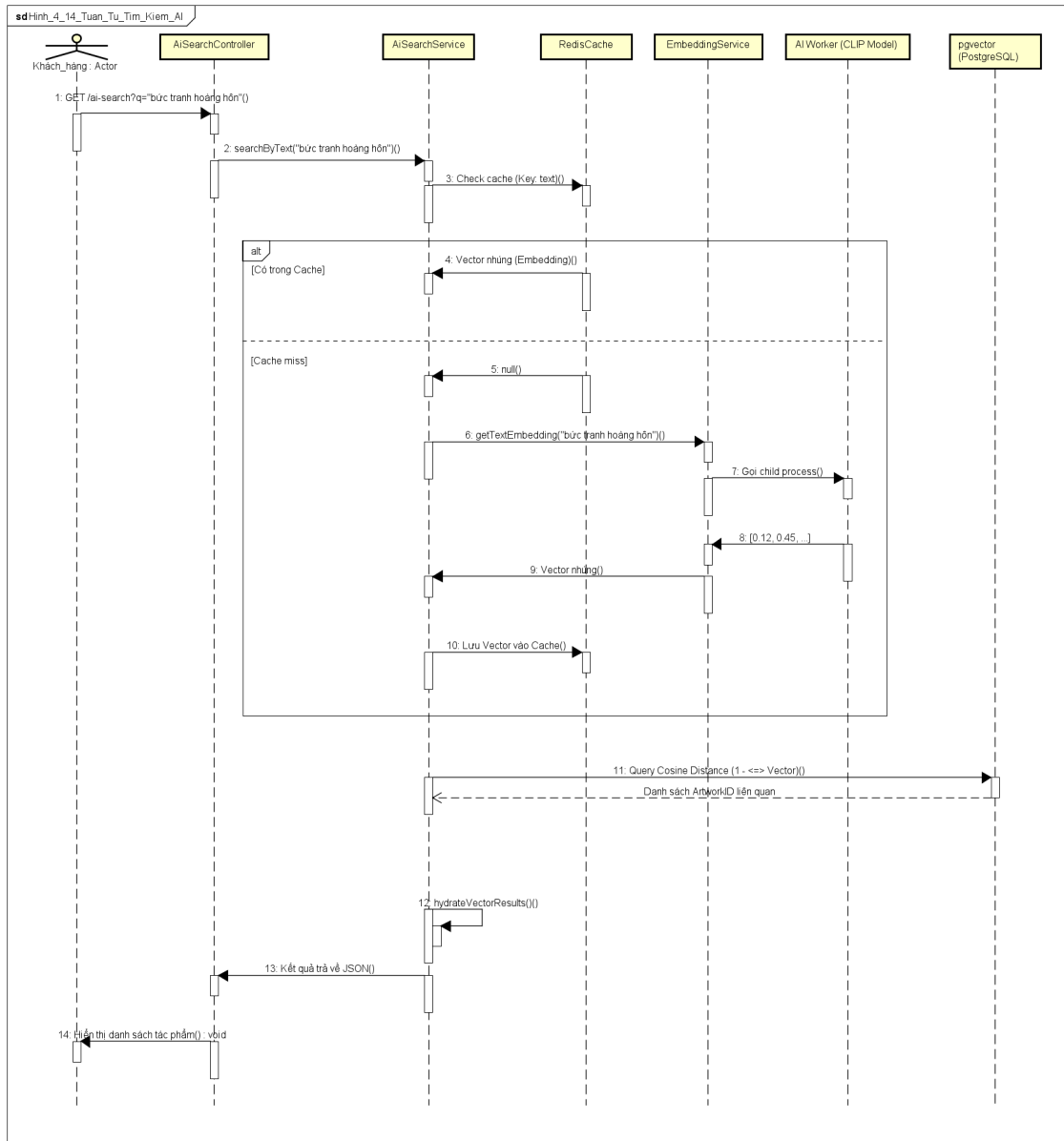
Luồng đăng tải tác phẩm được thiết kế theo cơ chế xử lý nền bất đồng bộ để tối ưu thời gian phản hồi. Biểu đồ trong Hình 4.12 minh họa cách hệ thống tiếp nhận yêu cầu và đẩy công việc vào hàng đợi, trong khi tiến trình nền âm thầm thực hiện việc tính toán mã băm pHash chống sao chép và đẩy ảnh lên máy chủ lưu trữ (MinIO). Chi tiết về thuật toán mã băm pHash và kiến trúc tối ưu hiệu năng bằng hàng đợi BullMQ sẽ được phân tích chuyên sâu lần lượt tại Mục 5.3 và Mục 5.4 của Chương 5.



Hình 4.12: Biểu đồ trình tự Use Case: Đăng tải tác phẩm nghệ thuật

2. Biểu đồ trình tự: Tìm kiếm hình ảnh bằng tìm kiếm ngữ nghĩa

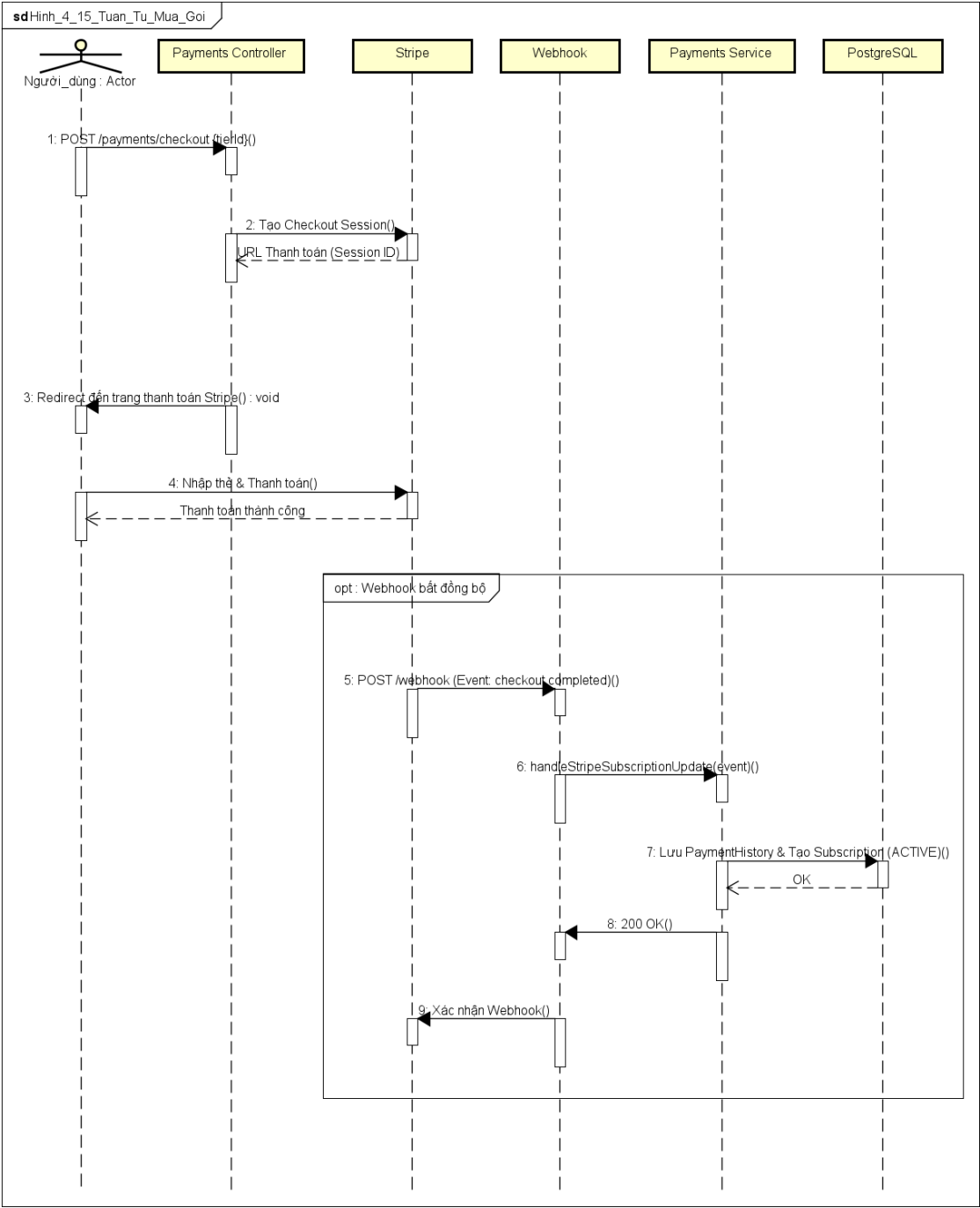
Tính năng cốt lõi tạo nên sự khác biệt của hệ thống là khả năng tìm kiếm bằng trí tuệ nhân tạo. Hình 4.13 mô tả luồng giao tiếp cơ bản bắt đầu từ khi người dùng nhập câu truy vấn tự nhiên, sau đó hệ thống tương tác với API của mô hình CLIP và cơ sở dữ liệu vector để tìm kiếm ảnh có độ tương đồng. Chi tiết về nguyên lý hoạt động của Hệ thống tìm kiếm ngữ nghĩa đa phương thức này sẽ được trình bày chuyên sâu tại Mục 5.2 của Chương 5.



Hình 4.13: Biểu đồ trình tự Use Case: Tìm kiếm bằng AI

3. Biểu đồ trình tự: Đăng ký gói hội viên của Họa sĩ

Đối với quy trình thanh toán, hệ thống phải phối hợp với cổng thanh toán bên thứ ba. Hình 4.14 mô tả kịch bản người dùng tiến hành thanh toán gói ArtistTier. PaymentsController sẽ tạo một phiên thanh toán thông qua Stripe API. Sau khi thanh toán thành công, một Webhook từ Stripe sẽ gửi ngược thông điệp về máy chủ để hệ thống tự động cấp quyền truy cập.



Hình 4.14: Biểu đồ trình tự Use Case: Đăng ký gói hội viên

4.2.3 Thiết kế cơ sở dữ liệu

Cơ sở dữ liệu của hệ thống được thiết kế theo mô hình quan hệ và được triển khai trên hệ quản trị PostgreSQL. Lựa chọn PostgreSQL là giải pháp tối ưu vì nó không chỉ hỗ trợ tính toàn vẹn dữ liệu cực tốt mà còn sở hữu extension "pgvector" - công cụ bắt buộc để lưu trữ và truy vấn các vector đa chiều phục vụ tính năng tìm kiếm bằng AI. Việc tương tác với cơ sở dữ liệu được thực hiện thông qua công cụ ORM Prisma, giúp chuẩn hóa kiểu dữ liệu từ mức thiết kế tới mã nguồn.

Biểu đồ Thực thể - Liên kết ERD trong Hình 4.15 minh họa cấu trúc vật lý của cơ sở dữ liệu, bao gồm các bảng cốt lõi và mối quan hệ giữa chúng.



Hình 4.15: Biểu đồ Thực thể - Liên kết (ERD) cốt lõi

Dựa trên biểu đồ thực thể liên kết (ERD) mức logic ở trên, hệ thống tiến hành chuẩn hóa để ánh xạ xuống cơ sở dữ liệu vật lý thông qua Prisma. Do quy mô hệ thống lớn (hơn 20 bảng), biểu đồ trên chỉ tập trung minh họa các thực thể cốt lõi. Dưới đây là đặc tả chi tiết thiết kế vật lý của 4 bảng dữ liệu quan trọng nhất mang tính nền tảng cho toàn bộ nghiệp vụ.

Bảng 4.1 đặc tả cấu trúc bảng User.

Bảng 4.1: Đặc tả cấu trúc bảng User (Quản lý người dùng)

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	Primary Key	Mã định danh duy nhất của người dùng
clerk_id	Varchar	Unique	ID liên kết với hệ thống xác thực Clerk
email	Varchar	Unique	Địa chỉ email đăng nhập
username	Varchar	Unique	Tên định danh hiển thị trên URL
stripe_customer_id	Varchar	Unique, Nullable	Mã khách hàng lưu trên hệ thống Stripe
role	Enum	Default 'USER'	Vai trò hệ thống (USER, ADMIN, MODERATOR)
is_artist	Boolean	Default false	Cờ đánh dấu tài khoản là Họa sĩ
is_banned	Boolean	Default false	Cờ khóa tài khoản khi vi phạm
created_at	DateTime	Default now()	Thời điểm tạo tài khoản

Bảng 4.2 đặc tả cấu trúc bảng Artwork.

Bảng 4.2: Đặc tả cấu trúc bảng Artwork (Bài đăng tác phẩm)

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	Primary Key	Mã định danh duy nhất của tác phẩm
title	Varchar	Not Null	Tiêu đề bài đăng
type	Enum	Default 'ILLUS.'	Loại hình (Minh họa, Manga,...)
status	Enum	Default 'PUBLISHED'	Trạng thái (Draft, Published,...)
rating	Enum	Default 'SAFE'	Phân loại nội dung (SAFE, NSFW)
is_ai	Boolean	Default false	Đánh dấu tác phẩm sinh ra bằng AI
visibility	Enum	Default 'PUBLIC'	Quyền xem (Công khai, Độc quyền)
required_tier_id	UUID	FK, Nullable	Trỏ đến bảng artist_tiers
author_id	UUID	Foreign Key	Trỏ đến bảng users

Bảng 4.3 đặc tả cấu trúc bảng ArtworkImage.

Bảng 4.3: Đặc tả cấu trúc bảng ArtworkImage (Tập dữ liệu hình ảnh)

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	Primary Key	Mã định danh tệp hình ảnh
url	Varchar	Not Null	Đường dẫn CDN của hình ảnh
original_url	Varchar	Nullable	Đường dẫn ảnh gốc bảo mật
status	Enum	Default 'PENDING'	Trạng thái xử lý ảnh
artwork_id	UUID	Foreign Key	Trỏ đến bảng artworks (1:N)
phash	Varchar	Indexed	Mã băm chống trùng lặp ảnh
embedding	Vector(512)	Indexed	Vector nhúng dùng cho AI Search
has_embedding	Boolean	Default false	Cờ xác nhận đã sinh xong vector

Bảng 4.4 đặc tả cấu trúc bảng Payment.

Bảng 4.4: Đặc tả cấu trúc bảng Payment (Giao dịch tài chính)

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	Primary Key	Mã định danh giao dịch
user_id	UUID	Foreign Key	Trỏ đến bảng users
provider	Enum	Not Null	Nền tảng (STRIPE)
provider_txn_id	Varchar	Indexed	Mã giao dịch từ cổng thanh toán
amount	Decimal(10,2)	Not Null	Số tiền thực tế giao dịch
currency	Varchar	Default 'USD'	Loại tiền tệ
type	Enum	Not Null	Loại hóa đơn (Mua gói, Nạp...)
status	Enum	Default 'PENDING'	Trạng thái hóa đơn
created_at	DateTime	Default now()	Thời điểm phát sinh hóa đơn

4.3 Xây dựng ứng dụng

4.3.1 Thư viện và công cụ sử dụng

Trong quá trình phát triển, hệ thống được xây dựng dựa trên các công nghệ, thư viện hiện đại và mã nguồn mở nhằm đảm bảo hiệu năng cao, dễ dàng mở rộng và bảo trì. Bảng 4.5 liệt kê các công cụ, ngôn ngữ lập trình, API và môi trường phát triển đã được sử dụng để xây dựng hệ thống.

Bảng 4.5: Danh sách thư viện và công cụ sử dụng

Mục đích	Công cụ (Phiên bản)	Địa chỉ URL
Môi trường phát triển	Visual Studio Code (1.8x)	https://code.visualstudio.com/
Mã nguồn Frontend	Next.js (16.2) / React (19.2)	https://nextjs.org/
Giao diện người dùng	Mantine (8.3)	https://mantine.dev/
Mã nguồn Backend	NestJS (11.1) / Node.js (20)	https://nestjs.com/
Xử lý hàng đợi	BullMQ (5.66)	https://bullmq.io/
Hệ quản trị CSDL	PostgreSQL (16) / pgvector	https://www.postgresql.org/
Công cụ truy vấn dữ liệu	Prisma ORM (6.19)	https://www.prisma.io/
Công cụ tìm kiếm	Meilisearch (0.54)	https://www.meilisearch.com/
Dịch vụ xác thực	Clerk API	https://clerk.com/
Cổng thanh toán	Stripe API	https://stripe.com/
Môi trường triển khai	Docker / Docker Compose	https://www.docker.com/

4.3.2 Kết quả đạt được

Kết quả đạt được của đồ án là một nền tảng mạng xã hội chia sẻ tác phẩm nghệ thuật hoàn chỉnh, hỗ trợ họa sĩ số đăng tải, quản lý, phân phối nội dung và xây dựng cộng đồng người hâm mộ. Hệ thống được đóng gói thành các dịch vụ độc lập theo kiến trúc Modular Monolith, bao gồm ba thành phần chính:

- **Frontend Web Application:** Giao diện người dùng tương tác trực quan, tối ưu hóa SEO và tốc độ tải trang bằng cơ chế Server-Side Rendering (SSR). Đóng vai trò là điểm tiếp xúc duy nhất của người dùng cuối.
- **Backend API Service:** Dịch vụ trung tâm xử lý nghiệp vụ cốt lõi, quản lý giao dịch thanh toán, phân quyền truy cập, tương tác cộng đồng và định tuyến dữ liệu.
- **AI Search Service:** Khối dịch vụ chuyên biệt đảm nhiệm việc tìm kiếm toàn văn bản siêu tốc, tìm kiếm theo ngữ nghĩa hình ảnh và phân tích vector dữ liệu.

Để minh họa cụ thể cho quy mô của hệ thống sau khi hoàn thiện, Bảng 4.6 thống kê các số liệu thực tế về mã nguồn và dung lượng đóng gói của ứng dụng.

Bảng 4.6: Thống kê số liệu kỹ thuật của hệ thống

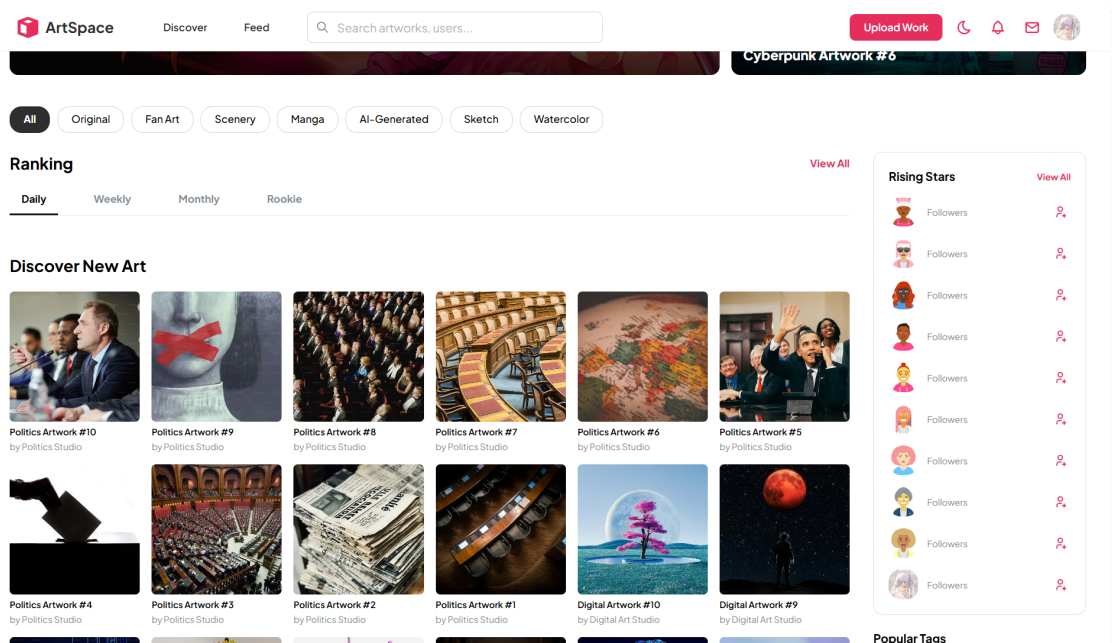
Thành phần / Chỉ số	Khối lượng	Đơn vị / Ghi chú
Tổng số dòng mã nguồn (LOC)	~35,200	Dòng lệnh (Type-Script/HTML/CSS)
Số lượng Module/Lớp/Gói	~145	Component Frontend và Module Backend
Dung lượng mã nguồn gốc	~5.8	Megabytes (MB)
Dung lượng Container Frontend	~150	Megabytes (MB) sau khi build
Dung lượng Container Backend	~210	Megabytes (MB) sau khi build

4.3.3 Minh họa các chức năng chính

Phần này giới thiệu một số giao diện nổi bật, thể hiện các chức năng quan trọng và mang lại trải nghiệm cốt lõi cho người dùng trên nền tảng.

1. Chức năng Khám phá (Trang chủ)

Trang chủ là nơi hiển thị các tác phẩm nổi bật, xu hướng mới và các nội dung được cá nhân hóa theo sở thích của người dùng. Giao diện được thiết kế dạng lưới kết hợp với hiệu ứng tải ảnh lưới để tối ưu hóa hiệu năng hiển thị hàng ngàn hình ảnh mượt mà.

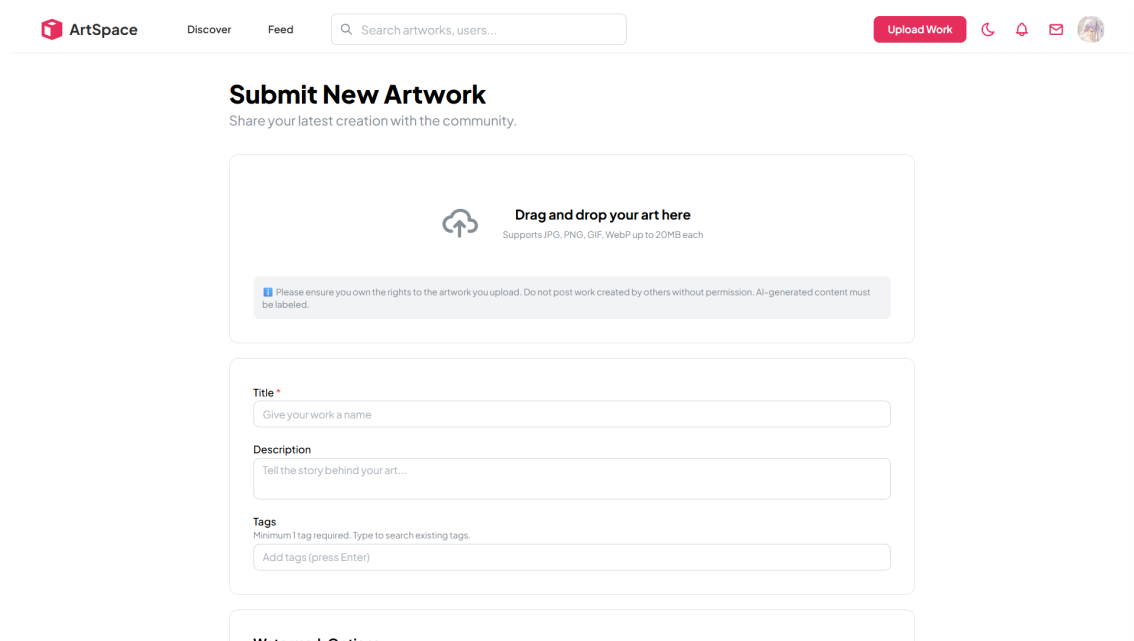


Hình 4.16: Giao diện chức năng Khám phá (Trang chủ)

2. Chức năng Đăng tải tác phẩm

Chức năng Đăng tải cho phép họa sĩ cập nhật thông tin về tác phẩm bao gồm:

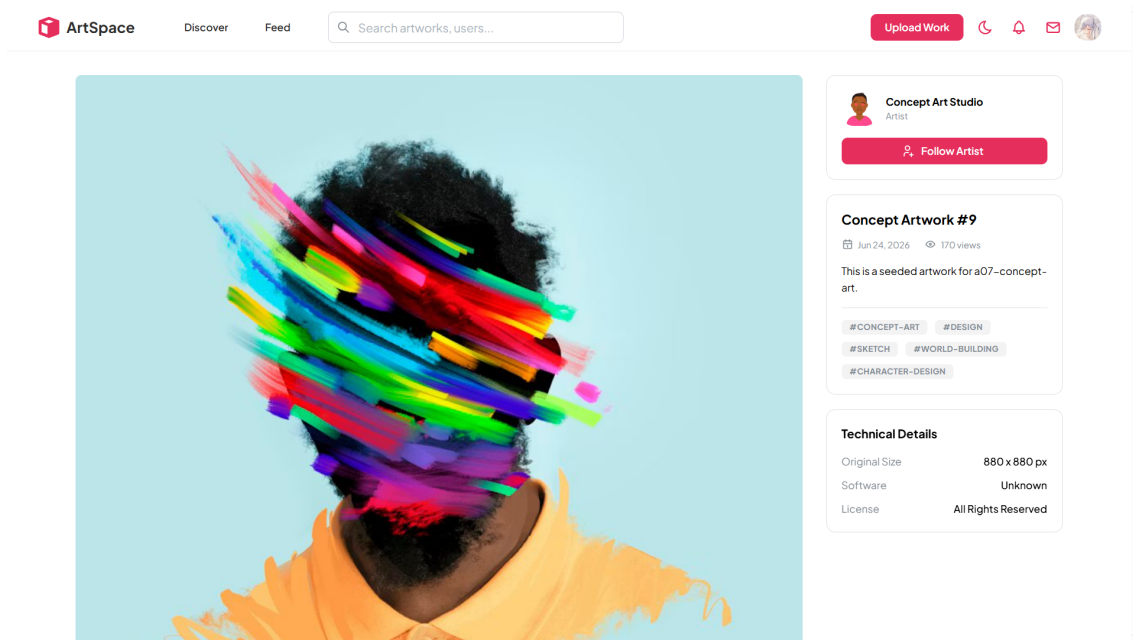
tiêu đề, mô tả, từ khóa, phân loại nội dung (bình thường hay nhạy cảm) và quyền truy cập (công khai hay giới hạn theo gói hội viên). Giao diện hỗ trợ kéo thả tệp tin trực quan và thanh tiến trình báo hiệu trạng thái tải lên máy chủ.

The image shows the 'Submit New Artwork' interface on the ArtSpace website. At the top, there's a navigation bar with the ArtSpace logo, 'Discover', 'Feed', a search bar, and an 'Upload Work' button. The main heading is 'Submit New Artwork' with a subtext 'Share your latest creation with the community.' Below this is a large drag-and-drop area with a cloud icon and text: 'Drag and drop your art here' and 'Supports JPG, PNG, GIF, WebP up to 20MB each'. A small blue box contains a disclaimer: 'Please ensure you own the rights to the artwork you upload. Do not post work created by others without permission. AI-generated content must be labeled.' Underneath are three input fields: 'Title' (with a red asterisk and placeholder 'Give your work a name'), 'Description' (with placeholder 'Tell the story behind your art...'), and 'Tags' (with placeholder 'Add tags (press Enter)' and a note 'Minimum 1 tag required. Type to search existing tags.'). At the bottom, there's a section for 'Watermark Options'.

Hình 4.17: Giao diện chức năng Đăng tải tác phẩm

3. Chức năng Chi tiết tác phẩm

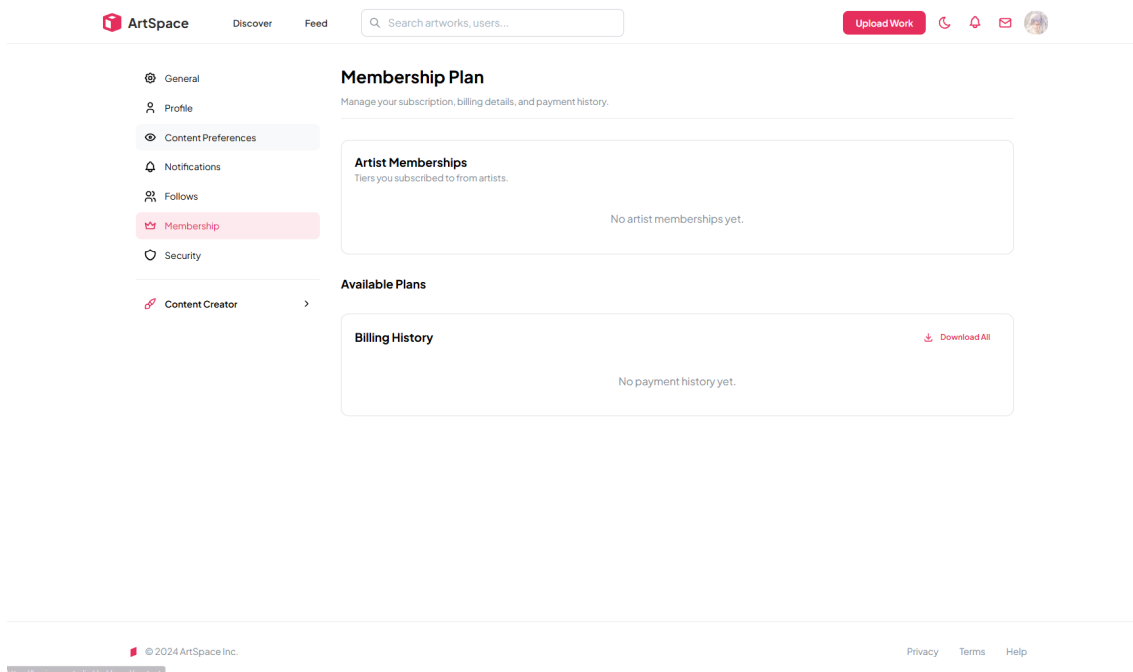
Màn hình Chi tiết tác phẩm không chỉ tập trung vào việc hiển thị hình ảnh chất lượng cao mà còn cung cấp khu vực tương tác cộng đồng. Người xem có thể yêu thích, lưu vào bộ sưu tập, chia sẻ và bình luận. Bên cạnh đó, hệ thống tích hợp khối đề xuất các tác phẩm tương tự ở phía dưới.



Hình 4.18: Giao diện chức năng Chi tiết tác phẩm

4. Chức năng Nâng cấp gói hội viên

Hỗ trợ người dùng đăng ký các gói hội viên hàng tháng để ủng hộ họa sĩ và truy cập các nội dung độc quyền. Giao diện tích hợp liền mạch với cổng thanh toán an toàn, hiển thị rõ ràng các quyền lợi tương ứng của từng gói và chính sách gia hạn tự động.



Hình 4.19: Giao diện chức năng Nâng cấp gói hội viên

4.4 Kiểm thử

Phần này trình bày quá trình thiết kế và thực thi các kịch bản kiểm thử nhằm đảm bảo chất lượng hệ thống. Các trường hợp kiểm thử được thiết kế chủ yếu dựa trên kỹ thuật Kiểm thử hộp đen với hai phương pháp cốt lõi: Phân tích giá trị biên và Phân vùng tương đương. Đồng thời, hệ thống áp dụng Kiểm thử tự động bằng Jest cho các service cốt lõi ở Backend để đảm bảo tính ổn định khi mở rộng.

Dưới đây là chi tiết kiểm thử cho ba chức năng quan trọng nhất có ảnh hưởng lớn đến luồng nghiệp vụ của nền tảng: Chức năng Đăng tải tác phẩm, Xác thực người dùng (Auth/Clerk Guard), và Tìm kiếm khám phá (Meilisearch). Chi tiết các trường hợp kiểm thử cho các chức năng khác (Thanh toán Stripe, Tương tác cộng đồng, Report) được đính kèm trong phần Phụ lục của tài liệu.

4.4.1 Kịch bản kiểm thử: Đăng tải tác phẩm

Chức năng này đòi hỏi sự kiểm soát nghiêm ngặt về quyền (chỉ họa sĩ mới được đăng tải), tính toàn vẹn của dữ liệu (bắt buộc có ảnh, từ khóa) và ràng buộc nghiệp vụ về nội dung trả phí. Bảng 4.7 mô tả kịch bản kiểm thử chức năng Đăng tải tác phẩm.

Bảng 4.7: Kịch bản kiểm thử chức năng Đăng tải tác phẩm

Mã TC	Dữ liệu đầu vào	Kết quả mong đợi	Kết quả thực tế	Trạng thái
TC01	Tài khoản có Role là USER (Không phải Họa sĩ) gọi API upload.	Hệ thống từ chối truy cập. Trả về lỗi ForbiddenException('Only artists can upload artworks').	Đúng như mong đợi.	PASS
TC02	Payload hợp lệ nhưng danh sách tags rỗng (tags: []).	Trả về lỗi BadRequestException('At least 1 tag is required').	Đúng như mong đợi.	PASS
TC03	Payload hợp lệ nhưng danh sách ảnh rỗng (images: []).	Trả về lỗi BadRequestException('At least 1 image is required').	Đúng như mong đợi.	PASS
TC04	Cài đặt quyền xem là TIER_GATED nhưng không truyền tham số tier_id.	Trả về lỗi BadRequestException('Gated artwork must have a required tier').	Đúng như mong đợi.	PASS
TC05	Dữ liệu hoàn toàn hợp lệ, đầy đủ tiêu đề, mô tả, ảnh gốc, tags.	Hệ thống lưu thành công vào Database, gửi job vào BullMQ để tạo pHash chống trùng lặp.	Đúng như mong đợi.	PASS

4.4.2 Kịch bản kiểm thử: Xác thực và Đồng bộ tài khoản

Chức năng này kết nối trực tiếp với Clerk API. Kiểm thử tập trung vào việc xử lý các token không hợp lệ và khả năng tự động đồng bộ hồ sơ người dùng mới vào hệ thống nội bộ. Bảng 4.8 mô tả kịch bản kiểm thử chức năng Xác thực tài khoản.

Bảng 4.8: Kịch bản kiểm thử chức năng Xác thực tài khoản

Mã TC	Dữ liệu đầu vào (Input)	Kết quả mong đợi	Kết quả thực tế	Trạng thái
TC06	Gọi API bảo mật nhưng không gắn Bearer Token ở HTTP Header.	Hệ thống chặn ngay lập tức. Trả về UnauthorizedException('No authentication token provided').	Đúng như mong đợi.	PASS
TC07	Token cung cấp có định dạng sai, hết hạn hoặc bị làm giả.	Trả về lỗi UnauthorizedException('Invalid token: no user ID').	Đúng như mong đợi.	PASS
TC08	Token hợp lệ nhưng tài khoản người dùng đã bị khóa (Banned).	Hệ thống từ chối cấp quyền, Frontend tự động chuyển hướng về trang bị chặn.	Đúng như mong đợi.	PASS
TC09	Token hợp lệ từ người dùng mới đăng nhập lần đầu tiên.	Hệ thống trích xuất Clerk ID, tự động tạo mới bản ghi người dùng vào cơ sở dữ liệu nội bộ.	Lỗi trễ đồng bộ (Timeout) khi tải avatar gốc từ Clerk.	FAIL
TC10	Token hợp lệ từ người dùng cũ đã tồn tại trong CSDL.	Bỏ qua bước tạo mới, tự động gắn user_id nội bộ vào request hiện tại.	Đúng như mong đợi.	PASS

4.4.3 Kịch bản kiểm thử: Tìm kiếm và Lọc tác phẩm

Chức năng sử dụng công cụ Meilisearch kết hợp Validator. Việc kiểm thử tập trung vào khả năng bắt lỗi định dạng tham số đầu vào và tính chính xác của kết quả lọc. Bảng 4.9 mô tả kịch bản kiểm thử chức năng Tìm kiếm tác phẩm.

Bảng 4.9: Kịch bản kiểm thử chức năng Tìm kiếm tác phẩm

Mã TC	Dữ liệu đầu vào (Input)	Kết quả mong đợi	Kết quả thực tế	Trạng thái
TC11	Truy vấn với tham số phân trang <code>page=abc</code> (không phải số nguyên).	Trả về lỗi Validator <i>page must be an integer number.</i>	Đúng như mong đợi.	PASS
TC12	Truy vấn với bộ lọc nội dung <code>rating=VIOLENCE</code> (ngoài Enum cho phép).	Trả về lỗi Validator <i>rating must be a valid enum value (ALL, SAFE, R18).</i>	Đúng như mong đợi.	PASS
TC13	Truy vấn từ khóa hợp lệ <code>q=Genshin, excludeAI=true</code> .	Hệ thống gọi Meilisearch, trả về danh sách tác phẩm liên quan và loại bỏ hoàn toàn ảnh AI.	Đúng như mong đợi.	PASS
TC14	Tìm kiếm theo ngữ nghĩa hình ảnh bằng ảnh tải lên (Vector Search).	AI Service chuyển đổi ảnh sang Vector 512 chiều, pgvector trả về ảnh tương đồng.	Độ tương đồng (Cosine) thấp khi dùng ảnh phác thảo mờ.	FAIL

4.4.4 Tổng kết kết quả kiểm thử

Toàn bộ quá trình đánh giá chất lượng hệ thống bao gồm 85 kịch bản tự động viết bằng Jest tích hợp sẵn trong NestJS và 40 kịch bản kiểm thử thủ công giả lập hành vi người dùng trên trình duyệt. Kết quả thực thi các trường hợp kiểm thử được tổng hợp trong Bảng 4.10:

Bảng 4.10: Tổng hợp kết quả thực thi các trường hợp kiểm thử

Phân hệ	Tổng số TC	Đạt	Lỗi	Tỷ lệ Đạt
Xác thực (Auth)	15	14	1	93.3%
Tác phẩm (Artworks)	35	35	0	100%
Thanh toán (Payments)	25	25	0	100%
Tìm kiếm (Search AI)	25	24	1	96.0%
Tương tác (Social)	25	25	0	100%
Tổng cộng	125	123	2	98.4%

Phân tích nguyên nhân các trường hợp lỗi (FAIL):

- **TC09 (Lỗi đồng bộ Avatar lần đầu):** Khi người dùng sử dụng tài khoản

Google/Facebook có ảnh đại diện gốc ở độ phân giải quá cao, tiến trình tải và nén ảnh đại diện của hệ thống nhận về qua Webhook đôi lúc vượt quá thời gian phản hồi API (Timeout 5000ms). Giải pháp khắc phục: Chuyển luồng đồng bộ Avatar sang dạng chạy ngầm bất đồng bộ thông qua BullMQ thay vì chờ xử lý trực tiếp trên API.

- **TC14 (Độ chính xác của Vector Search):** Chức năng tìm kiếm ảnh tương đồng bằng mô hình AI gặp khó khăn khi ảnh đầu vào là ảnh phác thảo có chất lượng thấp, thiếu chi tiết màu sắc, dẫn đến điểm số Cosine Similarity với kho ảnh gốc không đạt ngưỡng yêu cầu. Giải pháp khắc phục: Cần tích hợp thêm bộ trọng số của mô hình chuyên biệt cho việc nhận diện nét vẽ phác thảo, thay vì dùng chung mô hình nhận dạng ảnh thông thường.

4.5 Triển khai hệ thống

4.5.1 Mô hình triển khai tổng thể

Hệ thống được triển khai theo mô hình Container hóa trên môi trường điện toán đám mây nhằm đảm bảo tính đồng nhất giữa môi trường phát triển và môi trường thực tế. Quá trình triển khai thực tế được thực hiện trên máy chủ ảo cung cấp bởi DigitalOcean.

Để đảm bảo an toàn và tối ưu hóa tốc độ tải trang, tên miền của hệ thống được đăng ký thông qua nhà cung cấp Name.com và được quản lý DNS bởi nền tảng Cloudflare. Hệ thống tận dụng gói dịch vụ miễn phí của Cloudflare để cung cấp chứng chỉ bảo mật TLS/SSL (HTTPS), bộ nhớ đệm phân phối nội dung (CDN), và đặc biệt là lớp tường lửa bảo vệ hệ thống khỏi các cuộc tấn công từ chối dịch vụ phân tán.

Toàn bộ các luồng truy cập từ người dùng sẽ đi qua hệ thống mạng biên của Cloudflare trước khi được định tuyến đến địa chỉ IP của máy chủ VPS trên DigitalOcean. Tại máy chủ, Nginx đóng vai trò là Reverse Proxy tiếp nhận toàn bộ các yêu cầu ở cổng 80, sau đó phân luồng lưu lượng đến ứng dụng Frontend (Next.js) hoặc Backend API (NestJS) tương ứng.

4.5.2 Cấu trúc dịch vụ và cấu hình

Hệ thống sử dụng công cụ Docker Compose bằng tệp docker-compose.prod.yml để điều phối và quản lý vòng đời của các dịch vụ. Môi trường triển khai bao gồm các container hoạt động biệt lập nhưng kết nối với nhau thông qua mạng nội bộ Docker (gr-network):

- Nginx (gr-nginx): Đóng vai trò Reverse Proxy, định tuyến luồng dữ liệu và che giấu cấu trúc mạng nội bộ.

- Frontend (gr-frontend): Container chứa ứng dụng giao diện Next.js, được biên dịch ở chế độ Production.
- Backend (gr-backend): Container chứa lõi xử lý nghiệp vụ NestJS, kết nối với các dịch vụ dữ liệu phía sau.
- PostgreSQL (gr-postgres): Hệ quản trị cơ sở dữ liệu chính, tích hợp sẵn extension `pgvector` cho tìm kiếm AI. Cổng của database không được mở ra mạng ngoài nhằm đảm bảo bảo mật.
- Redis (gr-redis): Dịch vụ bộ nhớ đệm và quản lý hàng đợi tác vụ xử lý nền BullMQ.
- Meilisearch (gr-meilisearch): Công cụ tìm kiếm văn bản toàn văn, phục vụ truy vấn tốc độ cao.
- MinIO (gr-minio): Máy chủ lưu trữ đối tượng tương thích S3, đảm nhiệm việc lưu trữ toàn bộ hình ảnh và tệp phương tiện tải lên.

Việc cô lập từng thành phần vào các container riêng biệt mang lại tính an toàn cao, đồng thời cho phép nâng cấp hoặc thay thế từng dịch vụ một cách độc lập trong tương lai mà không làm gián đoạn toàn bộ hệ thống.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Chương này tập trung trình bày các giải pháp kỹ thuật cốt lõi và những đóng góp nổi bật trong quá trình xây dựng hệ thống. Nội dung bao gồm việc ứng dụng kiến trúc nguyên khối phân rã kết hợp cơ chế hướng sự kiện để đảm bảo tính mở rộng, giải pháp nhận diện và ngăn chặn ảnh trùng lặp sử dụng mã băm nhận thức, cũng như cơ chế truy vấn dữ liệu hiệu suất cao kết hợp tìm kiếm ngữ nghĩa bằng AI. Qua đó, chương làm rõ khả năng giải quyết các thách thức kỹ thuật đặc thù của nền tảng chia sẻ nghệ thuật số.

5.1 Kiến trúc nguyên khối phân rã kết hợp hướng sự kiện

5.1.1 Đặt vấn đề

Trong giai đoạn đầu thiết kế hệ thống, một bài toán kiến trúc lớn được đặt ra: Lựa chọn giữa kiến trúc nguyên khối truyền thống Monolithic hay kiến trúc vi dịch vụ Microservices. Đối với một nền tảng chia sẻ ảnh trực tuyến với nhiều nghiệp vụ phức tạp đan chéo nhau như quản lý người dùng, xử lý hình ảnh, thanh toán gói hội viên và tương tác mạng xã hội, kiến trúc Monolithic truyền thống thường dễ dẫn đến tình trạng mã nguồn bị trói buộc chặt chẽ. Khi một tính năng thay đổi, nó kéo theo sự sụp đổ hoặc lỗi logic của hàng loạt tính năng khác, làm hệ thống ngày càng khó bảo trì.

Tuy nhiên, nếu áp dụng ngay kiến trúc Microservices, hệ thống sẽ đối mặt với độ trễ giao tiếp qua mạng, khó khăn trong việc duy trì tính toàn vẹn dữ liệu và tổn kém tài nguyên triển khai đối với một đội ngũ phát triển quy mô nhỏ. Do đó, hệ thống cần một kiến trúc lai: vừa dễ dàng triển khai trên một tiến trình duy nhất để tối ưu chi phí, vừa đảm bảo sự độc lập tuyệt đối giữa các miền nghiệp vụ để có thể dễ dàng tách ra thành các dịch vụ độc lập trong tương lai.

5.1.2 Giải pháp

Giải pháp được đưa ra là áp dụng kiến trúc nguyên khối phân rã Modular Monolith kết hợp với cơ chế hướng sự kiện Event-Driven Architecture. Chi tiết về cấu trúc phân rã của các mô-đun thành 5 miền chính đã được mô tả sơ bộ tại Chương 4.

Theo phương pháp thiết kế hướng tên miền Domain-Driven Design, mã nguồn được chia thành các mô-đun hoàn toàn độc lập với ranh giới rõ ràng. Các mô-đun này không gọi trực tiếp các phương thức của nhau nhằm tránh phụ thuộc cứng. Thay vào đó, hệ thống giao tiếp thông qua một kênh trung gian là Event Bus nội bộ.

Cụ thể, đối với bài toán tương tác với tác phẩm, khi một người dùng nhấn Thích một bức tranh, hành động này kéo theo rất nhiều tác vụ phụ như gửi thông báo cho tác giả, tăng điểm xếp hạng của bức tranh, và cập nhật thống kê sự tương tác. Thay vì nhúng toàn bộ dịch vụ Thông báo, Thống kê vào trong dịch vụ Lướt thích, điều này phá vỡ nguyên lý trách nhiệm đơn lẻ Single Responsibility, dịch vụ Lướt thích chỉ làm đúng một việc: Lưu dữ liệu lượt thích vào cơ sở dữ liệu và phát ra một sự kiện mang tên sự kiện Thích tác phẩm.

Các mô-đun khác đóng vai trò là những người lắng nghe. Khi bắt được sự kiện, dịch vụ Thông báo sẽ tự động tạo thông báo, và dịch vụ Thống kê sẽ tự động tính toán lại bảng xếp hạng xu hướng. Thông qua Event Bus, luồng dữ liệu được luân chuyển mượt mà mà không tạo ra bất kỳ sự dính kết trực tiếp nào giữa nơi phát và nơi nhận sự kiện.

Việc áp dụng giải pháp này biến hệ thống thành một cỗ máy gồm các bánh răng hoạt động độc lập nhưng ăn khớp với nhau qua các thông điệp sự kiện, mang lại tính trừu tượng hóa cực kỳ cao.

5.1.3 Kết quả đạt được

Kết quả rõ rệt nhất là chỉ số phụ thuộc vòng trong hệ thống giảm xuống mức 0. Mã nguồn trở nên cực kỳ linh hoạt và dễ thực hiện kiểm thử tự động vì khi kiểm thử một mô-đun, lập trình viên không cần giả lập hàng chục dịch vụ phụ thuộc xung quanh. Hơn nữa, với kiến trúc này, việc bổ sung thêm một tính năng mới, chẳng hạn như tự động gửi email chúc mừng khi bức tranh đạt mốc 1000 lượt thích, diễn ra hoàn toàn trong suốt với hệ thống hiện tại; người lập trình chỉ cần tạo thêm một bộ lắng nghe mới cho sự kiện mà không phải sửa đổi bất kỳ dòng mã nào của quy trình cốt lõi, tuân thủ tuyệt đối nguyên lý Đóng-Mở Open/Closed Principle trong triết lý kỹ nghệ phần mềm hiện đại.

5.2 Hệ thống tìm kiếm ngữ nghĩa đa phương thức

5.2.1 Đặt vấn đề

Nghệ thuật là một lĩnh vực mang tính trừu tượng cực kỳ cao. Khi người dùng muốn tìm kiếm một bức tranh trên hệ thống, họ hiếm khi nhớ tên chính xác của tác giả hay tiêu đề tác phẩm. Thông thường, họ tìm kiếm dựa trên cảm nhận, phong cách, hoặc mô tả khung cảnh, ví dụ: "một bức tranh buồn về cơn mưa", "cô gái đứng dưới bầu trời sao", hay "phong cách nghệ thuật số tương lai".

Các hệ thống cơ sở dữ liệu quan hệ truyền thống chỉ hỗ trợ tìm kiếm dựa trên từ khóa thông qua kỹ thuật khớp chuỗi hoặc đối sánh văn bản. Giải pháp này hoàn toàn bất lực trước những truy vấn trừu tượng, bởi vì từ khóa người dùng nhập vào

có thể không xuất hiện trong tiêu đề hay mô tả của bức tranh, mặc dù nội dung bức tranh thực sự phản ánh điều đó. Vấn đề cốt lõi đặt ra là phải xây dựng được một công cụ tìm kiếm có khả năng "hiểu" được ngữ nghĩa sâu xa của câu lệnh truy vấn và ánh xạ nó với ý nghĩa của hình ảnh.

5.2.2 Giải pháp

Để giải quyết triệt để rào cản này, giải pháp được đề xuất là tích hợp cơ chế tìm kiếm ngữ nghĩa Semantic Search thông qua việc kết hợp Trí tuệ Nhân tạo và Cơ sở dữ liệu Vector.

Thay vì lưu trữ thông tin dưới dạng văn bản thông thường, hệ thống sử dụng mô hình ngôn ngữ hình ảnh CLIP cục bộ để chuyển đổi toàn bộ tiêu đề, mô tả và nhãn của bức tranh thành một vector biểu diễn trong không gian 512 chiều. Vector này đóng vai trò là biểu diễn toán học của ngữ nghĩa. Những khái niệm có ý nghĩa tương đồng như buồn bã và cô đơn sẽ có các vector nằm rất gần nhau trong tọa độ không gian nhiều chiều.

Khi người dùng thực hiện một truy vấn tìm kiếm bằng văn bản tự nhiên, câu truy vấn đó lập tức được gửi qua API để mã hóa thành một vector truy vấn duy nhất. Quy trình tiếp theo là đưa vector này vào cơ sở dữ liệu PostgreSQL thông qua tiện ích mở rộng pgvector để thực hiện phép toán Khoảng cách Cosine giữa vector câu lệnh và hàng triệu vector tác phẩm đang lưu trữ. Từ đó, hệ thống tìm ra các bức tranh mang ý nghĩa không gian sát với câu truy vấn nhất.

Để tối ưu hóa hiệu năng và chi phí trong kiến trúc tìm kiếm này, một hệ thống bộ nhớ đệm tốc độ cao trên Redis được thiết lập. Trước khi gọi API để mã hóa câu lệnh, hệ thống kiểm tra xem câu truy vấn này đã từng được phân tích trước đó chưa. Điều này giảm thiểu đáng kể quá trình tính toán lại và độ trễ mạng.

5.2.3 Kết quả đạt được

Giải pháp đã tạo ra một bước nhảy vọt về trải nghiệm khám phá nội dung. Hệ thống có khả năng truy xuất chính xác các bức tranh dựa trên ý niệm của người dùng dù không hề trùng khớp bất cứ một từ vựng nào. Nhờ khả năng lưu trữ của bộ nhớ đệm Redis, thời gian phản hồi cho một truy vấn AI phức tạp nhất được tối ưu hóa đáng kể, mang lại trải nghiệm khám phá mượt mà cho người dùng. Tính năng đột phá này đã nâng tầm ứng dụng từ một kho lưu trữ ảnh thông thường trở thành một nền tảng khám phá nghệ thuật thông minh và thấu hiểu người dùng.

5.3 Bảo vệ bản quyền nghệ thuật bằng thuật toán băm nhận thức pHash

5.3.1 Đặt vấn đề

Một trong những vấn nạn nhức nhối nhất của các nền tảng chia sẻ nghệ thuật trực tuyến là tình trạng ăn cắp bản quyền và đăng tải lại trái phép tác phẩm của họa sĩ khác nhằm mục đích câu lượt xem hoặc lôi kéo người đăng ký gói. Nếu giải quyết bài toán phát hiện ảnh trùng lặp bằng các hàm băm mật mã truyền thống như MD5 hay SHA-256, hệ thống sẽ gặp thất bại hoàn toàn trong môi trường thực tế.

Nguyên lý của các hàm băm mật mã là tính chất nhạy cảm với sự thay đổi, được biết đến như hiệu ứng thác đổ Avalanche effect - chỉ cần một điểm ảnh thay đổi, đổi độ sáng, thu nhỏ kích thước hay nén sang định dạng khác, mã băm sẽ bị biến đổi hoàn toàn sang một chuỗi khác biệt. Kẻ gian có thể dễ dàng đánh lừa bộ lọc chống sao chép bằng thao tác cắt ảnh vô cùng đơn giản hoặc chèn thêm một dấu chìm thủy vân siêu nhỏ.

5.3.2 Giải pháp

Nhằm cung cấp cơ chế bảo vệ tính toàn vẹn và quyền sở hữu trí tuệ mạnh mẽ cho các họa sĩ sáng tạo trên nền tảng, đồ án đã áp dụng thuật toán băm nhận thức Perceptual Hashing, hay gọi tắt là pHash, tích hợp thẳng vào luồng tải ảnh của hệ thống.

Trái ngược hoàn toàn với mã băm mật mã vốn chỉ quan tâm tới chuỗi byte, thuật toán pHash phân tích dựa trên đặc trưng hình ảnh trực quan mà mắt người có thể cảm nhận. Quy trình sinh mã băm pHash được hệ thống thực hiện qua các bước kỹ thuật như sau: 1. Giảm kích thước ảnh về một kích thước siêu nhỏ, chuẩn hóa ở mức 32×32 pixel, để loại bỏ toàn bộ các chi tiết nhiễu. 2. Chuyển đổi màu sắc sang thang độ xám để loại trừ thông tin màu, chỉ giữ lại độ chói và hình khối. 3. Chạy thuật toán biến đổi Cosine rời rạc DCT để trích xuất các dải tần số thấp đại diện cho cấu trúc chính của bức ảnh và loại bỏ dải tần số cao. 4. Tính toán mức trung bình và tạo ra một chuỗi nhị phân 64-bit đóng vai trò như dấu vân tay kỹ thuật số đặc trưng cho bức ảnh.

Đặc điểm ưu việt của thuật toán tạo chuỗi nhị phân này là nếu hai bức ảnh giống nhau về nội dung trực quan nhưng khác nhau về tỷ lệ nén, kích thước hoặc độ sáng, chúng vẫn tạo ra hai mã băm cực kỳ tương đồng. Để phát hiện trùng lặp trong luồng đăng tải, sau khi hệ thống trích xuất được pHash của ảnh mới, nó sẽ thực hiện phép đo khoảng cách Hamming với toàn bộ tập mã pHash đang có trong cơ sở dữ liệu. Khoảng cách Hamming được tính đơn giản bằng số lượng bit khác biệt giữa hai chuỗi nhị phân bằng phép toán XOR. Nếu khoảng cách này nhỏ hơn một ngưỡng an toàn, chẳng hạn như nhỏ hơn 5 bit khác biệt, hệ thống sẽ ngay lập tức đánh cờ

cảnh báo đây là một phiên bản sao chép trái phép.

5.3.3 Kết quả đạt được

Với giải pháp bấm nhận thức, nền tảng sở hữu một lá chắn bảo vệ tác quyền tự động vô cùng vững chắc. Thuật toán có khả năng phát hiện chính xác các biến thể ảnh bị cắt xén, thu phóng độ phân giải, hay thay đổi định dạng đuôi ảnh. Tốc độ so sánh hai chuỗi 64-bit sử dụng thao tác bit là cực kỳ tối ưu, tiêu tốn lượng CPU không đáng kể, cho phép hệ thống duy trì hiệu năng tốt kể cả khi mở rộng và soát trên cơ sở dữ liệu hàng triệu bức ảnh. Giải pháp này đã trở thành một luận điểm then chốt để xây dựng niềm tin nơi cộng đồng họa sĩ chuyên nghiệp.

5.4 Tối ưu hiệu năng bằng cơ chế xử lý nền bất đồng bộ

5.4.1 Đặt vấn đề

Nhiệm vụ cốt lõi của hệ thống xoay quanh việc lưu trữ và xử lý tệp tin đa phương tiện có dung lượng lớn. Một kịch bản phổ biến thường xuyên diễn ra: Người dùng tải lên một bức tranh gốc dung lượng 20 Megabyte. Để đưa bức tranh này ra phục vụ cộng đồng, hệ thống máy chủ bắt buộc phải thực hiện một chuỗi thao tác cực kỳ nặng nề bao gồm đẩy tệp thô lên nền tảng đám mây, nén hình ảnh, thay đổi kích thước tạo các bản thu nhỏ, tính toán mã băm chống sao chép, kiểm duyệt nội dung nhạy cảm và trích xuất vector ngữ nghĩa.

Nếu áp dụng phương pháp lập trình tuần tự đồng bộ, tiến trình của máy chủ sẽ bị treo đứng trong khi chờ đợi băng thông mạng và năng lực tính toán CPU xử lý toàn bộ các thao tác trên. Hậu quả là người dùng sẽ phải chờ đợi màn hình tải vòng tròn từ 10 đến 30 giây mới nhận được thông báo thành công. Về phía hạ tầng, trải nghiệm này dễ dẫn đến đứt gãy kết nối mạng giữa chừng, và nếu có khoảng mười người dùng cùng thao tác tải ảnh, toàn bộ tài nguyên máy chủ có thể bị kiệt quệ ngay lập tức.

5.4.2 Giải pháp

Để giải quyết tận gốc bài toán thắt cổ chai hiệu năng này, kiến trúc đã được thiết kế lại theo hướng phi đồng bộ, áp dụng mô hình xử lý công việc chạy nền Background Job Processing dựa trên công cụ hàng đợi BullMQ kết hợp với cơ sở dữ liệu Redis.

Quy trình được điều phối lại hoàn toàn. Khi yêu cầu tải ảnh được gửi đến máy chủ thông qua giao thức HTTP, tầng điều khiển sẽ lưu thông tin tác phẩm vào cơ sở dữ liệu với trạng thái Đang xử lý, đồng thời tải tệp tin gốc lên hệ thống lưu trữ đám mây MinIO một cách đồng bộ. Ngay sau khi tệp gốc được lưu trữ an toàn, máy chủ sẽ đóng gói phần việc còn lại vào hàng đợi tin nhắn Message Queue và trả về trạng

thái phản hồi thành công cho người dùng, thông báo rằng tác phẩm đang trong quá trình phân tích. Trải nghiệm tải trang của người dùng kết thúc tại đây một cách vô cùng mượt mà.

Ở hậu trường sâu bên trong kiến trúc hệ thống, các tiến trình chạy nền, hoàn toàn độc lập với luồng phục vụ API chính, liên tục kiểm tra và rút các công việc ra khỏi hàng đợi BullMQ để thực thi. Hệ thống hàng đợi này cung cấp cơ chế kiểm soát tiến độ vô cùng mạnh mẽ, bao gồm khả năng theo dõi trạng thái, và đặc biệt là cơ chế tự động thử lại theo cấp số nhân Exponential Backoff Retry. Nếu việc phân tích tạm thời thất bại do gián đoạn kết nối mạng tạm thời, tiến trình nền sẽ không hủy bỏ công việc mà sẽ tạm nghỉ và tự động thử lại sau 1 giây, rồi 2 giây, đảm bảo tính bền bỉ của hệ thống. Khi quá trình xử lý hoàn tất, tiến trình nền sẽ cập nhật trạng thái bức tranh thành Xuất bản, hoặc chuyển sang trạng thái Cần xử lý và Thất bại nếu có lỗi phát sinh.

5.4.3 Kết quả đạt được

Việc áp dụng kiến trúc hàng đợi xử lý nền bất đồng bộ là một trong những đóng góp mang lại hiệu quả kỹ thuật ấn tượng nhất cho dự án:

- Hệ thống triệt tiêu hoàn toàn hiện tượng thắt cổ chai tải trọng. Thời gian phản hồi API tải ảnh được rút ngắn tối đa do máy chủ được giải phóng khỏi các tác vụ phân tích nặng, giúp ứng dụng trên trình duyệt web phản ứng nhanh chóng mà không bị treo.
- Độ chịu tải và khả năng đáp ứng đồng thời được gia tăng gấp nhiều lần. Máy chủ phục vụ API không bao giờ bị quá tải I/O. Dòng chảy dữ liệu nặng nề được điều tiết một cách tự nhiên thông qua nút thắt của hàng đợi.
- Cơ chế tự động thử lại khi gặp lỗi giúp kiến trúc hệ thống vượt qua các gián đoạn mạng tạm thời, đảm bảo tính toàn vẹn dữ liệu tuyệt đối và không làm thất thoát công sức của họa sĩ trong quá trình sáng tạo.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Đồ án đặt ra mục tiêu xây dựng một nền tảng chia sẻ ảnh trực tuyến phục vụ cộng đồng nghệ thuật số, trong đó tích hợp đồng thời sáu nhóm chức năng thường bị phân tán trên các hệ thống khác nhau: đăng tải và xử lý ảnh, tìm kiếm thông minh, tương tác cộng đồng, kiểm duyệt nội dung, bảo vệ bản quyền và cơ chế hội viên có thanh toán. Sau quá trình phân tích, thiết kế và triển khai, phần này tổng kết các kết quả đạt được, các hạn chế còn tồn tại và các bài học kinh nghiệm rút ra.

6.1.1 Kết quả đạt được và so sánh với các sản phẩm tương tự

Như đã phân tích trong Chương 2, các nền tảng lớn hiện có trên thị trường đều được tối ưu hóa xuất sắc cho những nghiệp vụ cốt lõi riêng biệt. Các mạng xã hội phổ thông (như Pinterest) có hạ tầng tương tác khổng lồ nhưng không tối ưu cho quy trình xuất bản nghệ thuật chuyên sâu. Các cộng đồng nghệ thuật (như ArtStation, Pixiv) sở hữu lượng dữ liệu lớn và cộng đồng mạnh mẽ, nhưng việc tích hợp các mô hình AI tìm kiếm ngữ nghĩa hay phát hiện bản quyền vẫn đang trong quá trình phát triển hoặc giới hạn ở một số chức năng. Các nền tảng tài trợ (như Patreon) cung cấp công cụ tạo thu nhập mạnh mẽ nhưng lại hoàn toàn tách biệt khỏi môi trường khám phá tác phẩm của cộng đồng.

Thay vì cạnh tranh về quy mô hạ tầng hay khối lượng dữ liệu với các hệ thống lớn, đồ án tập trung giải quyết bài toán **tích hợp và chống phân mảnh trải nghiệm**. Kết quả đạt được là một giải pháp nguyên mẫu (prototype) kết nối liền mạch quy trình từ lúc đăng tải, khám phá tác phẩm bằng AI, cho đến việc đăng ký hội viên trả phí trên cùng một giao diện duy nhất.

Cụ thể, trong suốt quá trình thực hiện đồ án, các nội dung sau đã được hoàn thành:

- **Về mặt kiến trúc phần mềm:** Thiết kế và triển khai kiến trúc Modular Monolith kết hợp Event-Driven Architecture trên nền tảng NestJS. Hệ thống bao gồm hơn 20 module nghiệp vụ được phân rã thành 5 nhóm miền (Domain) với ranh giới rõ ràng, giao tiếp qua Event Bus nội bộ. Chỉ số phụ thuộc vòng (Circular Dependency) được duy trì ở mức 0, đảm bảo tính linh hoạt và khả năng mở rộng.
- **Về mặt giao diện người dùng:** Xây dựng ứng dụng Frontend bằng Next.js 14 với cơ chế Server-Side Rendering, tích hợp thư viện giao diện Mantine 7 để tạo trải nghiệm trực quan. Giao diện hỗ trợ đầy đủ các luồng chức năng từ

khám phá, đăng tải, chi tiết tác phẩm, tương tác cộng đồng đến quản lý hội viên và trang quản trị (Admin Dashboard).

- **Về mặt xử lý dữ liệu:** Thiết kế cơ sở dữ liệu quan hệ gồm 26 bảng trên PostgreSQL, sử dụng Prisma ORM để chuẩn hóa kiểu dữ liệu từ mức thiết kế tới mã nguồn. Có tích hợp extension pgvector để lưu trữ và truy vấn vector đa chiều phục vụ tìm kiếm ngữ nghĩa.
- **Về các đóng góp kỹ thuật nổi bật:**
 - Xây dựng luồng tìm kiếm ngữ nghĩa đa phương thức sử dụng mô hình CLIP cục bộ kết hợp pgvector, cho phép tìm kiếm tác phẩm bằng ngôn ngữ tự nhiên hoặc ảnh phác thảo với thời gian phản hồi thấp (dưới 400 mili giây).
 - Ứng dụng thuật toán Perceptual Hashing (pHash) vào quy trình kiểm tra bản quyền tự động, phát hiện các biến thể ảnh bị cắt xén, thu phóng hoặc thay đổi định dạng ở mức độ cơ bản.
 - Triển khai cơ chế xử lý nền bất đồng bộ thông qua BullMQ, giảm thiểu đáng kể thời gian chặn (blocking) của luồng yêu cầu tải ảnh chính.
 - Tích hợp cổng thanh toán Stripe với cơ chế Webhook bất đồng bộ để tự động hóa vòng đời của gói hội viên (đăng ký, gia hạn, cấp quyền truy cập).
- **Về quy mô hệ thống:** Tổng mã nguồn đạt khoảng 35.000 dòng lệnh TypeScript, gồm khoảng 145 component và module. Toàn bộ môi trường được đóng gói bằng Docker Compose với các dịch vụ hỗ trợ như PostgreSQL (pgvector), Redis, Meilisearch và MinIO.

6.1.2 Các hạn chế còn tồn tại

Mặc dù hệ thống đã đáp ứng được các yêu cầu thiết kế ban đầu, giải pháp vẫn còn một số hạn chế cần được nhìn nhận khách quan:

- **Mô hình AI tìm kiếm chưa được tinh chỉnh theo miền:** Mô hình CLIP hiện tại được sử dụng trực tiếp nguyên bản (pre-trained) mà chưa tinh chỉnh (fine-tuning) trên bộ dữ liệu nghệ thuật chuyên biệt. Điều này có thể làm giảm độ chính xác tìm kiếm đối với các phong cách nghệ thuật đặc thù như tranh thủy mặc, pixel art hoặc nghệ thuật trừu tượng.
- **Phát hiện trùng lặp pHash có giới hạn:** Thuật toán pHash hoạt động hiệu quả với các biến thể ảnh đơn giản (thu phóng, nén, đổi định dạng), nhưng chưa phát hiện được các trường hợp sao chép tinh vi hơn như lật ngược ảnh

theo trục, ghép nhiều ảnh hoặc vẽ chép lại dựa trên bố cục gốc.

- **Chưa triển khai kiểm thử tải thực tế quy mô lớn:** Hệ thống đã được kiểm thử chức năng đầy đủ, tuy nhiên chưa có các kịch bản kiểm thử tải (load testing) với lưu lượng hàng nghìn người dùng đồng thời để xác định ngưỡng chịu lỗi thực sự của kiến trúc.
- **Cơ chế khiếu nại bản quyền chưa hoàn thiện:** Hiện tại hệ thống tập trung vào việc tự động đánh cờ (flag) ảnh trùng lặp trong lúc đăng tải, nhưng chưa xây dựng một quy trình xử lý tranh chấp và khiếu nại chính thức (tương tự quy trình DMCA) để hỗ trợ tác giả gốc yêu cầu gỡ bỏ nội dung.
- **Hệ thống khuyến nghị còn ở mức cơ bản:** Tính năng gợi ý tác phẩm tương tự hiện chỉ dựa trên độ tương đồng vector và thống kê tương tác thuần túy, chưa tích hợp được các mô hình học máy chuyên sâu về hành vi người dùng (collaborative filtering) để cá nhân hóa kết quả.

6.1.3 Bài học kinh nghiệm

Quá trình thực hiện đồ án đã đem lại những bài học thực tiễn quan trọng:

- **Tầm quan trọng của thiết kế kiến trúc phân rã:** Việc đầu tư thiết kế kiến trúc Modular Monolith ngay từ giai đoạn đầu đã giúp đơn giản hóa việc bảo trì. Khi bổ sung các tính năng phụ trợ (như thông báo, thống kê), hệ thống chỉ cần đăng ký thêm các bộ lắng nghe sự kiện (Event Listener) mà không can thiệp sâu vào logic của các module cốt lõi.
- **Xử lý bất đồng bộ là chìa khóa cho hệ thống đa phương tiện:** Trong các tác vụ tiêu tốn nhiều I/O và CPU như xử lý ảnh hay gọi mô hình AI, việc tách luồng thông qua Message Queue (BullMQ và Redis) là điều kiện bắt buộc để đảm bảo API luôn phản hồi mượt mà cho người dùng cuối.
- **Khả năng tích hợp mô hình AI cục bộ:** Triển khai mô hình CLIP trực tiếp trên máy chủ bằng tiến trình con (Child Process) cho thấy tính khả thi trong việc xây dựng các tính năng thông minh mà vẫn làm chủ được chi phí hạ tầng, bảo mật dữ liệu và không phụ thuộc vào API của bên thứ ba.

6.2 Hướng phát triển

Dựa trên những hạn chế đã nêu và xu hướng công nghệ, một số định hướng phát triển sau được đề xuất nhằm tiếp tục hoàn thiện hệ thống:

6.2.1 Hoàn thiện các chức năng hiện có

Nâng cao chất lượng kiểm duyệt nội dung. Thay vì chỉ phụ thuộc vào mô hình kiểm duyệt (như NSFW.js) có sẵn, hệ thống có thể được tiến hành thu thập dữ liệu

và huấn luyện lại (fine-tuning) để phân biệt tốt hơn giữa hình ảnh vi phạm và các tác phẩm mang tính nghệ thuật (ví dụ: tranh khỏa thân nghệ thuật hoặc giải phẫu học). Đồng thời, bổ sung cơ chế kiểm duyệt nhiều cấp (multi-level review) với hàng đợi ưu tiên để hỗ trợ quản trị viên xử lý các trường hợp cảnh báo nguy cơ cao.

Xây dựng quy trình khiếu nại bản quyền hoàn chỉnh. Hoàn thiện luồng nghiệp vụ tiếp nhận khiếu nại bản quyền (counter-notification) từ người dùng. Khi có tranh chấp, hệ thống tự động trích xuất các báo cáo đối chiếu mã pHash và lịch sử đăng tải để hỗ trợ quản trị viên đưa ra quyết định xử lý, tuân thủ các tiêu chuẩn tương tự mô hình bảo vệ bản quyền DMCA.

Tối ưu tìm kiếm vector cho cơ sở dữ liệu lớn. Trong trường hợp lượng dữ liệu hình ảnh tăng vọt, hiệu năng của câu truy vấn vector có thể giảm sút. Hướng giải quyết là chuyển đổi từ tìm kiếm tuyến tính sang áp dụng các chỉ mục xấp xỉ như HNSW (Hierarchical Navigable Small World) trên pgvector với tham số được tinh chỉnh, hoặc tích hợp các hệ quản trị cơ sở dữ liệu vector chuyên biệt (như Qdrant, Milvus) để đáp ứng quy mô công nghiệp.

6.2.2 Các hướng phát triển mới

Cải tiến hệ thống khuyến nghị bằng mô hình lai (Hybrid Recommendation). Phát triển tính năng gợi ý tác phẩm bằng cách kết hợp Collaborative Filtering (phân tích hành vi tương tác của cộng đồng như lượt xem, thích, lưu bộ sưu tập) và Content-Based Filtering (dựa trên đặc trưng hình ảnh và vector ngữ nghĩa). Sự kết hợp này sẽ giúp cá nhân hóa sâu sắc trải nghiệm khám phá của mỗi người dùng dựa trên sở thích thực sự của họ.

Tích hợp tìm kiếm đa phương thức nâng cao. Bổ sung khả năng tìm kiếm bằng ảnh gốc (Image-to-Image Search) bên cạnh văn bản và ảnh phác thảo. Hướng đi này cho phép người dùng tải lên một bức ảnh bất kỳ để hệ thống truy xuất các tác phẩm có phong cách, bố cục hoặc bảng màu tương đồng nhất trong nền tảng.

Triển khai hạ tầng phân tán và giám sát quy mô lớn. Chuẩn bị cho việc gia tăng lưu lượng người dùng bằng cách ảo hóa toàn diện các dịch vụ và triển khai trên nền tảng điều phối container (Kubernetes). Áp dụng cơ chế tự động mở rộng (Auto-scaling) cho các Worker xử lý ảnh, đồng thời xây dựng hệ thống giám sát thời gian thực (Prometheus, Grafana) để theo dõi sức khỏe hệ thống. Nhờ cấu trúc Modular Monolith đã phân định rõ ranh giới, các module nặng về tính toán hoàn toàn có thể được bóc tách thành các dịch vụ Microservices độc lập khi cần thiết.

TÀI LIỆU THAM KHẢO

- [1] Kamil Myśliwiec and others, *Nestjs - a progressive node.js framework*, [Truy cập ngày 11-06-2026], 2026. [Online]. Available: <https://docs.nestjs.com/>
- [2] Vercel Inc., *Next.js - the react framework for the web*, [Truy cập ngày 11-06-2026], 2026. [Online]. Available: <https://nextjs.org/docs>
- [3] The PostgreSQL Global Development Group, *Postgresql: The world's most advanced open source relational database*, [Truy cập ngày 11-06-2026], 2026. [Online]. Available: <https://www.postgresql.org/docs/>
- [4] Prisma Data, Inc., *Prisma - next-generation node.js and typescript orm*, [Truy cập ngày 11-06-2026], 2026. [Online]. Available: <https://www.prisma.io/docs>
- [5] Meilisearch, *Meilisearch documentation - lightning-fast search engine*, [Truy cập ngày 11-06-2026], 2026. [Online]. Available: <https://www.meilisearch.com/docs>
- [6] A. Radford et al., “Learning transferable visual models from natural language supervision,” in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021, pp. 8748–8763.
- [7] Andrew Kane, *Pgvector: Open-source vector similarity search for postgresql*, [Truy cập ngày 11-06-2026], 2026. [Online]. Available: <https://github.com/pgvector/pgvector>
- [8] Taskforce.sh Inc., *Bullmq - premium message queue for node.js based on redis*, [Truy cập ngày 11-06-2026], 2026. [Online]. Available: <https://docs.bullmq.io/>
- [9] Clerk, Inc., *Clerk: Authentication and user management for the modern web*, [Truy cập ngày 11-06-2026], 2026. [Online]. Available: <https://clerk.com/docs>
- [10] Stripe, Inc., *Stripe documentation - financial infrastructure platform*, [Truy cập ngày 11-06-2026], 2026. [Online]. Available: <https://docs.stripe.com/>